



mathematics



Article

From Theory to Application: A Practical Introduction to Neural Operators in Scientific Computing

Prashant K. Jha



<https://doi.org/10.3390/math14132421>

Article

From Theory to Application: A Practical Introduction to Neural Operators in Scientific Computing

Prashant K. Jha 

Department of Mechanical Engineering, South Dakota School of Mines and Technology,
Rapid City, SD 57701, USA; prashant.jha@sdsmt.edu or pjha.sci@gmail.com

Abstract

This review examines neural operator architectures for learning solution operators of parametric partial differential equations (PDEs), with an emphasis on conceptual clarity and practical implementation. The work analyzes key models, including DeepONet, PCANet, and the Fourier Neural Operator, highlighting their underlying representations, computational structures, and comparative performance. These architectures are demonstrated on three canonical PDE problems: the Poisson equation, a linear elasticity problem, and a hyperelasticity problem. To make the presentation self-contained, key foundational topics are introduced, including finite-dimensional representations of function spaces, singular-value decomposition, and sampling from infinite-dimensional function spaces. Beyond forward modeling, the review discusses the use of neural operators as surrogate models within a Bayesian inverse-problem framework, including prior specification, forward-map approximation, and posterior computation. The performance of the three neural-operator architectures is evaluated on in-distribution samples, out-of-distribution samples, and Bayesian inference tasks. The review also discusses challenges related to prediction accuracy and generalization, outlining emerging strategies such as residual-based error correction and multi-level training. The review concludes by positioning neural operators within broader scientific-computing workflows and by identifying directions for reliable, scalable operator learning.

Keywords: neural operators; neural networks; operator learning; surrogate modeling; Bayesian inference

MSC: 62M45; 68T07



Academic Editors: Luigi Rodino and
Silvestru Sever Dragomir

Received: 5 May 2026

Revised: 9 June 2026

Accepted: 2 July 2026

Published: 6 July 2026

Copyright: © 2026 by the author.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Neural operators have emerged as powerful tools for approximating solution operators of parametric partial differential equations (PDEs). Their key advantage lies in learning mappings between infinite-dimensional function spaces, enabling efficient surrogate modeling for complex systems. In addition, their ability to provide rapid evaluations makes them particularly attractive for applications such as optimization, control, and inverse problems.

This article provides a practical and hands-on introduction to several neural operator architectures that have become central to scientific computing. The following neural operators are considered:

1. Deep Operator Network (DeepONet) [1,2];
2. Principal Component Analysis/Proper Orthogonal Decomposition-based Neural Operator (PCANet/PODNet) [3,4];

3. Fourier Neural Operator (FNO) [5,6].

The presentation is designed to be self-contained and implementation-focused. Neural operator architectures, variants, and applications have been surveyed from broader taxonomic and benchmarking perspectives in several recent review articles [6–8]. In contrast, the present article focuses on conceptual understanding, practical implementation of representative neural operator architectures, and the application of neural operator surrogates to Bayesian inverse problems, where these surrogates are used to accelerate repeated forward evaluations in MCMC simulations. To support this objective, several key topics are addressed:

- Sampling random functions using Gaussian measures on function spaces;
- Construction of data representations tailored to different neural operator architectures;
- Algorithms and Python implementations for essential components, including random function sampling and Markov chain Monte Carlo (MCMC) methods for Bayesian inverse problems;
- Application of neural operators as surrogate models in Bayesian inverse problems.

To illustrate these ideas, two classical linear parametric PDEs and one nonlinear parametric PDE are used as model problems. The first is the Poisson equation for temperature distribution on a rectangular domain, where the input parameter field is the diffusivity. The second is a two-dimensional linear elasticity problem, where the input parameter field is Young's modulus. The third is a hyperelasticity problem that serves as a nonlinear counterpart of the second problem. For these problems, neural operators are constructed to approximate the solution operator, and their accuracy is assessed on random samples of parameter fields.

In addition, neural operators are employed as surrogate models in Bayesian inverse problems, where the underlying parameter fields are inferred from observational data. The resulting posterior estimates are compared against those obtained using the true forward model, demonstrating that neural operators can provide accurate and computationally efficient approximations in this setting. The model problems considered here exhibit rapid decay of singular values in both input and output representations, indicating a low-dimensional structure that is favorable for operator learning. This is reflected in the high predictive accuracy of the neural operators when test samples are drawn from the same distribution as the training data and when the prior for Bayesian inference is well-aligned with the training distribution. However, when applying the trained neural operators to out-of-distribution samples, such as those with higher frequencies or larger magnitudes than the training data, significant errors can arise. This is shown through performance assessments on out-of-distribution samples, where the neural operators exhibit large prediction errors.

For complex settings, such as highly nonlinear PDEs or challenging inference and optimization problems, neural operators may exhibit significant errors. This limitation motivates the need for improved strategies for controlling prediction accuracy. These issues are discussed in the concluding section, where existing approaches for error control are surveyed. The discussion also highlights emerging directions, including foundation models for PDEs that aim to construct surrogate operators capable of generalizing across families of PDEs, as well as related frameworks such as multi-task and universal neural operators.

Organization of the Article

- Section 2 introduces notation and key mathematical preliminaries, including finite-dimensional approximations of function spaces, singular value decomposition, and sampling of random functions using Gaussian measures.
- Section 3 presents the parametric PDEs used as model problems.

- Section 4 discusses the architectures and implementations of DeepONet, PCANet, and FNO.
- Section 5 presents the application of neural operators to Bayesian inverse problems, where they are used as surrogate models in MCMC simulations.
- Section 6 evaluates the performance of the neural operators for in-distribution prediction, out-of-distribution prediction, and Bayesian inference.
- Section 7 collects selected recent work in the field, offers a perspective on important open issues, and concludes the article. The subsection on prediction accuracy (Section 7.2) reviews related work and identifies future research directions, while Section 7.3 discusses emerging approaches based on foundation models for PDEs. Final remarks are provided in Section 7.4.
- Appendices A and B present additional results on neural operator predictions and Bayesian inference, respectively.

Code and Jupyter notebooks are available at: https://github.com/CEADpx/neural_operators/ (accessed on 7 June 2026; tag `survey26_v2`), with an accompanying Zenodo repository archive [9]. The dataset is shared separately in the Dropbox folder: `NeuralOperator_Survey_Shared_Data_June2026` (<https://www.dropbox.com/scl/fo/co5v2bozvr5y8uv5kc29y/ACKiT1sBBQCTKV2wZYcAIII?rlkey=agt87l1tf89g967gf8ofe5nik&st=3no4j03k&dl=0>, accessed on 1 July 2026).

2. Preliminaries

This section establishes the notation and core mathematical framework used throughout the article. It introduces finite-dimensional representations of function spaces, sampling strategies for random functions, and related concepts that are central to the formulation and implementation of neural operators.

2.1. Notations

Let $\mathbb{N}, \mathbb{Z}, \mathbb{R}$ denote the spaces of natural numbers, integers, and real numbers, respectively, and let $\mathbb{R}^+$ denote the space of all non-negative real numbers. $\mathbb{R}^n$ denotes the n -dimensional Euclidean space. The space of square-integrable functions $f : D \subset \mathbb{R}^q \rightarrow \mathbb{R}^d$ is denoted by $L^2(D; \mathbb{R}^d)$. $H^s(D; \mathbb{R}^d)$ denotes the space of functions in $L^2(D; \mathbb{R}^d)$ whose weak derivatives up to order s belong to $L^2(D; \mathbb{R}^d)$. $L(M; U)$ denotes the space of continuous linear maps from M to U , and $C^1(A; U)$ denotes the space of continuously differentiable maps from $A \subset M$ to U . Given a generic complete normed (function) Banach space A , $\|\cdot\|$ denotes the norm, and, if it is a Hilbert space, (u, v) denotes the inner product for $u, v \in A$. A^* denotes the dual of the Banach space A , and $\langle b, a \rangle$ denotes the duality pairing between $b \in A^*$ and $a \in A$.

Throughout the text, $m \in M$ will denote the input or parameter field in the parametric boundary value problem, M the appropriate Banach function space for input fields, and $u \in U$ the solution field, where U is the Banach space for solutions of the PDE. The solution operator or forward operator mapping the parameter field $m \in M$ to the PDE solution $u \in U$ is denoted by $F(m)$. The neural operator approximation of the forward operator is denoted by $F_{\text{NOP}}(m)$. The finite-dimensional approximations of functions $m \in M$ and $u \in U$ are denoted by the Roman letters m and u ; more generally, for any $a \in A$, the approximation is denoted by a . The forward map between finite-dimensional approximations is denoted by F . The projections of m and u onto lower-dimensional subspaces are denoted by $\tilde{m}$ and $\tilde{u}$, respectively. The mapping between lower-dimensional subspaces will be denoted by placing a tilde over the corresponding symbol, e.g., $\tilde{F}$.

For vectors $a, b \in \mathbb{R}^d$, $a \cdot b = \sum_{i=1}^d a_i b_i$ denotes the dot product. For second-order tensors $A, B \in \mathbb{R}^{d \times d}$, $\text{tr}(A) = \sum_{i=1}^d A_{ii}$ is the trace, A^T is the transpose,

$A : B := \text{tr}(A^T B) = \sum_{i,j} A_{ij} B_{ij}$ is the tensor inner product, and $\det(A)$ is the determinant of A .

The key notations used throughout the text are collected in Table 1 below.

Table 1. Key notations.

Symbol	Description
m	Typical parameter field in the parametric PDEs. This is also the input function to the neural operator
u	Typical solution of the PDE given m . This is also the output of the neural operator
D_A	Domain of a generic function $a \in A$
$\partial D, \Gamma_b$	Domain boundary and subset of ∂D tagged by b , respectively
q_A	Dimension of the domain of function a
d_A	Dimension of the pointwise values of the function a
$x \in D_M$	A point in the domain of input/parameter functions
$y \in D_U$	A point in the domain of output/solution functions
$M \subset \{m : D_M \subseteq \mathbb{R}^{q_M} \rightarrow \mathbb{R}^{d_M}\}$	Function space of the parameters in the parametric PDEs
$U \subset \{u : D_U \subseteq \mathbb{R}^{q_U} \rightarrow \mathbb{R}^{d_U}\}$	Function space of solutions of the PDE
$F : M \rightarrow U$	Forward solution operator, which is also the target operator of neural operators
$\phi^A = \{\phi_i^A\}_{i=1}^\infty \subset A$	Basis functions for the function space A ; an element $a \in A$ has a representation $a = \sum_i a_i \phi_i^A$, where $\{a_i\}$ are the coefficients
$\mathbf{a} \in \mathbb{R}^{p_A}$	Coefficient vector for the finite-dimensional representation of the function a
p_A	Dimension of the finite-dimensional representation of $a \in A$
$\mathbf{a}_i$	i th component of a vector $\mathbf{a}$
$\mathbf{F} : \mathbb{R}^{p_M} \rightarrow \mathbb{R}^{p_U}$	Finite-dimensional approximation of the operator $F : M \rightarrow U$
r_A	Dimension of the reduced representation of $\mathbf{a} \in \mathbb{R}^{p_A}$, $r_A < p_A$
$\tilde{\mathbf{a}} \in \mathbb{R}^{r_A}$	The reduced-dimensional representation of $\mathbf{a} \in \mathbb{R}^{p_A}$
$\tilde{\mathbf{P}}_A : \mathbb{R}^{p_A} \rightarrow \mathbb{R}^{r_A}$	Projection operator that takes $\mathbf{a} \in \mathbb{R}^{p_A}$ to the reduced space $\tilde{\mathbf{a}} = \tilde{\mathbf{P}}_A(\mathbf{a}) \in \mathbb{R}^{r_A}$
$\tilde{\mathbf{F}} : \mathbb{R}^{r_M} \rightarrow \mathbb{R}^{r_U}$	Reduced-order approximation of $\mathbf{F}$ mapping between reduced-dimensional (latent) spaces of input and output functions
$N(\bar{w}, C)$	Gaussian random field with mean $\bar{w} \in W$ and covariance operator $C : W \rightarrow W$
$N(w, C)$	Gaussian random field in the finite-dimensional space $\mathbb{R}^{p_W}$, where $w \in \mathbb{R}^{p_W}$ and $C \in \mathbb{R}^{p_W \times p_W}$
$F_{NOp} : M \times \mathbb{R}^{p_\Theta} \rightarrow U$	Neural operator approximation of $F : M \rightarrow U$; p_Θ is the number of trainable parameters in the neural operator

Table 1. Cont.

Symbol	Description
$F_{NOP} : \mathbb{R}^{p_M} \times \mathbb{R}^{p_\Theta} \rightarrow \mathbb{R}^{p_U}$	Neural operator approximation of finite-dimensional representation of the map $F : \mathbb{R}^{p_M} \rightarrow \mathbb{R}^{p_U}$
N	Number of samples used for training and testing neural operators
$m^I, u^I, \mathbf{m}^I, \mathbf{u}^I$	I th input–output sample in function and finite-dimensional spaces
$\mathbf{X}^I, \mathbf{Y}^I$	I th sample, $1 \leq I \leq N$, of the data from $\mathbf{X}, \mathbf{Y}$, respectively
$\mathbf{g} \in \mathbb{R}^{d_g}$	Observed data, where d_g is the dimension of the data
$\bar{\mathbf{B}} : U \rightarrow \mathbb{R}^{d_g}$	State-to-observable map, which maps the output function u to the observed data $\mathbf{g}$
$\mathbf{B} : W \rightarrow \mathbb{R}^{d_g}$	Parameter-to-observable map, which maps the parameter field to be inferred, $w \in W$, to the predicted data $\mathbf{g} = \mathbf{B}(w)$
$\eta \sim \mathcal{N}(0, \Gamma_g)$	Gaussian noise with zero mean and covariance Γ_g . Example: $\Gamma_g = \sigma_g^2 \mathbf{I}$, where σ_g^2 is the variance and $\mathbf{I}$ is the identity matrix
$\Phi(w), \pi_{\text{like}}(\mathbf{g} w)$	Likelihood potential of w and the likelihood of $\mathbf{g}$ given w
μ^o, μ^g	Prior and posterior measures on W , respectively

2.2. Series Representation of Functions and Finite-Dimensional Approximation

One of the central ideas that many neural operators leverage is the finite-dimensional representation of functions in terms of coefficients and basis functions in their respective spaces. Following the notations in the previous section and Table 1, suppose that M and U are Hilbert spaces and, therefore, have orthonormal bases $\phi^M = \{\phi_i^M\}_{i=1}^\infty \subset M$ and $\phi^U = \{\phi_i^U\}_{i=1}^\infty \subset U$. Then a function $m \in M$ can be represented as

$$m(x) = \sum_{i=1}^{\infty} m_i \phi_i^M(x), \quad \forall x \in D_M, \quad (1)$$

where $m_i = (m, \phi_i^M) \in \mathbb{R}$ (inner product between m and ϕ_i^M) are the coefficients or degrees of freedom associated with the i th mode. Note that, for each i , $\phi_i^M(x) \in \mathbb{R}^{d_M}$ for $x \in D_M$, and $\phi_i^U(y) \in \mathbb{R}^{d_U}$ for $y \in D_U$.

It is useful to consider the example where the domain is the unit interval $D_M = (0, 1)$ and $m \in L^2(D_M; \mathbb{R})$. In this case, one can write $m(x) = \sum_{i=1}^{\infty} m_i \phi_i^M(x)$, where the basis functions take the form

$$\begin{aligned} \phi_1^M &= 1, \quad \phi_2^M = \sqrt{2} \cos(2\pi x), \quad \phi_3^M = \sqrt{2} \sin(2\pi x), \dots, \\ \phi_{2j}^M &= \sqrt{2} \cos(2j\pi x), \quad \phi_{2j+1}^M = \sqrt{2} \sin(2j\pi x), \dots \end{aligned} \quad (2)$$

and the coefficients are given by

$$m_i = (m, \phi_i^M) = \int_0^1 m(x) \phi_i^M(x) dx, \quad \forall i. \quad (3)$$

Focusing on the abstract setting, let $\phi^M = \{\phi_i^M\}_{i=1}^{p_M}$, where p_M is a finite integer and ϕ^M is a finite collection of basis functions, and let $\{m_i\}_{i=1}^{p_M}$ be the corresponding

coefficients. Then, the finite-dimensional approximation is $\sum_{i=1}^{p_M} m_i \phi_i^M(x) \approx m(x)$ with the error given by $\left\| m - \left(\sum_{i=1}^{p_M} m_i \phi_i^M \right) \right\|$. Similarly, the function $u \in U$ can be approximated as $u(y) \approx \sum_{i=1}^{p_U} u_i \phi_i^U(y)$, $y \in D_U$. Neural operators often leverage this finite-dimensional representation viewpoint, where the goal is to determine, or learn, the basis functions $\phi^U = \{\phi_i^U\}_{i=1}^{p_U}$ (or their pointwise values $\{\phi_i^U(y)\}_{i=1}^{p_U}$ for $y \in D_U$ [e.g., in DeepONet]) and the coefficients $u = \{u_i\}_{i=1}^{p_U}$ such that $\sum_{i=1}^{p_U} u_i \phi_i^U$ provides the “best” approximation of $u = F(m)$, where $m \in M$ is the input function to the operator.

Remark 1. PCANet explicitly constructs reduced-dimensional representations of the input and output functions via singular value decomposition and learns the map between them (see Section 2.3). FNO follows a different construction, utilizing Fourier transforms to represent functions in the frequency domain and learning mappings through spectral convolutions applied to the Fourier modes.

Finite Element Approximation

For a general class of function spaces $M \subseteq \{m : D_M \subseteq \mathbb{R}^{q_M} \rightarrow \mathbb{R}^{d_M}\}$ and spatial domain D_M , the preceding construction of finite-dimensional approximations can be restrictive. A more straightforward numerical way to obtain finite-dimensional approximations is to use numerical techniques such as finite difference and finite element methods. In this work, the finite element method is used (e.g., to generate samples of input functions using Gaussian priors and solve PDE-based problems).

To be more precise, consider a finite element discretization D_{M_h} of the domain D_M consisting of simplex elements $\{T_e\}_{e=1}^{N_e}$, so that $D_{M_h} = \cup_e T_e \approx D_M$. Suppose $\phi_i^M : D_{M_h} \rightarrow \mathbb{R}^{d_M}$ denotes the basis function associated with the i th scalar degree of freedom and $m \in \mathbb{R}^{p_M}$ the vector of coefficients associated with the basis functions $\{\phi_i^M\}_{i=1}^{p_M}$. Then, the finite-dimensional approximation of m is given by $m_h(x) = \sum_{i=1}^{p_M} m_i \phi_i^M(x)$ for $x \in D_{M_h}$. The same procedure can be used to construct the finite-dimensional approximation of u using the basis functions $\{\phi_i^U\}_{i=1}^{p_U}$ and coefficients $u \in \mathbb{R}^{p_U}$.

Remark 2. When m is vector-valued, that is, $m(x) \in \mathbb{R}^{d_M}$ with $d_M > 1$, the finite element approximation must represent each component of m independently while using a common spatial discretization. This leads naturally to vector-valued basis functions.

Let $\{\psi_j : D_{M_h} \rightarrow \mathbb{R}\}_{j=1}^n$ denote the standard scalar Lagrange basis functions associated with the n mesh vertices. Since each vertex carries d_M degrees of freedom, the total number of degrees of freedom is $p_M = n d_M$.

The vector-valued basis functions are constructed by combining the scalar spatial basis with the canonical basis of $\mathbb{R}^{d_M}$. Specifically, for each degree of freedom index i with $1 \leq i \leq p_M$, there exist unique indices $j \in \{1, \dots, n\}$ and $k \in \{1, \dots, d_M\}$ such that

$$i = (j - 1)d_M + k,$$

where j identifies the vertex in the mesh and k identifies the component. The corresponding basis function associated with this degree of freedom is defined as

$$\phi_i^M(x) = \psi_j(x) \mathbf{e}_k, \quad x \in D_{M_h},$$

where $\mathbf{e}_k$ is the k th canonical basis vector in $\mathbb{R}^{d_M}$, i.e., $\mathbf{e}_k = (0, \dots, 1, \dots, 0)^\top$ with the 1 in the k th position.

Let $V_{M_h} = \text{span}\{\phi_i^M\}_{i=1}^{p_M}$, where p_M is the number of degrees of freedom. Then, the function $m \in M$ can be approximated by a function $m_h \in V_{M_h}$ given by

$$m(x) \approx m_h(x) = \sum_{i=1}^{p_M} m_i \phi_i^M(x), \quad \forall x \in D_{M_h}, \quad (4)$$

provided the coefficients $\mathbf{m} \in \mathbb{R}^{p_M}$ are chosen appropriately. For example, $\mathbf{m}$ is selected such that it minimizes the L^2 -error $\|m - m_h\|_{L^2}$.

Given finite-dimensional approximations m_h and u_h of m and u , respectively, the map $F(m) = u$ is also approximated by $F_h(m_h) = u_h$ in the sense that F_h takes m_h and returns the output u_h such that the operator error

$$\|F(m) - F_h(m_h)\|$$

is small for the relevant class of inputs m .

Finally, note that, for a fixed mesh and the basis functions ϕ_i^M , it is easy to see that, if $\mathbf{m} \in \mathbb{R}^{p_M}$ is fixed, then the function m_h is completely characterized. If m_h is fixed, using the unique representation of m_h , the coefficients $\mathbf{m}$ are completely characterized. Thus, V_{M_h} can be identified using $\mathbb{R}^{p_M}$ (and vice versa). This makes it possible to represent the finite-dimensional function space V_{M_h} by the Euclidean space $\mathbb{R}^{p_M}$ of the coefficients. Throughout the article, functions m and u will be represented by the finite-dimensional approximations $\mathbf{m}$ and $\mathbf{u}$, where the corresponding functional representations, $m_h = \sum_{i=1}^{p_M} m_i \phi_i^M$ and $u_h = \sum_{i=1}^{p_U} u_i \phi_i^U$, are assumed implicitly. In the same spirit, using $u_h = F_h(m_h)$, the operator F_h can be identified with a map $F: \mathbb{R}^{p_M} \rightarrow \mathbb{R}^{p_U}$:

$$F(\mathbf{m}) = \mathbf{u} \quad \Rightarrow \quad F_h(m_h) = u_h = \sum_{i=1}^{p_U} u_i \phi_i^U \quad \text{with} \quad m_h = \sum_{i=1}^{p_M} m_i \phi_i^M. \quad (5)$$

Here, F maps the coefficient vector $\mathbf{m}$ to $\mathbf{u}$ and is induced by the map F_h .

2.3. Dimensional Reduction and Singular-Value Decomposition (SVD)

While the theory is based on functions defined on a continuum domain, computer implementations introduce discretizations of the domain and, consequently, discrete approximations of functions. For example, the training data for neural operators is typically a collection of pairs $(\mathbf{m}^I, \mathbf{u}^I)$, where $\mathbf{m}^I \in \mathbb{R}^{p_M}$ and $\mathbf{u}^I = F(\mathbf{m}^I) \in \mathbb{R}^{p_U}$ are discrete approximations of functions in M and U , where F is the finite-dimensional mapping between $\mathbb{R}^{p_M}$ and $\mathbb{R}^{p_U}$ that approximates the operator of interest F . Generally speaking, the dimensions of the discretized input and target functions, p_M and p_U , are large, and the problem of approximating the map F between high-dimensional spaces becomes challenging and potentially ill-posed.

The second key idea used in various neural operators, after the linear basis representation discussed earlier, is to reduce the dimensions of discretized input and output functions; see [3]. If $\mathbf{m} \in \mathbb{R}^{p_M}$ and $\mathbf{u} \in \mathbb{R}^{p_U}$, and the goal is to determine a map $\mathbf{m} \mapsto \mathbf{u} = F(\mathbf{m})$ from the data $\{(\mathbf{m}^I, \mathbf{u}^I)\}_{I=1}^N$, then, as an alternative to learning or approximating the map F , one could attempt to characterize the map $\tilde{F}$, where

$$\mathbf{m} \mapsto \mathbf{u} = \tilde{P}_U^T (\tilde{F} (\tilde{P}_M(\mathbf{m}))). \quad (6)$$

Here, $\tilde{P}_M \in \mathbb{R}^{r_M \times p_M}$ is the projection operator that projects $\mathbf{m} \in \mathbb{R}^{p_M}$ into a low-dimensional subspace, $\tilde{\mathbf{m}} := \tilde{P}_M(\mathbf{m}) \in \mathbb{R}^{r_M}$ (with $r_M \ll p_M$). $\tilde{P}_U \in \mathbb{R}^{r_U \times p_U}$ has the same role as $\tilde{P}_M$ but for target functions $\mathbf{u} \in \mathbb{R}^{p_U}$. The transpose of $\tilde{P}_U$, $\tilde{P}_U^T$, projects an element in $\mathbb{R}^{r_U}$ to $\mathbb{R}^{p_U}$. Note that $\tilde{F}: \mathbb{R}^{r_M} \rightarrow \mathbb{R}^{r_U}$, which needs to be learned, is the mapping between

two low-dimensional spaces, and hence identifying $\tilde{F}$ is less daunting than identifying F . In summary, using $\tilde{P}_M$ and $\tilde{P}_U$, the dimensions of the operator inference problem are significantly reduced, and, by controlling r_M and r_U , one can balance the trade-off between accuracy and computational cost.

Projectors via SVD

The projectors $\tilde{P}_M$ and $\tilde{P}_U$ used for dimensional reduction can be obtained via singular-value decomposition (SVD). Focusing on the input space $\mathbb{R}^{p_M}$, let R denote a $p_M \times N$ matrix such that, for $1 \leq I \leq N$, m^I makes up the I th column of the matrix R . Here, N may be smaller or larger than p_M , and since the samples may be linearly dependent, the rank of R need not be equal to $\min\{p_M, N\}$. Let

$$r = \text{rank}(R) \leq \min\{p_M, N\}. \quad (7)$$

Consider the compact SVD

$$R = UDV^T, \quad (8)$$

where $U \in \mathbb{R}^{p_M \times r}$ and $V \in \mathbb{R}^{N \times r}$ have orthonormal columns, and $D \in \mathbb{R}^{r \times r}$ is diagonal with positive singular values. Denote the columns of U by $\{w^I\}_{I=1}^r$, i.e.,

$$U = \begin{bmatrix} | & | & \dots & | \\ w^1 & w^2 & \dots & w^r \\ | & | & \dots & | \end{bmatrix}. \quad (9)$$

The vectors $\{w^I\}_{I=1}^r$ form an orthonormal basis for the column space of R , which is an r -dimensional subspace of $\mathbb{R}^{p_M}$.

Let $r_M > 0$ be the prescribed reduced dimension, with $r_M \leq r$. Given r_M , define $U_{r_M} \in \mathbb{R}^{p_M \times r_M}$ by retaining the first r_M columns of U :

$$U_{r_M} = \begin{bmatrix} | & | & \dots & | \\ w^1 & w^2 & \dots & w^{r_M} \\ | & | & \dots & | \end{bmatrix}. \quad (10)$$

Noting the properties of U_{r_M} (e.g., see [10] (Section 3.2.1)), $U_{r_M}^T$ is taken as the projector, i.e.,

$$\tilde{P}_M := U_{r_M}^T. \quad (11)$$

Remark 3. The rank of R depends on the linear independence of the sampled coefficient vectors $\{m^I\}_{I=1}^N$. If these vectors are linearly dependent, then $\text{rank}(R) < \min\{p_M, N\}$. In contrast, if the samples are drawn independently from a distribution on $\mathbb{R}^{p_M}$ that is not supported on a proper linear subspace, then $\text{rank}(R) = \min\{p_M, N\}$. Numerically, the effective rank is determined by the decay of the singular values and the tolerance used to distinguish nonzero singular values.

For u , the projector $\tilde{P}_U$ is obtained following the same procedure as above using a matrix R of size $p_U \times N$ with u^I making its I th column. The left singular-vector matrix from the SVD of R , denoted by U , is truncated by retaining the first r_U columns of U . If the truncated matrix is U_{r_U} , then $\tilde{P}_U := U_{r_U}^T$.

In Figure 1, the normalized singular values of representative centered and normalized input and output data are shown. The grey, brown, and cadet blue dots on each curve represent the singular value at mode 100, and the mode with a normalized singular value of 0.1 and 0.01. Based on the plot, projecting u onto SVD subspaces of dimensions $r_U = 10, 49$, and 100 gives approximate projection errors of 10%, 1%, and 0.3%, respectively, relative

to the largest singular value. Similarly, for m , projection into reduced dimension $r_M = 35$, 100, and 290 will result in average projection errors of approximately 10%, 3.3%, and 1%, respectively.

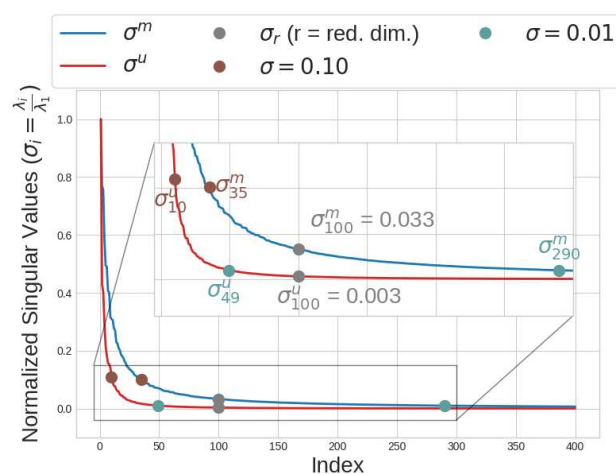


Figure 1. Normalized singular values of the centered and normalized input and output data $\{m^I = \exp(w^I) \mid w^I \sim N(0, C)\}$ and $\{u^I = F(m^I)\}$, where $N(0, C)$ is the p_M -dimensional Gaussian distribution obtained from the finite element discretization of $N(0, C)$ with $C = L_\Delta^{-2}$; see Section 3.1 for details. Here, m^I denotes a discretized input and F denotes the discretized solution operator associated with the PDE. σ^a , $a \in \{m, u\}$, represents the normalized singular values. Small dots indicate the modes at which the normalized singular value is 0.1 or 0.01, as well as at specified reduced dimension $r = 100$ (100th mode).

2.4. Sampling Functions as Infinite-Dimensional Random Variables

The final topic in this section is the sampling of random parameters, which are functions. Consider a probability space $(\Omega, \mathcal{F}, \mathbb{P})$, where Ω is a sample space and $\mathcal{F}$ is a σ -algebra on which the probability measure $\mathbb{P}$ is defined with $\mathbb{P}(\Omega) = 1$. The goal is to draw W -valued random fields, where W is assumed to be a separable Hilbert space. Equivalently, the goal is to define a random field $\mathcal{Z} : \Omega \rightarrow W$ such that, for a given $z \in \Omega$, $w = \mathcal{Z}(z) \in W$. Suppose such a $\mathcal{Z}$ is available; then, the probability that $\mathcal{Z}$ takes values in a subset $A \subseteq W$ defines the pushforward measure $\mu_{\mathcal{Z}}$, given by

$$\mu_{\mathcal{Z}}(A) = \text{probability of } \mathcal{Z} \in A = \mathbb{P}(\{z \in \Omega : \mathcal{Z}(z) \in A\}) = \mathbb{P}(\mathcal{Z}^{-1}(A)), \quad (12)$$

where $\mathcal{Z}^{-1}(A) \in \mathcal{F}$ is assumed to be measurable in the probability space $(\Omega, \mathcal{F}, \mathbb{P})$. Thus, $\mu_{\mathcal{Z}}$ is a measure on W induced by the random field $\mathcal{Z}$. Now, suppose that a random field $\mathcal{Z}$ is such that the measure $\mu_{\mathcal{Z}}$ is Gaussian in the sense of [11,12]; specifically, see [11] (Definition 6 and Lemma 23). In this case, $\mu_{\mathcal{Z}}$ is written as $\mu_{\mathcal{Z}} = N(\bar{w}, C)$. Here, $\bar{w} \in W$ is the mean function, and $C : W \rightarrow W$ is called the covariance operator. At the outset, C is assumed to be a trace-class operator ([11] (Lemma 23) and [12] (Theorem 7)).

Our next goal is to consider specific examples of $\mathcal{Z}$ such that $\mu_{\mathcal{Z}}$ is Gaussian, as mentioned above, and understand how random function samples are generated. In this direction, it is useful to highlight the role of C in generating samples $w = \mathcal{Z}(z) \in W$; this helps explain why commonly used forms of C are effective. Consider another random field $\mathcal{S} : \Omega \rightarrow W$ such that $\mu_{\mathcal{S}} = N(0, 1)$, where $0 \in W$ is the mean function and $1 : W \rightarrow W$ is the identity covariance operator. Given a function $\bar{w} \in W$ and a covariance operator $C : W \rightarrow W$, let $\mathcal{Z}$ be such that $\mu_{\mathcal{Z}} = N(\bar{w}, C)$. Then, a sample $w = \mathcal{Z}(z)$ can be obtained by transforming the sample $s = \mathcal{S}(z)$ as follows:

$$\mathcal{Z}(z) = w := \bar{w} + C^{1/2}s. \quad (13)$$

Thus, $C^{1/2}$, the square root of the covariance operator, plays a key role in generating random functions. It is therefore useful to choose C so that $C^{1/2}$ is computationally tractable while ensuring that the W -valued samples are well-defined, have the desired regularity, and exhibit controlled correlation between pointwise values.

Gaussian Measures Based on Laplacian-like Operators

Following [13], let $C = L_{\Delta}^{-2}$, where $L_{\Delta} : W_{L_{\Delta}} \subseteq W \rightarrow W$ is a Laplacian-like operator given by

$$L_{\Delta} := \begin{cases} -a_c \nabla \cdot b_c \nabla + c_c, & \text{in } D_W, \\ a_c n \cdot b_c \nabla, & \text{on } \partial D_W. \end{cases} \quad (14)$$

Here, D_W is the domain of functions w , and a_c, b_c, c_c are parameters in the operator (they could vary over the domain or be taken as constants). In this work, a_c and c_c will be constant, and in some situations, b_c will be considered to be a spatially varying scalar-valued $L^2(D_W)$ function. In the literature, it is also common to take b_c as $\mathbb{R}^{q_W \times d_W} \times \mathbb{R}^{q_W \times d_W}$ -valued function allowing one to encode anisotropy and inhomogeneous behavior in the prior. In the above, $W_{L_{\Delta}} \subseteq W$ is the domain of the operator L_{Δ} such that $L_{\Delta}(w)$ is well-defined for all $w \in W_{L_{\Delta}}$. The natural choice is $W_{L_{\Delta}} = \{w \in W : \|w\|_{H^2} < \infty\}$. If L_{Δ} is defined through the bilinear form

$$(v, L_{\Delta} w) = \int_{D_W} [a_c b_c \nabla w \cdot \nabla v + c_c w v] dx, \quad (15)$$

then, the operator domain can be taken as

$$W_{L_{\Delta}} = W \cap H^1(D_W; \mathbb{R}^{d_W}), \quad (16)$$

and take $v, w \in W_{L_{\Delta}}$. In the bilinear form above, note that the boundary term in (14) cancels out the boundary term after integration by parts.

With this specific form of C , the sampling of functions $w \in W$ can be summarized as follows:

Sampling a Gaussian random field with covariance operator $C = L_{\Delta}^{-2}$

1. Fix $\bar{w} \in W$, $C = L_{\Delta}^{-2}$, and let $\mathcal{S} : \Omega \rightarrow W$ be a random field such that $\mu_{\mathcal{S}} = N(0, 1)$.
2. Draw a sample $s = \mathcal{S}(z)$, $z \in \Omega$.
3. Generate a new sample $w \sim N(\bar{w}, C)$ as follows:

$$\begin{aligned} w &= \bar{w} + C^{1/2}(s) \\ \Rightarrow w &= \bar{w} + L_{\Delta}^{-1}(s) \\ \Rightarrow w &= \bar{w} + v, \text{ where } v \text{ satisfies } s = L_{\Delta}(v) \\ \Rightarrow w &= \bar{w} + v, \text{ where } v \text{ satisfies } (g, s) = (g, L_{\Delta}(v)), \quad \forall g \in W_{L_{\Delta}}, \\ \Rightarrow w &= \bar{w} + v, \text{ where } v \text{ satisfies} \\ &\int_{D_W} s g dx = \int_{D_W} [a_c b_c \nabla v \cdot \nabla g + c_c v g] dx, \quad \forall g \in W_{L_{\Delta}}, \end{aligned} \quad (17)$$

where the last two equations express the weak-form definition of L_{Δ} that can be solved using the finite element method.

In terms of the numerical implementation, consider a finite element mesh D_{W_h} and a finite element function space $V_h \subset W$ with basis functions $\phi^W = \{\phi_i^W\}_{i=1}^{p_W}$. Let the

mean function $\bar{w} \in W$ be projected onto V_h , with projection given by $\bar{w}_h = \sum_i \bar{w}_{h_i} \phi_i^W$. The procedure to sample $w_h \sim N(\bar{w}_h, C)$ is as follows:

Sampling a Gaussian random field with $C = L_\Delta^{-2}$ in a finite element setting

1. To draw a sample $s = \mathcal{S}(z) \in V_h$, consider the following steps:
 - (a) For each i , $1 \leq i \leq p_W$, draw a number $s_i \sim N(0, 1)$, where $N(0, 1)$ is the standard normal distribution on $\mathbb{R}$; and
 - (b) Take $s = \sum_i s_i \phi_i^W$, i.e., s is a finite element function with coefficients s .
2. Solve for $v_h \in V_h$ such that

$$\int_{D_{W_h}} s g \, dx = \int_{D_{W_h}} [a_c b_c \nabla v_h \cdot \nabla g + c_c v_h g] \, dx, \quad \forall g \in V_h. \quad (18)$$

This yields coefficients $v_h \in \mathbb{R}^{p_W}$ and the finite element function $v_h = \sum_i v_{h_i} \phi_i^W$.

3. A new sample $w_h \sim N(\bar{w}_h, C)$ is given by

$$w_h = \sum_i w_{h_i} \phi_i^W = \bar{w}_h + v_h, \quad \text{where} \quad w_{h_i} = \bar{w}_{h_i} + v_{h_i}. \quad (19)$$

Remark 4. Spectral methods, including Karhunen–Loève expansions, can also be used to sample Gaussian random fields with covariance operators defined through Laplacian-like operators. These methods are often efficient on simple domains with compatible boundary conditions, especially when the eigenpairs of the covariance operator or the associated differential operator are available. The finite element construction is used here because it is straightforward and directly compatible with the PDE discretizations used later for data generation and Bayesian inference. It also extends naturally to general geometries and boundary conditions, unstructured and higher-order meshes, spatially varying coefficients such as b_c , and vector-valued finite element spaces.

The above algorithm is implemented in Listing 11 using the Python library FEniCSx (version 0.10.0); see [14]. In Figure 2, results from a Gaussian sampler with parameters $a_c = 0.01$, $c_c = 0.2$, an inhomogeneous diffusivity function b_c , and zero mean $\bar{w} = 0 \in W$ are shown. This example of a prior is considered in the the notebook `PriorSampler.ipynb` (repository [9], tag `survey26_v2`).

Listing 1. Finite element implementation of Gaussian random field sampling with covariance $C = L_\Delta^{-2}$.

```
...
import numpy as np
import ufl
from dolfinx import default_scalar_type, fem
from dolfinx.fem.petsc import assemble_matrix, assemble_vector, create_vector
...

class PriorSampler:

    def __init__(self, V, a, c, seed=0):
        ...
        # function space
        self.V = V

        # vertex to dof vector and dof vector to vertex maps
        self.V_vec2vv, self.V_vv2vec = build_vector_vertex_maps(self.V)
        ...
        self.a_form = self.a*self.b_fn\
```

Listing 1. Cont.

```

        *ufl.inner(ufl.grad(self.u_trial), \
                    ufl.grad(self.u_test))*ufl.dx \
        + self.c*self.u_trial*self.u_test*ufl.dx
self.L_form = self.s_fn*self.u_test*ufl.dx
...

def __call__(self, m = None):
    # forcing term
    self.s_fn.x.array[:] = np.random.normal(0.0, 1.0, self.s_dim)
    ...
    # assemble rhs only
    self._assemble_rhs()

    # solve
    self.u_fn.x.array[:] = 0.0
    self._solve(self.u_fn)

    # add mean
    self.u_fn.x.petsc_vec.axy(1.0, self.mean_fn.x.petsc_vec)

    # vertex_dof ordered
    self.u = self.u_fn.x.array[self.V_vec2vv].copy()

    log_prior = self._log_prior_from_source()
    ...
    if m is not None:
        m = self.u.copy()
        return m, log_prior
    else:
        return self.u.copy(), log_prior

```

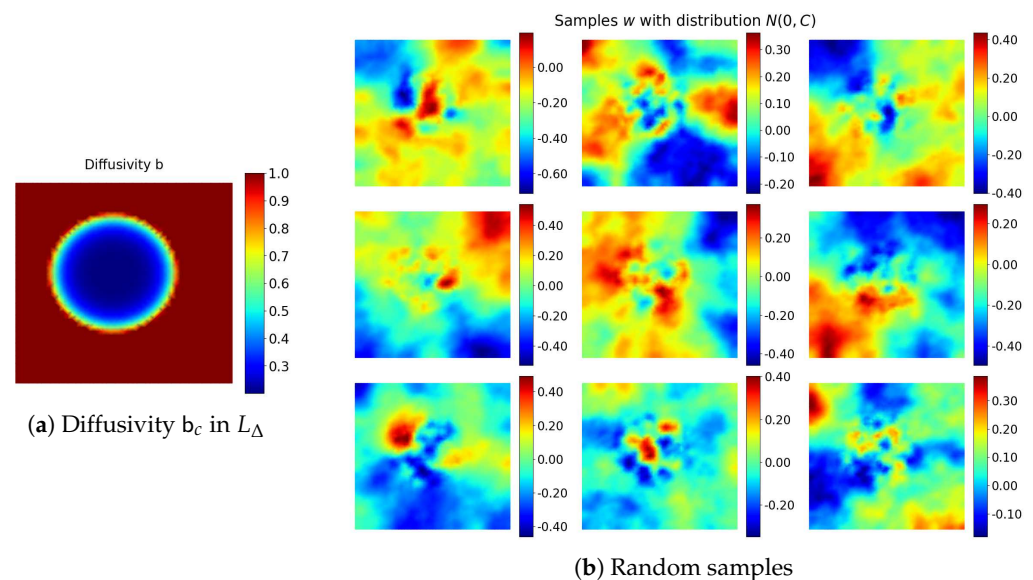


Figure 2. Random samples w generated using a Gaussian measure based on the Laplacian-like operator L_Δ defined in (14).

3. Model Problems

Neural operators will be discussed in the context of solving three PDE problems. These are presented in the following subsections.

3.1. Poisson Problem

Consider a two-dimensional domain given by a rectangle $D_U = (0, L_1) \times (0, L_2) \subset \mathbb{R}^2$ and suppose $u : D_U \rightarrow \mathbb{R}$ denotes the temperature field. The governing equation based on the balance of energy reads:

$$\begin{aligned} -\nabla \cdot (m(x) \nabla u(x)) &= f(x), & \forall x \in D_U, \\ u(x) &= 0, & \forall x \in \Gamma_{u_d}, \\ m(x) \nabla u(x) \cdot n(x) &= q(x), & \forall x \in \Gamma_{u_q} = \partial D_U - \Gamma_{u_d}, \end{aligned} \quad (20)$$

where $\Gamma_{u_d} := \{x \in \partial D_U : x_1 < L_1\}$ (i.e., all sides except the right side of the rectangular domain) and $\Gamma_{u_q} := \partial D_U - \Gamma_{u_d}$. Here, $f(x)$ is the specified heat source ($\text{J}/\text{m}^3/\text{s}$), $m(x)$ is the diffusivity ($\text{J}/\text{K}/\text{m}/\text{s}$), $n(x)$ is the unit outward normal, and $q(x)$ is the specified heat flux ($\text{J}/\text{m}^2/\text{s}$). To be consistent with the notations in Section 2.1, here $D_M = D_W = D_U$ (domains of functions m , w (w to be introduced shortly), and u), $q_W = q_M = q_U = 2$ (dimensions of the domains of functions), and $d_M = d_W = d_U = 1$ (dimensions of the pointwise values of functions). Working in the variational setting, the appropriate function spaces for diffusivity and temperature are as follows:

$$M := \{m : D_M \rightarrow \mathbb{R}^+ : \|m\|_{L^2} < \infty\}, \quad U := \{u \in H^1(D_U; \mathbb{R}) : u = 0 \text{ on } \Gamma_{u_d}\}. \quad (21)$$

The weak form of the problem is as follows: given $m \in M$, find $u \in U$ such that

$$\int_{D_U} m \nabla u \cdot \nabla v \, dx = \int_{D_U} f v \, dx + \int_{\Gamma_{u_q}} q v \, ds, \quad \forall v \in U. \quad (22)$$

The diffusivity field m is assumed to be the unknown and uncertain parameter field. To ensure that m is a positive function (diffusivity cannot be zero or negative), consider a random field $\mathcal{Z} : \Omega \rightarrow W := L^2(D_W; \mathbb{R})$ (with $D_W = D_M = D_U$) and suppose the associated measure $\mu_{\mathcal{Z}}$ on W is Gaussian $N(0, C)$ (0 mean and C covariance operator defined in Section 2.4 with parameters a_c, b_c, c_c). Now, given a sample $w = \mathcal{Z}(z) \in W$, sample $m = \mathcal{Q}(z)$ is computed as follows:

$$\mathcal{Q}(z) = m = \alpha_m \exp(w) + \beta_m, \quad \text{where } z \in \Omega, w = \mathcal{Z}(z), \text{ and } \mu_{\mathcal{Z}} = N(0, C). \quad (23)$$

Here, α_m and β_m are constants. Thus, m is a positive function and comes from a log-normal distribution. The generation of samples w is discussed in Section 2.4, and given w , computing m using the formula above is straightforward; see `transform_gaussian_pointwise()` in Listing 2.

Finally, given $m \in M$, let $F(m) = u \in U$ be the solution of boundary value problem (BVP) (20). This $F : M \rightarrow U$ is the solution/forward operator and target of neural operator learning.

Setup Details and Data Generation

Let $L_1 = L_2 = 1$, and consider the following f and q :

$$f(x) = 1000(1 - x_2)x_2(1 - x_1)^2 \quad \text{and} \quad q(x) = 50 \sin(5\pi x_2). \quad (24)$$

The covariance operator in $\mu_{\mathcal{Z}} = N(0, C)$ is taken as $C = L_{\Delta}^{-2}$, where L_{Δ} is defined in (14). The parameters in L_{Δ} and the transformation of w into m are

$$a_c = 0.005, \quad b_c = 1, \quad c_c = 0.2, \quad \alpha_m = 1, \quad \beta_m = 0. \quad (25)$$

Data for neural operator training is generated using finite element discretization with triangle elements and linear interpolation for both input and output functions. The number of elements is 5000, the number of nodes is $N_{nodes} = 2601$, and $p_M = p_U = N_{nodes} = 2601$. Listing 2 details the setup and solution of the Poisson model problem (20), building on the abstract class `PDEModel` shown in Listing 3. The sampling of uncertain parameter $m = \alpha_m \exp(w) + \beta_m \sim \mu_Q$, where $w \sim \mu_Z = N(0, C)$, is straightforward using the method and implementation of sampling $w \sim \mu_Z$ discussed in Section 2.4 and Listing 11. In Figure 3, samples of w and corresponding (m, u) pairs are shown. The notebook `Poisson.ipynb` (repository [9], tag `survey26_v2`) implements methods to generate and post-process data for neural operator training.

Samples $w \sim N(0, C)$ and corresponding $m = \alpha_m \exp(w) + \beta_m$ and $u = F(m)$

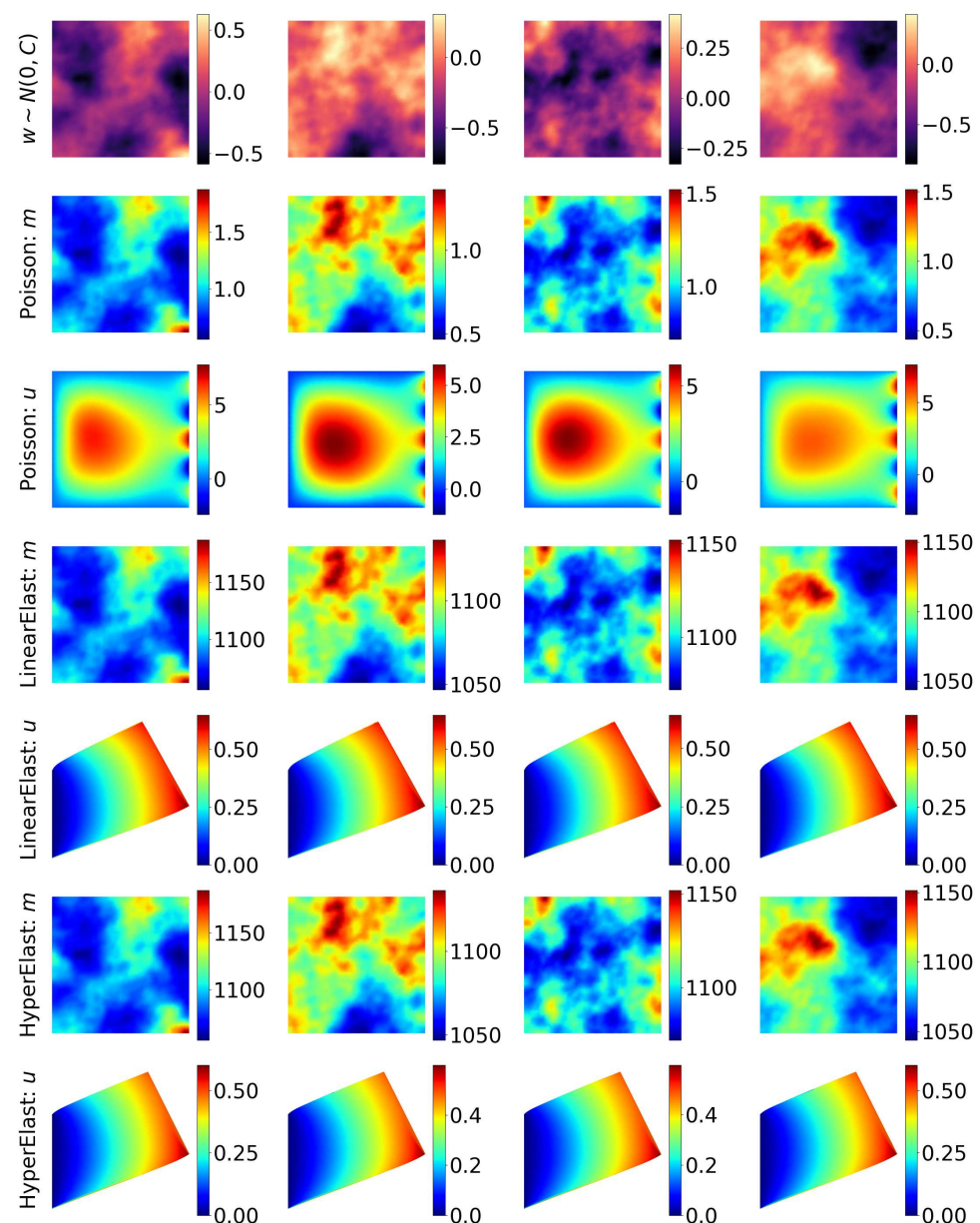


Figure 3. Some representative data samples for the three problems. For the linear elasticity and hyperelasticity problems, the displacement (PDE solution) u is visualized by plotting the deformed configuration of the domain with coordinates $z = x + u(x)$ for $x \in D_U$. The color map indicates the displacement magnitude $|u(x)|$.

Listing 2. Key components of the Poisson model implementation.

```

...
import ufl
from dolfinx import default_scalar_type, fem
from dolfinx.fem.petsc import (
    apply_lifting,
    assemble_matrix,
    ...
)
from pdeModel import PDEModel
...

class PoissonModel(PDEModel):
    def __init__(self, Vm, Vu, prior_sampler, logn_scale=1., logn_translate=0.,
                 seed=0):
        super().__init__(Vm, Vu, prior_sampler, seed)
        ...
        # External heat source and boundary flux
        self.f_expr = 1000 * (1 - x[1]) * x[1] * (1 - x[0]) * (1 - x[0])
        self.q_expr = 50 * ufl.sin(5 * ufl.pi * x[1])
        ...
        # variational form
        self.a_form = self.m_fn \
            * ufl.inner(ufl.grad(self.u_trial), ufl.grad(self.u_test)) *
dx
        self.L_form = self.f_expr * self.u_test * dx + self.q_expr * self.u_test *
ds
        ...
        # Dirichlet boundary condition
        ...
        dofs = fem.locate_dofs_geometrical(self.Vu, self._dirichlet_boundary)
        self.bc = [fem.dirichletbc(default_scalar_type(0.0), dofs, self.Vu)]
        ...
    ...

    def transform_gaussian_pointwise(self, w, m_local=None):
        if m_local is None:
            self.m_transformed = self.logn_scale * np.exp(w) + self.logn_translate
            return self.m_transformed.copy()
        return self.logn_scale * np.exp(w) + self.logn_translate

    def compute_mean(self, m):
        return self.transform_gaussian_pointwise(self.prior_sampler.mean, m)

    def solveFwd(self, u=None, m=None, transform_m=False):
        ...
        # solve
        self.u_fn.x.petsc_vec.set(0.0)
        self._setup_solver()
        self._ksp.solve(self.rhs, self.u_fn.x.petsc_vec)
        ...
        return self.function_to_vertex(self.u_fn, u, is_m=False)

    def samplePrior(self, m=None, transform_m=False):
        if transform_m:
            w, _ = self.prior_sampler()
            self.m_transformed = self.transform_gaussian_pointwise(w, \
                self.m_transformed)
        else:
            self.m_transformed = self.prior_sampler()[0]

```

Listing 2. Cont.

```

if m is None:
    return self.m_transformed.copy()
m = self.m_transformed.copy()
return m

```

Listing 3. Abstract PDE class.

```

import numpy as np
from dolfinx import fem
from fenicsUtilities import build_vector_vertex_maps

class PDEModel:

    def __init__(self, Vm, Vu, prior_sampler, seed = 0):
        ...
        # prior and transform parameters
        self.prior_sampler = prior_sampler

        # FE setup
        self.Vm, self.Vu, self.mesh = Vm, Vu, Vm.mesh
        ...

    # following functions must be defined in the inherited class
    # boundaryU(x, on_boundary)
    # is_point_on_dirichlet_boundary(x)
    # assemble(self, assemble_lhs=True, assemble_rhs=True)
    # compute_mean(self, m)
    # solveFwd(self, u=None, m=None, transform_m=False)
    # samplePrior(self, m=None, transform_m=False)

```

3.2. Linear Elasticity Problem

The second problem concerns deformation of a two-dimensional rectangular domain $D_U = (0, L_1) \times (0, L_2) \subset \mathbb{R}^2$. The deformation is governed by the linear elasticity equations. Suppose $m(x)$ denotes the Young's modulus field at a point $x \in D_U$, $u = (u_1, u_2) : D_U \rightarrow \mathbb{R}^2$ denotes the displacement field, $e(x) = (\nabla u + \nabla u^T)/2$ denotes the linearized strain, $\sigma(x)$ denotes the Cauchy stress, and $b(x)$ denotes the body force per unit volume. The equation for u is based on the balance of linear momentum and reads:

$$\begin{aligned}
 -\nabla \cdot \sigma(x) &= b(x), & \forall x \in D_U, \\
 \sigma(x) &= \lambda(x) \text{tr}(e) \mathbf{I} + 2\mu(x)e, & \forall x \in D_U, \\
 u(x) &= 0, & \forall x \in \Gamma_{u_d}, \\
 \sigma(x)n(x) &= t(x), & \forall x \in \Gamma_{u_q} = \partial D_U - \Gamma_{u_d},
 \end{aligned} \tag{26}$$

where $\text{tr}(e) = e_{11} + e_{22}$ is the trace of the strain tensor, $\mathbf{I}$ is the identity second-order tensor in $\mathbb{R}^2$, and λ and μ are Lamé parameters related to the Young's modulus m and the Poisson ratio ν as follows:

$$\lambda(x) = \frac{m(x)\nu}{(1+\nu)(1-2\nu)} \quad \text{and} \quad \mu(x) = \frac{m(x)}{2(1+\nu)}. \tag{27}$$

In (26), $\Gamma_{u_d} := \{x \in \partial D_U : x_1 = 0\}$ and $\Gamma_{u_q} := \partial D_U - \Gamma_{u_d}$, n is the unit outward normal, and t is the specified traction. The scalar field $m \in M$ is considered to be uncertain, and the forward map $F : M \rightarrow U$ is defined such that, given $m \in M$, the function $u = F(m) \in U$ solves the BVP (26). To be consistent with the notations, here $D_W = D_M = D_U$, $q_W = q_M = q_U = 2$, $d_W = d_M = 1$, and $d_U = 2$. Working in the variational setting, appropriate function spaces for the parameter field and solution are as follows:

$$M := \{m : D_M \rightarrow \mathbb{R}^+ : \|m\|_{L^2} < \infty\}, \quad U := \{u \in H^1(D_U; \mathbb{R}) : u = 0 \text{ on } \Gamma_{u_d}\}. \quad (28)$$

The weak form of the problem is as follows: given $m \in M$, find $u \in U$ such that

$$\int_{D_U} \sigma : \frac{(\nabla v + \nabla v^T)}{2} dx = \int_{D_U} b \cdot v dx + \int_{\Gamma_{u_q}} t \cdot v ds, \quad \forall v \in U, \quad (29)$$

where $A : B = \sum_{i,j=1}^2 A_{ij} B_{ij}$ denotes the tensor inner product between second-order tensors A and B . To ensure positivity, a log-normal distribution is used for m :

$$\mathcal{Q}(z) = m = \alpha_m \exp(w) + \beta_m, \quad \text{where } z \in \Omega, w = \mathcal{Z}(z), \text{ and } \mu_{\mathcal{Z}} = N(0, C). \quad (30)$$

Setup Details and Data Generation

The domain is the unit square, $L_1 = L_2 = 1$. The body force is zero, and a constant traction is applied on Γ_{u_q} :

$$b(x) = 0\hat{e}_1 + 0\hat{e}_2 \quad \text{and} \quad t(x) = 20\hat{e}_1 + 50\hat{e}_2. \quad (31)$$

The Poisson ratio is fixed at $\nu = 0.45$. The covariance operator is taken as $C = L_{\Delta}^{-2}$, with the parameters

$$a_c = 0.005, \quad b_c = 1, \quad c_c = 0.2, \quad \alpha_m = 100, \quad \beta_m = 1000. \quad (32)$$

As in the Poisson problem, a finite element approximation with triangle elements and linear interpolation is used for both the input and output functions. The number of elements is 5000, the number of nodes is $N_{nodes} = 2601$, $p_U = 2N_{nodes} = 5202$, and $p_M = N_{nodes} = 2601$. Listing 4 outlines crucial steps in solving the problem. The sampling of m is similar to problem 1. Figure 3 shows representative samples of w and corresponding (m, u) pairs. The notebook `LinearElasticity.ipynb` (repository [9], tag `survey26_v2`) implements methods to generate and post-process data for neural operator training.

Listing 4. Class for the linear elasticity problem. Here, only the initialization part is shown, as all other functions are similar to the implementation of the Poisson problem in Listing 2.

```
from pdeModel import PDEModel
...
class LinearElasticityModel(PDEModel):

    def __init__(self, Vm, Vu, prior_sampler, logn_scale=1., logn_translate=0.,
                 seed=0):
        ...
        self.nu = 0.45
        self.lam_fact = self.nu / ((1 + self.nu) * (1 - 2 * self.nu))
        self.mu_fact = 1.0 / (2 * (1 + self.nu))

        # External body force and boundary traction
        self.b = ufl.as_vector((0.0, 0.0))
        self._traction_x = fem.Constant(self.mesh, default_scalar_type(20.0))
        self._traction_y = fem.Constant(self.mesh, default_scalar_type(50.0))
        self.t = ufl.as_vector((self._traction_x, self._traction_y))

        # variational form
        I = ufl.Identity(spatial_dim)
        sigma = self.lam_fact * ufl.tr(ufl.grad(self.u_trial)) * I \
```

Listing 4. *Cont.*

```

        + 2 * self.mu_fact * ufl.sym(ufl.grad(self.u_trial))

        self.a_form = self.m_fn * ufl.inner(sigma, ufl.sym(ufl.grad(self.u_test)))
    * dx
    self.L_form = ufl.inner(self.b, self.u_test) * dx \
        + ufl.inner(self.t, self.u_test) * ds
    ...
    ...
    def solveFwd(self, u = None, m = None, transform_m = False):
        # similar to the solveFwd() for the Poisson problem (Listing 3)

    def samplePrior(self, m = None, transform_m = False):
        # similar to the samplePrior() for the Poisson problem (Listing 3)

```

3.3. Hyperelasticity Problem

The third problem concerns hyperelastic deformation on the same rectangular domain considered in Section 3.2. The uncertain parameter remains the Young's modulus field $m(x)$, and the displacement field is $u : D_U \rightarrow \mathbb{R}^2$. The deformation gradient, right Cauchy–Green tensor, first invariant, and Jacobian are

$$\mathbf{F} = \mathbf{I} + \nabla u, \quad \mathbf{C} = \mathbf{F}^T \mathbf{F}, \quad I_1 = \text{tr}(\mathbf{C}), \quad J = \det(\mathbf{F}). \quad (33)$$

A compressible Neo-Hookean strain-energy density [15] is used:

$$W(\mathbf{F}) = \frac{\mu}{2}(I_1 - 3 - 2 \ln J) + \frac{\lambda}{2}(\ln J)^2, \quad (34)$$

where λ and μ are the Lamé parameters related to the Young's modulus m and Poisson ratio ν , as in (27). Using the chain rule and the identities (Eqs. (1.226), (1.228), and (1.232) in [16])

$$\frac{\partial I_1}{\partial \mathbf{F}} = 2\mathbf{F}, \quad \frac{\partial J}{\partial \mathbf{F}} = J\mathbf{F}^{-T}, \quad \frac{\partial \ln J}{\partial \mathbf{F}} = \mathbf{F}^{-T}, \quad (35)$$

the first Piola–Kirchhoff stress for the strain-energy density in (34) is

$$\mathbf{P} = \frac{\partial W}{\partial \mathbf{F}} = \mu \mathbf{F} + (\lambda \ln J - \mu) \mathbf{F}^{-T}. \quad (36)$$

The strong form of the hyperelasticity problem on the displacement field u is written in the reference configuration as

$$\begin{aligned} -\nabla \cdot \mathbf{P}(x) &= b(x), & \forall x \in D_U, \\ u(x) &= 0, & \forall x \in \Gamma_{u_d}, \\ \mathbf{P}(x)n(x) &= t(x), & \forall x \in \Gamma_{u_q} = \partial D_U - \Gamma_{u_d}, \end{aligned} \quad (37)$$

As in the case of linear elasticity, consider the function spaces

$$M := \{m : D_M \rightarrow \mathbb{R}^+ : \|m\|_{L^2} < \infty\}, \quad U := \{u \in H^1(D_U; \mathbb{R}^2) : u = 0 \text{ on } \Gamma_{u_d}\}. \quad (38)$$

Then, the weak form of the problem is as follows: given $m \in M$, find $u \in U$ such that

$$\int_{D_U} \mathbf{P} : \nabla v \, dx = \int_{D_U} b \cdot v \, dx + \int_{\Gamma_{u_q}} t \cdot v \, ds, \quad \forall v \in U. \quad (39)$$

As before, the forward map $F : M \rightarrow U$ is defined by $u = F(m)$, where $m \in M$ is the uncertain Young's modulus field.

The transformation from w to m and the prior model are the same as in Section 3.2.

Setup Details and Data Generation

The setup, including the domain, external loading, Poisson ratio, and sampling details, is the same as in Section 3.2. The main computational change is that each forward solve requires a Newton iteration because the hyperelasticity problem is nonlinear. To improve robustness, the traction load is applied incrementally over 20 load steps during the non-linear solve; see `solveFwd` in Listing 5 for the key implementation steps. Figure 3 shows representative samples of w and corresponding (m, u) pairs. The notebook `Hyperelasticity.ipynb` (repository [9], tag `survey26_v2`) implements methods to generate and post-process data for neural operator training.

Listing 5. Class for the hyperelasticity problem. Only the initialization and nonlinear-solver components are shown because the sampling and data-handling functions are similar to the linear elasticity implementation.

```
...
from pdeModel import PDEModel

class HyperelasticityModel(PDEModel):

    def __init__(self, Vm, Vu, prior_sampler, logn_scale=1., logn_translate=0.,
                 seed=0):
        # most setup details (nu, b, t, ...) follow LinearElasticityModel
        ...
        # variational form
        spatial_dim = self.mesh.geometry.dim
        I = ufl.variable(ufl.Identity(spatial_dim))
        F = ufl.variable(I + ufl.grad(self.u_fn))
        C = ufl.variable(F.T * F)
        Ic = ufl.variable(ufl.tr(C))
        J = ufl.variable(ufl.det(F))

        mu = self.m_fn * self.mu_fact
        lam = self.m_fn * self.lam_fact
        # Stable compressible Neo-Hookean
        W = (mu / 2.0) * (Ic - 3.0 - 2.0 * ufl.ln(J)) + (lam / 2.0) * (ufl.ln(J))
    ** 2
        P = ufl.diff(W, F)

        self._residual_form = ufl.inner(self.b, self.u_test) * dx \
                               + ufl.inner(self.t, self.u_test) * ds \
                               - ufl.inner(ufl.grad(self.u_test), P) * dx

        self._nonlinear_problem = None
        self._newton_solver = None
        ...

    def _setup_solver(self):
        if self._nonlinear_problem is None:
            self._nonlinear_problem = NewtonSolverNonlinearProblem(
                self._residual_form, self.u_fn, bcs = self.bc)
            self._newton_solver = \
                NewtonSolver(self.mesh.comm, self._nonlinear_problem)
            ...

    def _run_newton(self):
        self._setup_solver()
        n_iter, converged = self._newton_solver.solve(self.u_fn)
```

Listing 5. Cont.

```

if not converged:
    raise RuntimeError("Hyperelasticity Newton solver did not converge.")
return n_iter

def solveFwd(self, u = None, m = None, transform_m = False):
    ...
    # load stepping for robust convergence
    target_tx = float(self._traction_x.value)
    target_ty = float(self._traction_y.value)
    n_steps = self.n_load_steps if self.reset_u else 1

    for step in range(1, n_steps + 1):
        if n_steps > 1:
            load_frac = step / n_steps
            self._traction_x.value = target_tx * load_frac
            self._traction_y.value = target_ty * load_frac
            self._run_newton()
        ...
    return self.function_to_vertex(self.u_fn, u, is_m = False)

def samplePrior(self, m = None, transform_m = False):
    # similar to the samplePrior() for the Poisson problem (Listing 3)

```

4. Neural Operators as Surrogates for the Forward Problem

Over the years, several neural operator architectures have emerged that leverage neural networks to construct efficient approximations of F . This section presents representative approaches, emphasizing their underlying principles and implementation details to develop a practical understanding of their methods.

The following subsections introduce DeepONet, PCANet, and FNO, along with their implementation details.

4.1. Deep Operator Network (DeepONet)

DeepONet was first introduced in [17], and over the years, various extensions of the general framework and applications have been realized. Consider some m , and suppose $\{\phi_i^U\}_{i=1}^{p_U} \subset U$ is a finite collection of basis functions; then, at $x \in D_U$,

$$F(m)(x) = u(x) \approx \sum \underbrace{(F(m), \phi_i^U)}_{=: \alpha_i(m)} \phi_i^U(x), \quad (40)$$

where coefficients $\alpha_i = \alpha_i(m)$ depend on m . DeepONet's underlying idea is to learn the above finite-dimensional representation of the output of operator F . That is, identify the linear bases, or more precisely, learn the values of basis functions $\phi_i^U(x)$ at coordinates $x \in D_U$, and the coefficients associated with the bases dependent on the input m , $\{\alpha_i(m)\}$ so that $\sum_i \alpha_i(m) \phi_i^U(x)$ is approximately equal to $F(m)(x) = u(x)$. Towards this, DeepONet considers two neural networks, so-called branch and trunk networks; see Figure 4. The branch network takes discretization of the input function, denoted by $\mathbf{m} \in \mathbb{R}^{p_M}$, and the neural network produces N_{br} number of coefficients, which are used as $\{\alpha_i\}_{i=1}^{N_{br}}$. On the other hand, the trunk network takes as input the spatial coordinate, x , and outputs $d_U N_{tr}$ numbers, which play the role of $\{\phi_i^U(x)\}_{i=1}^{N_{tr}}$ in (40). If m and u are scalar fields, then $N_{tr} = N_{br}$. Finally, the loss function is defined in terms of the norm of the difference between the ground-truth and approximation by the joint output from the branch and trunk networks, $\sum_{i=1}^{N_{br}} \alpha_i(m) \phi_i^U(x)$.

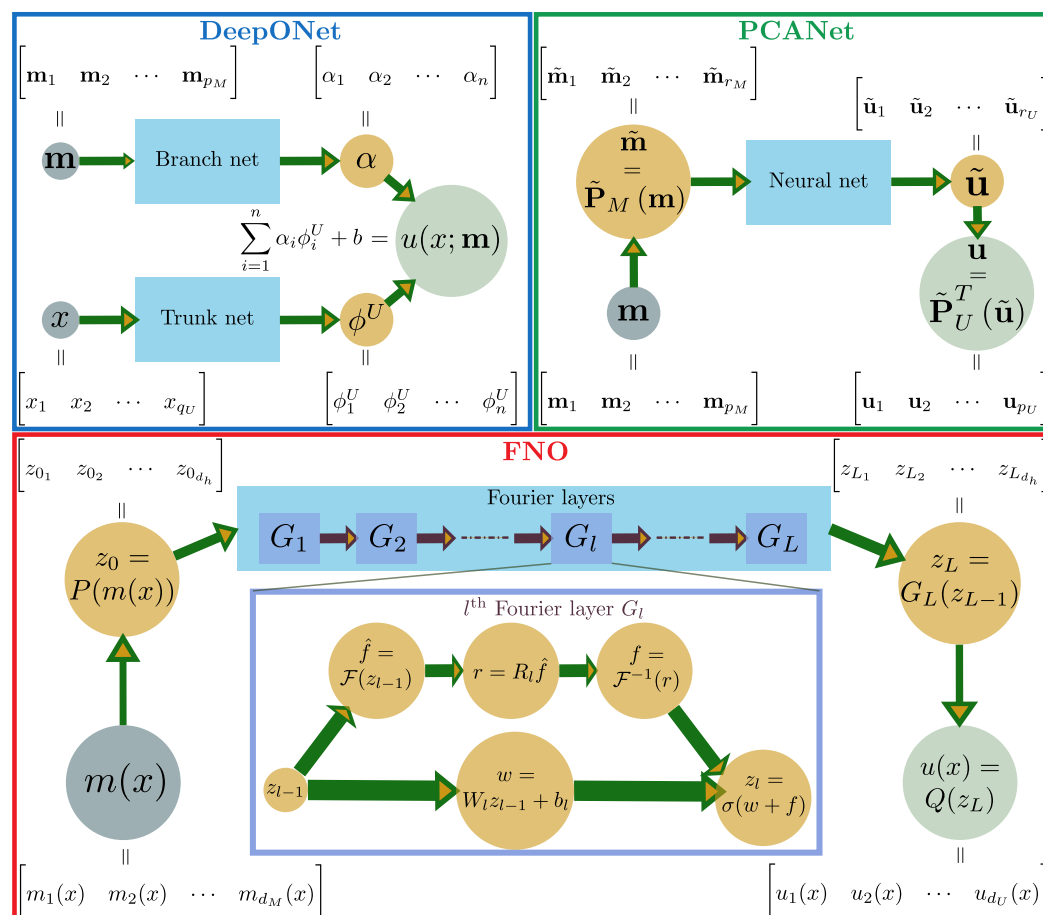


Figure 4. Schematics of three neural operators, DeepONet, PCANet, and FNO. Grey and light green circles represent the input and output of neural operators. The blue box includes a parameterized neural network-based map. In this work, the blue boxes for DeepONet and PCANet employ fully connected multi-layer perceptron neural networks. In the case of FNO, trainable parameters (namely, R_l, W_l, b_l) appear within each Fourier layer.

In summary, the operator approximation in DeepONet takes the form, for $m \in M$ (input actually is the finite-dimensional discretization $\mathbf{m} \in \mathbb{R}^{p_M}$ of $m \in M$) and $x \in D_U$,

$$F_{NOp}(m)(x) = \sum_{i=1}^{N_{br}} \alpha_i(m; \theta_{br}) \phi_i^U(x; \theta_{tr}) + b, \quad (41)$$

where

- (i) $\alpha_i = \alpha_i(m; \theta_{br}) \in \mathbb{R}$, $1 \leq i \leq N_{br}$ are outputs of the branch network with θ_{br} neural network trainable parameters;
- (ii) $\phi_i^U(x; \theta_{tr}) \in \mathbb{R}^{d_U}$, $1 \leq i \leq N_{br}$ are outputs of the trunk network (total $N_{tr} = d_U N_{br}$ numbers) with θ_{tr} neural network trainable parameters;
- (iii) $b \in \mathbb{R}^{d_U}$ is a bias.

A key feature of DeepONet is the separation between learning input-dependent coefficients and spatial representations via the branch and trunk networks. Another crucial property of DeepONet is that an approximation of $u = F(m)$ at any arbitrary point $x \in D_U$ can be computed.

4.1.1. Implementation of DeepONet

To simplify the presentation, the input and output functions, $m \in M$ and $u \in U$, respectively, are assumed to be scalar-valued, and they are appropriately discretized, e.g., using a finite element approximation. Extending the cases when one or both of these functions are vector-valued is trivial; see Remark 5.

Let $\mathbf{m} \in \mathbb{R}^{p_M}$ be the input discretized function, and let $\mathbf{u} = F(\mathbf{m}) \in \mathbb{R}^{p_U}$ be the corresponding output discretized function, where F is the discretization of the operator of interest F . The following constitutes data for DeepONet:

1. Consider the $N_M \times p_M$ -matrix, $\mathbf{X}_{br}$, where N_M is the number of input function samples and I th row of $\mathbf{X}_{br}$ is the sample $\mathbf{m}^I \in \mathbb{R}^{p_M}$. Each row of $\mathbf{X}_{br}$ will be the input to the branch network.
2. Select N_x number of locations, $\{x^I\}_{I=1}^{N_x}$, where $x^I \in D_U \subseteq \mathbb{R}^{q_U}$ and q_U is the dimension of the domain. Each location x^I will serve as input to the trunk neural network, so the DeepONet output is the prediction of the target function's value at x^I . In the present implementation, the typical input coordinate x^I corresponds to the I th discretization grid or node of a mesh so that the value of the output function at x^I is represented by the element u_I in the vector $\mathbf{u}$ corresponding to that grid/node (this is true only for scalar-valued functions). The matrix $\mathbf{X}_{tr}$ of size $N_x \times q_U$ is the data for the trunk network, and each row of $\mathbf{X}_{tr}$ is the input to the trunk network.
3. $N_M \times N_x$ -matrix $\mathbf{Y}$, such that an element $\mathbf{Y}_{IJ}$ is the pointwise value of the output data corresponding to $\mathbf{m}^I$ at location x^J , i.e.,

$$\mathbf{Y}_{IJ} = (\mathbf{u}^I)_J \approx F(\mathbf{m}^I)(x^J). \quad (42)$$

i.e., $\mathbf{Y}_{IJ}$ is the pointwise value of the output data corresponding to the branch network input data function $\mathbf{m}^I$ (I th row of $\mathbf{X}_{br}$) at trunk network input location x^J (J th row of $\mathbf{X}_{tr}$).

Next, the combined training parameters $\theta = \{\theta_{br}, \theta_{tr}, b\} \in \mathbb{R}^{p_\Theta}$ are optimized to minimize the error:

$$\theta^* = \arg \min_{\theta = \{\theta_{br}, \theta_{tr}, b\} \in \mathbb{R}^{p_\Theta}} \frac{1}{N_M N_x} \sum_{I=1}^{N_M} \sum_{J=1}^{N_x} \left| \mathbf{Y}_{IJ} - \underbrace{\left(\sum_{k=1}^{N_{br}} \alpha_k(\mathbf{m}^I; \theta_{br}) \phi_k^U(x^J; \theta_{tr}) + b \right)}_{\text{DeepONet output}} \right|^2. \quad (43)$$

Here, p_Θ is the total number of trainable parameters in the DeepONet architecture.

Remark 5. The DeepONet framework described so far can be easily extended to the case where the operator's target function is vector-valued. Suppose $\mathbf{u}(x) = (u_1(x), u_2(x), \dots, u_{d_U}(x)) \in \mathbb{R}^{d_U}$, u_j being the j th component function. One way to extend the DeepONet framework discussed above to vector-valued target functions is by considering N_{br} branch outputs $\{\alpha_k\}_{k=1}^{N_{br}}$ and $N_{tr} = d_U N_{br}$ trunk outputs $\{\phi_k^U\}_{k=1}^{N_{tr}}$ and using (41). Here, ϕ_k^U are $\mathbb{R}^{d_U}$ -valued functions, so the number of scalar outputs from the trunk network is $N_{tr} = d_U N_{br}$.

The alternative construction considered in this work uses a trunk network that produces N_{tr} outputs $\{\psi_k\}_{k=1}^{N_{tr}}$, where $\psi_k(x) \in \mathbb{R}$. The branch network produces $N_{br} = d_U N_{tr}$ outputs $\{\alpha_k\}_{k=1}^{N_{br}}$. Then, for the x^J row of $\mathbf{X}_{tr}$ and the $\mathbf{m}^I$ row of $\mathbf{X}_{br}$, the prediction $u_{pred} = (u_{pred,1}, u_{pred,2}, \dots, u_{pred,d_U})$ of $\mathbf{u}$ is given by

$$\begin{aligned}
 u_{pred,1} &= \sum_{k=1}^{N_{tr}} \alpha_k(\mathbf{m}^I; \theta_{br}) \psi_k(x^J; \theta_{tr}) + b_1 \\
 u_{pred,2} &= \sum_{k=1}^{N_{tr}} \alpha_{k+N_{tr}}(\mathbf{m}^I; \theta_{br}) \psi_k(x^J; \theta_{tr}) + b_2 \\
 &\vdots \\
 u_{pred,d_U} &= \sum_{k=1}^{N_{tr}} \alpha_{k+(d_U-1)N_{tr}}(\mathbf{m}^I; \theta_{br}) \psi_k(x^J; \theta_{tr}) + b_{d_U},
 \end{aligned} \tag{44}$$

i.e., the first N_{tr} outputs of the branch are used to predict u_1 component, the next N_{tr} branch outputs are used to predict u_2 component, and so on. In the above, the same trunk outputs are used for all components of the target functions.

Multi-layer perception (MLP) is used as branch and trunk networks. The implementation used in this work is based on the DeepONet GitHub repository (<https://github.com/GideonIlung/DeepONet>, accessed on 26 November 2024) with some minor modifications. Listing 6 shows the implementation of MLP and Listing 7 implements the DeepONet framework.

Listing 6. Multi-layer Perceptron (MLP) implementation.

```

import torch
import torch.nn as nn

class MLP(nn.Module):

    def __init__(self, input_size, hidden_size, num_classes, depth, act):
        super(MLP, self).__init__()
        self.layers = nn.ModuleList()
        self.act = act

        # input layer
        self.layers.append(nn.Linear(input_size, hidden_size))

        # hidden layers
        for _ in range(depth - 2):
            self.layers.append(nn.Linear(hidden_size, hidden_size))

        # output layer
        self.layers.append(nn.Linear(hidden_size, num_classes))

    def forward(self, x, final_act=False):
        for i in range(len(self.layers) - 1):
            x = self.act(self.layers[i](x))

        # last layer
        x = self.layers[-1](x)
        if final_act == False:
            return x
        else:
            return torch.relu(x)

```

Listing 7. Core implementation of the DeepONet architecture.

```

...
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
...
from torch_mlp import MLP
from dataMethods import DataHandler
...

class DeepONet(nn.Module):

    def __init__(self, ...):
        super(DeepONet, self).__init__()
        ...
        # branch network
        self.branch_net = MLP(input_size=num_inp_fn_points, hidden_size=
num_neurons, \
                                num_classes=num_br_outputs, depth=num_layers, act=
act)
        self.branch_net.float()

        # trunk network
        self.trunk_net = MLP(input_size=out_coordinate_dimension, \
                                hidden_size=num_neurons, num_classes=num_tr_outputs,
\
                                depth=num_layers, act=act)
        self.trunk_net.float()

        # bias added to the product of branch and trunk networks
        self.bias = [nn.Parameter(torch.ones((1,)), requires_grad=True) \
                                for i in range(num_Y_components)]
        ...
    ...
    def forward(self, X, X_trunk):
        ...
        br_out = self.branch_net.forward(X)
        tr_out = self.trunk_net.forward(X_trunk, final_act=True)
        ntr = self.num_tr_outputs

        if self.num_Y_components == 1:
            output = br_out @ tr_out.t() + self.bias[0]
        else:
            # if d_u > 1, split the branch output and compute the product
            output = []
            for i in range(self.num_Y_components):
                output.append(br_out[:, i*ntr:(i+1)*ntr] @ tr_out.t() + self.bias[i
])

            # stack and reshape
            output = torch.stack(output, dim=-1)
            output = output.reshape(-1, X_trunk.shape[0] * self.num_Y_components)

        return output

    def train(self, train_data, test_data, batch_size, epochs, lr, ...):
        ...
        # train and test dataloaders to sample batches of data
        dataset = DataHandler(train_data['X_train'], \
                                train_data['X_trunk'], train_data['Y_train'])
        dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

```

Listing 7. Cont.

```

test_dataset = DataHandler(test_data['X_train'], \
                           test_data['X_trunk'], test_data['Y_train'])
test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=
True)

# loss and optimizer setup
criterion = nn.MSELoss()
optimizer = optim.Adam(self.parameters(), lr=lr, weight_decay=1e-4)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=100, gamma=0.5)
...
# number of trainable parameters in the model
self.trainable_params = sum(p.numel() for p in \
                             self.parameters() if p.requires_grad)
...
for epoch in range(1, epochs+1):
    ...
    # training loop
    for X_train, _, Y_train in dataloader:
        ...
        # clear gradients
        optimizer.zero_grad()

        # forward pass through model
        Y_train_pred = self.forward(X_train, X_trunk)

        # compute and save loss
        loss = criterion(Y_train_pred, Y_train)
        train_losses.append(loss.item())

        # backward pass
        loss.backward()

        # update parameters
        optimizer.step()

    # update learning rate
    scheduler.step()

    # testing loop
    with torch.no_grad():
        for X_test, _, Y_test in test_dataloader:
            # forward pass through model
            Y_test_pred = self.forward(X_test, X_trunk)

            # compute and save test loss
            test_loss = criterion(Y_test_pred, Y_test)
            test_losses.append(test_loss.item())
        # log losses, print progress, save model checkpoints, etc.
    ...
def predict(self, X, X_trunk):
    with torch.no_grad():
        return self.forward(X, X_trunk)

```

4.1.2. Architecture and Preliminary Results

A three-layer fully connected network is considered for both the branch and trunk networks. Neural network and optimization-related parameters are tabulated in Table 2. For the linear elasticity and hyperelasticity problems, $N_{br} = 200$ and $N_{tr} = 100$, and the prediction of vector-valued outputs is based on Remark 5.

For the Poisson problem, Figure 5 shows the typical prediction error using DeepONet. In Figures A1 and A2, the results for the linear elasticity and hyperelasticity problems are shown. These figures also compare the accuracy of PCANet and FNO, which will be discussed in the following two subsections. The notebooks `DeepONet-Poisson`, `DeepONet-Linear Elasticity`, and `DeepONet-Hyperelasticity` in the repository [9] (tag `survey26_v2`) describe the steps used to construct, train, and test the DeepONet models for the three problems.

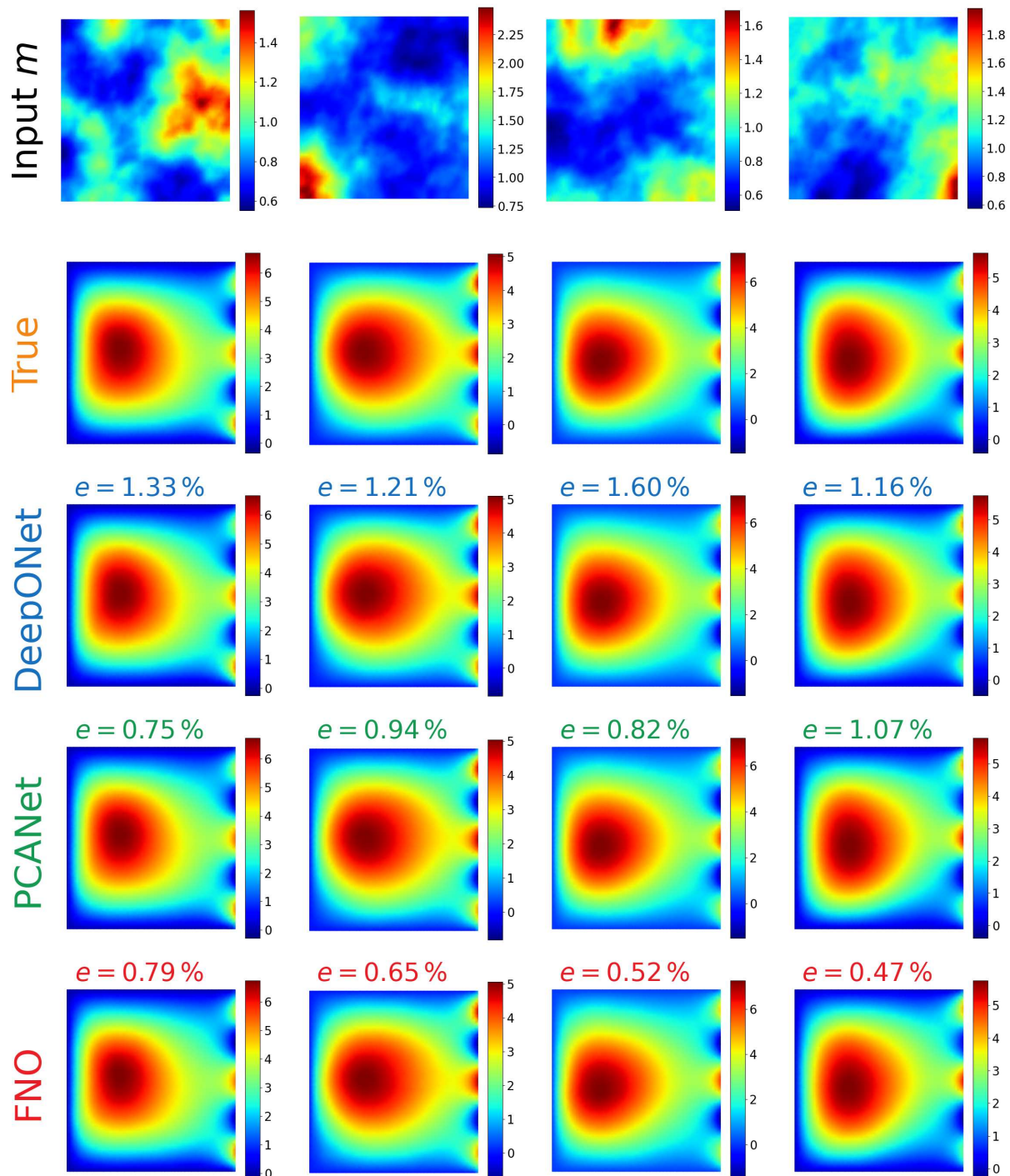


Figure 5. Comparison of DeepONet, PCANet, and FNO predictions with the finite element solution for four samples of diffusivity in the Poisson problem. Here, e denotes the relative percentage ℓ^2 error between the true finite element solution and the surrogate solution.

Table 2. Summary of parameters used in neural operator training and testing.

Parameter	Value	Description
Layers L	3	Number of layers in DeepONet (branch and trunk each), PCANet, and FNO
Width	64	Number of neurons per hidden layer (DeepONet)
Width	300	Number of neurons per hidden layer (PCANet)
d_h	22	Dimension of FNO layer outputs
N_{train}	3200	Number of training data pairs (m^I, u^I)
N_{test}	800	Number of testing data pairs (m^I, u^I)
N_m	3200	N_m is same as N_{train} (notation used for DeepONet)
N_x	2601	Number of coordinates for evaluation of u for DeepONet
p_M, p_U	2601, $2601d_U$	Dimensions of discretized functions ($d_U = 2$ for elasticity)
r_M, r_U	200, 100	Reduced dimensions for the three problems for PCANet
N_{br}, N_{tr}	$100d_U, 100$	Number of branch and trunk outputs in DeepONet
Batch	20	Neural networks are trained on “batch size” samples
Epochs	1000	Number of optimization steps
Epochs	400	Number of optimization steps (FNO)
LR	0.001	Learning rate: controls parameter update during training
Activation	ReLU	Activation function
k_{max}	8	Number of Fourier modes to keep in FNO
n_1, n_2	51, 51	Number of grid points in $D_U = (0, 1)^2$ for FNO

4.2. Principal Component Analysis-Based Neural Operator (PCANet)

The second neural operator of interest is the PCANet introduced in [3], which utilizes Principal Component Analysis (PCA) to reduce the dimensions of input and output discretized functions and poses an operator learning problem in the reduced-dimensional spaces, thereby making learning more efficient. Consider $\{(m^I, u^I = F(m^I))\}_{I=1}^N$ set of paired data, where $m^I \in \mathbb{R}^{p_M}$ and $u^I \in \mathbb{R}^{p_U}$. In general, learning the approximation of the target map $F : \mathbb{R}^{p_M} \rightarrow \mathbb{R}^{p_U}$ becomes challenging if p_M and p_U are large. PCANet alleviates this challenge by introducing a low-dimensional approximation of F . Specifically, in PCANet, the dimensions of input and output functions are reduced using SVD, and the neural network between reduced-dimensional inputs and outputs is introduced to approximate the mapping. The effectiveness of this approach depends on whether the input and output data admits accurate low-dimensional representations, which is typically reflected in the decay of the singular values of the corresponding data matrices. When the singular values decay slowly, a much larger reduced dimension may be required, and the efficiency and accuracy advantages of PCANet can be reduced.

To make this precise, let r_M and r_U denote the reduced dimensions for input and output functions, respectively, and $\tilde{P}_M \in \mathbb{R}^{r_M \times p_M}$ and $\tilde{P}_U \in \mathbb{R}^{r_U \times p_U}$ are the projectors based on SVD for dimension reduction. Now consider a parameterized neural network map $\tilde{F}_\theta : \mathbb{R}^{r_M} \rightarrow \mathbb{R}^{r_U}$ to construct the approximation F_{NOp} of F as follows, for a given $m \in \mathbb{R}^{p_M}$,

$$\mathbb{R}^{p_U} \ni u = F(m) \quad \approx \quad F_{NOp}(m) = \underbrace{\tilde{P}_U^T \tilde{F}_\theta \left(\underbrace{\tilde{P}_M(m)}_{=: \tilde{m} \text{ (project } m)} \right)}_{=: \tilde{u} \text{ (low-dim map)}} \in \mathbb{R}^{p_U}. \quad (45)$$

lift $\tilde{u}$

Conceptually, the parameters θ can be determined via the optimization problem:

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^{p_\Theta}} \frac{1}{N} \sum_{I=1}^N \left\| F(m^I) - \tilde{P}_U^T \left(\tilde{F}_\theta \left(\tilde{P}_M(m^I) \right) \right) \right\|^2, \quad (46)$$

where $\|\cdot\|$ denotes the ℓ^2 -norm and p_Θ is the number of trainable parameters. Here, the projection operators are fixed, and the only learnable component is the reduced map $\tilde{F}_\theta$.

Thus, unlike DeepONet, which learns basis representations explicitly, PCANet relies on fixed data-driven bases obtained via SVD and focuses on learning the mapping in the reduced space. In practice, the optimization problem (46) is reformulated in the reduced space, i.e., the parameters θ in $\tilde{F}_\theta$ are trained using projected data. Figure 4 presents the schematic of PCANet and the associated projection steps.

4.2.1. Implementation of PCANet

Let $\mathbf{X}$ and $\mathbf{Y}$ be two $N \times p_M$ and $N \times p_U$ matrices, respectively, such that rows of $\mathbf{X}$ and $\mathbf{Y}$ are input and output function samples, $m^I \in \mathbb{R}^{p_M}$ and $u^I \in \mathbb{R}^{p_U}$, where $1 \leq I \leq N$. The data for the neural network-based map $\tilde{F}_\theta$ in (45) is constructed using the following steps, which include the preprocessing to construct SVD-based projectors:

1. *Center and scale* the data matrices $\mathbf{X}$ and $\mathbf{Y}$ by subtracting the sample mean and dividing elementwise by the sample standard deviation. For example, if $\bar{\mathbf{X}}$ and σ_X are $1 \times p_M$ row vectors containing the mean and standard deviation of the columns of $\mathbf{X}$, then the normalized input data $\hat{\mathbf{X}}$ is obtained by

$$\hat{\mathbf{X}} = \frac{\mathbf{X} - \bar{\mathbf{X}}}{\sigma_X + \text{tol}},$$

where each row of $\mathbf{X}$ is subtracted by the row vector $\bar{\mathbf{X}}$, and the division by $\sigma_X + \text{tol}$ is performed elementwise. The small number tol is introduced to ensure numerical stability. The normalized output data $\hat{\mathbf{Y}}$ is obtained in the same way.

2. *SVD projectors* for input and output data are determined following the procedure in Section 2.3. For example, take $\mathbf{R} = \hat{\mathbf{X}}^T$ and compute the projector $\tilde{P}_M \in \mathbb{R}^{r_M \times p_M}$, where r_M is the specified reduced dimension. Similarly, the projector $\tilde{P}_U$ is obtained using $\mathbf{R} = \hat{\mathbf{Y}}^T$.
3. *Projected data for the neural network* are computed by projecting the rows of $\hat{\mathbf{X}}$ into the reduced space. To be specific, let the I th row of $\hat{\mathbf{X}}$ be $(\hat{m}^I)^T \in \mathbb{R}^{1 \times p_M}$, where $\hat{m}^I$ is the centered and scaled I th input sample. Its projection is $\tilde{m}^I = \tilde{P}_M(\hat{m}^I) \in \mathbb{R}^{r_M}$. Using this projection, a new matrix $\tilde{\mathbf{X}}$ of size $N \times r_M$ is formed, where $(\tilde{m}^I)^T \in \mathbb{R}^{1 \times r_M}$ is the I th row. Similarly, $\tilde{\mathbf{Y}}$ of size $N \times r_U$ is obtained by applying $\tilde{P}_U$ to each centered and scaled output sample.
4. For the reduced-dimensional map $\tilde{F}_\theta : \mathbb{R}^{r_M} \rightarrow \mathbb{R}^{r_U}$, defined by $\tilde{u} = \tilde{F}_\theta(\tilde{m})$ (see (45)), the matrices $\tilde{\mathbf{X}}$ and $\tilde{\mathbf{Y}}$ constitute the input and output data, respectively.

Given the projected data matrices $\tilde{\mathbf{X}}$ and $\tilde{\mathbf{Y}}$ and a neural network-based map $\tilde{F}_\theta$, the optimization problem for determining θ is

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^{p_\Theta}} \frac{1}{N} \sum_{I=1}^N \left\| \tilde{\mathbf{Y}}_I - \tilde{F}_\theta(\tilde{\mathbf{X}}_I) \right\|^2, \quad (47)$$

where $(\cdot)_I$ denotes the I th row of a matrix. Once θ^* is determined, the map $\tilde{F}_{\theta^*}$ can be used as in (45) to obtain the neural operator surrogate F_{NOP} of the target operator F . The core steps in implementing PCANet, i.e., $\tilde{F}_\theta$, are shown in Listing 8.

Listing 8. PCANet implementation. Its interface is similar to that of DeepONet, except that PCANet does not take spatial coordinates as input.

```
...
from torch_mlp import MLP
...

class PCANet(nn.Module):

    def __init__(self, ...):
        super(PCANet, self).__init__()
        ...
        self.net = MLP(input_size=num_inp_red_dim, hidden_size=num_neurons, \
                        num_classes=num_out_red_dim, depth=num_layers, act=act
        )
        self.net.float()
        ...

    def forward(self, X):
        X = self.convert_np_to_tensor(X)
        return self.net.forward(X)

    def train(self, train_data, test_data, batch_size, epochs, lr, ...):
        ...
        # train and test dataloaders to sample batches of data
        dataset = DataHandler(train_data['X_train'], None, train_data['Y_train'])
        dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

        test_dataset = DataHandler(test_data['X_train'], None, test_data['Y_train'
        ])
        test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=
        True)

        # rest is similar to DeepONet in Listing 7 without X_trunk
        ...

    def predict(self, X):
        with torch.no_grad():
            return self.forward(X)
```

4.2.2. Architecture and Preliminary Results

A fully connected neural network with three layers is used to test PCANet. Other parameters, including the reduced dimensions, are listed in Table 2. First, the singular values of the input and output data are analyzed in Figure 6 to identify reasonably small reduced dimensions without significantly compromising accuracy. For all three problems, both input data and output data show a rapid decay of singular values. Since the input data exhibits a relatively slower decay, the reduced dimension for the input data is selected to be higher than that for the output data; see Table 2.

The predictions from trained PCANet are compared with the finite element solution and the other two neural operators in Figure 5 for the Poisson problem. The results for the linear elasticity and hyperelasticity problems are shown in Figure A1 and Figure A2, respectively. The notebooks PCANet-Poisson, PCANet-Linear Elasticity, and PCANet-Hyperelasticity in the repository [9] (tag survey26_v2) describe the construction, training, and testing of PCANet models for the three problems.

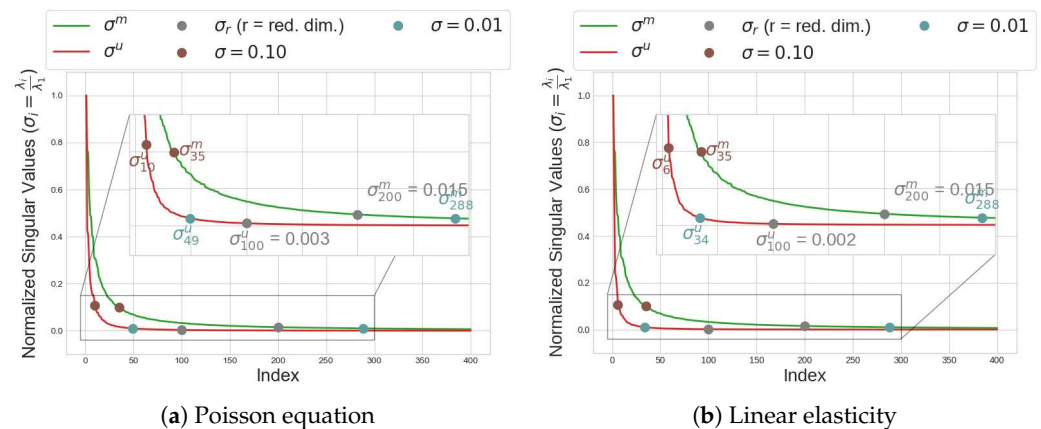


Figure 6. Normalized singular values of the centered and scaled input and output data matrices $\hat{\mathbf{X}}$ and $\hat{\mathbf{Y}}$. The green and red curves represent the normalized singular values of the input and output data, respectively. For both curves, annotations are shown near the 10% (brown dots) and 1% (blue dots) levels, i.e., 0.1 and 0.01 fractions of the largest singular value. The grey dots correspond to the fixed reduced dimensions used in PCANet. The rapid decay of singular values shows that the Poisson and linear elasticity problems are inherently low-dimensional. For all problems, the input samples are drawn from the same distribution using the same random-number-generator seed; therefore, the singular-value curves for the input data are the same in the left and right panels. The hyperelasticity problem exhibits a similar spectral decay to the linear elasticity problem.

4.3. Fourier Neural Operator (FNO)

Fourier neural operator considers the composition of layers, with a typical layer involving an affine transformation and an integral kernel operator followed by nonlinear activation. These affine and integral kernel operations are parameterized. While there are multiple choices of integral kernel operator [6], this work uses the Fourier transform. Consider the case of $D_M = D_U = D \subset \mathbb{R}^q$ (therefore, $q_M = q_U = q$) and the neural operator approximation F_{NOp} such that

$$u(x) = F(m)(x) \approx F_{NOp}(m)(x) := Q \left(\underbrace{G_L \left(G_{L-1} \cdots G_1 \left(\underbrace{P(m(x))}_{=z_0 \in \mathbb{R}^{d_h}} \right) \right)}_{=z_L(x) \in \mathbb{R}^{d_h}} \right), \quad (48)$$

or, concisely,

$$F_{NOp} = Q \circ G_L \circ G_{L-1} \circ \cdots \circ G_1 \circ P, \quad (49)$$

where $\circ$ denotes the composition of maps. The map F_{NOp} involves the following operations:

- Lifting $m(x) \in \mathbb{R}^{d_M}$ to $\mathbb{R}^{d_h}$, where d_h is the dimension of outputs from hidden layers via the trainable matrix P of size $d_h \times d_M$.
- Projecting the final hidden-layer output $z_L(x) \in \mathbb{R}^{d_h}$ onto $\mathbb{R}^{d_U}$ (space of $u(x)$) via the $d_U \times d_h$ trainable matrix Q . Thus, z_L is a d_h -dimensional feature field defined over the domain D .
- Application of operators G_l , $1 \leq l \leq L$, where G_l is defined via the following:

$$\mathbb{R}^{d_h} \ni z_l = G_l(z_{l-1}) = \sigma_l \left(\underbrace{W_l z_{l-1} + b_l}_{\text{linear/local operation}} + \underbrace{K_l(z_{l-1})}_{\text{nonlocal operation}} \right). \quad (50)$$

Here, $z_l = z_l(x)$ denotes a d_h -dimensional feature field defined over the domain D . Note that z_{l-1} is the output of the preceding layer, σ_l activation function, W_l weight matrix, b_l a bias vector, and $K_l(\cdot)$ an integral kernel operator.

Up until now, the map F_{NOp} in Equation (48) with the above definitions of P , Q , G_l , $l = 1, \dots, L$, is abstract due to the generality of K_l . The linear operation in (50) captures the local effects on the reconstructed function, while the K_l is designed to capture the nonlocal effects (interaction with other degrees of freedom) via the integral kernel operation. There are various choices for K_l , as discussed in [6]. For example:

- (1) Low-rank Neural Operator (LNO) in which K_l takes the form

$$K_l(z) = \sum_{j=1}^r \left(\psi^{(j)}, z \right) \varphi^{(j)}(x),$$

where $\varphi^{(i)}$ and $\psi^{(i)}$ are some parameterized functions;

- (2) Graph Neural Operator (GNO) in which

$$K_l = \frac{1}{|B(x, r)|} \sum_{y_i \in B(x, r)} k(x, y_i) z(y_i),$$

where $k(\cdot, \cdot)$ is a kernel function;

- (3) Fourier Neural Operator (FNO) maps z to Fourier space, followed by mapping the weighted Fourier modes back to the real space. Since FNO is the main focus of this article, it is discussed in more detail next. Readers are referred to [5,6] for further discussion of FNO and associated ideas.

In FNO, $K_l(z)$, for $z : D \rightarrow \mathbb{R}^{d_h}$, takes the following form:

$$K_l(z) = \mathcal{F}^{-1}(R_l \mathcal{F}(z)), \quad (51)$$

where

- $\mathcal{F}(z)$ is the Fourier transform applied to each component of the feature field $z(x)$ over the spatial domain. Only the first k_{max} modes are retained, so the output of $\mathcal{F}(z)$ is in $\mathbb{C}^{d_h \times k_{max} \times k_{max}}$.
- R_l applies the weighting to different Fourier modes. R_l is a complex-valued $d_h \times d_h \times k_{max} \times k_{max}$ trainable tensor and $R_l \mathcal{F}(z) \in \mathbb{C}^{d_h \times k_{max} \times k_{max}}$.
- $\mathcal{F}^{-1}(\cdot) \in \mathbb{R}^{d_h}$ is the inverse Fourier transform applied component-wise to reconstruct the feature field in physical space.

Figure 4 displays the FNO framework based on Fourier transforms. In practice, in the discrete setting, one evaluates the FNO output at all grid points to have

$$\mathbb{R}^{p_u} \ni \mathbf{u} = \mathbf{F}(\mathbf{m}) \approx \mathbf{F}_{NOp}(\mathbf{m}) := \mathbf{Q}(\mathbf{G}_L(\mathbf{G}_{L-1} \cdots \mathbf{G}_1(\mathbf{P}(\mathbf{m})))), \quad \text{for } \mathbf{m} \in \mathbb{R}^{p_M}. \quad (52)$$

The optimization problem to determine θ is given by

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^{p_\Theta}} \frac{1}{N_m} \sum_{I=1}^{N_m} \left\| \mathbf{u}^I - \mathbf{F}_{NOp}(\mathbf{m}^I) \right\|^2, \quad (53)$$

where p_Θ is the number of trainable parameters in FNO.

4.3.1. Implementation of FNO

The implementation of FNO requires function values at grid locations, and thus, preprocessing is required to obtain these values at grid points. Suppose $D_M = D_U = D = (0, 1)^2$ and consider the grid division of $\text{closure}(D)$ consisting of n_1 and n_2 number of points in x^1 and x^2 directions, respectively. Linear interpolation is used to map finite element solutions onto a structured grid required by FNO. The following describes the data:

1. Let $\mathbf{X}$ be an $N \times n_1 \times n_2 \times 3$ matrix, where the outer dimension corresponds to the number of samples. The element of $\mathbf{X}$ at l th outer index is a $n_1 \times n_2 \times 3$ matrix containing the interpolated values of a function m^l (or discretization of this function given by m^l on a mesh) at all grid (x^1, x^2) points and the coordinates x^1 and x^2 of all grid points; thus, the inner dimension is three. The spatial coordinates are included to allow the network to learn non-translation-invariant features. Next, the function values are centered and scaled using the mean and standard deviation computed from the samples $1 \leq l \leq N$.
2. Let $\mathbf{Y}$ be $N \times n_1 \times n_2 \times d_U$ matrix such that the element of $\mathbf{Y}$ at l th outer index is a $n_1 \times n_2 \times d_U$ matrix containing the interpolated values of d_U -valued function u^l (or discretization of this function given by u^l on a mesh) at all grid points. Note that the elements of $\mathbf{X}$ and $\mathbf{Y}$ at the l th outer index correspond to m^l and $u^l = F(m^l)$ on a grid, thereby establishing the correspondence between input and output. Function values at grid points are also centered and scaled using the sample mean and standard deviation.

Given a sample from $\mathbf{X}$, denoted $\mathbf{x} = (m^l(x^1, x^2), x^1, x^2) \in \mathbb{R}^3$, it is first lifted into $\mathbb{R}^{d_h}$, where $d_h > 3$ is a dimension of the input and output spaces of FNO layers, to get $z_0 = P(\mathbf{x}) = W_P \mathbf{x} + b_P$, where $W_P \in \mathbb{R}^{d_h \times 3}$ and $b_P \in \mathbb{R}^{d_h}$. This representation is applied pointwise at each grid location, and the full input consists of all such values arranged over the grid. Next, z_0 goes through L layers such that given z_{l-1} an output of the $(l-1)$ th layer, $z_l = G_l(z_{l-1})$. The Listing 9 presents the implementation of an FNO layer G_l based on the `Operator-Learning` repository (<https://github.com/Zhengyu-Huang/Operator-Learning>, accessed on 26 November 2024) [8]. The l th layer involves a linear transformation $z_l^1 = W_l z_{l-1} + b_l$ and Fourier-based transformation

$$z_l^2 = \mathcal{F}^{-1}(R_l \mathcal{F}(z_{l-1})),$$

where R_l is a complex matrix of size $d_h \times d_h \times k_{\max} \times k_{\max}$, $k_{\max}$ denoting the number of Fourier modes that are retained after the Fourier transform. Here, W_l and b_l are the weight and bias parameters, respectively. The output of the final layer $z_L = G_L(z_{L-1}) \in \mathbb{R}^{d_h}$ represents a d_h -dimensional feature field evaluated at all grid points and is projected to $u_{\text{NOP}}(x^1, x^2) = Q(z_L) \in \mathbb{R}^{d_U}$; the projection operator Q introduces $W_Q \in \mathbb{R}^{d_U \times d_h}$ and $b_Q \in \mathbb{R}^{d_U}$ parameters. In Listing 10, lift, FNO layer applications, and projection are combined to create an FNO model; see the `forward` method. The trainable parameters are as follows:

$$\theta := \left\{ (W_P, b_P) \in \mathbb{R}^{d_h \times 3} \times \mathbb{R}^{d_h}, \quad (W_Q, b_Q) \in \mathbb{R}^{d_U \times d_h} \times \mathbb{R}^{d_U}, \right. \\ \left. \left\{ W_l \in \mathbb{R}^{d_h \times d_h}, b_l \in \mathbb{R}^{d_h}, R_l \in \mathbb{C}^{d_h \times d_h \times k_{\max} \times k_{\max}}, \quad 1 \leq l \leq L \right\} \right\}. \quad (54)$$

It should be noted that extending the above to a vector-valued function (for a linear elasticity problem) as input and output is relatively straightforward.

Before writing the loss function, note that, while an input to FNO is a triplet $(m(x^1, x^2), x^1, x^2)$, during training and testing, the input is processed as a batch of samples, each defined over the full spatial grid. The FNO is applied to the input matrix $\mathbf{X}$ of size $N \times n_1 \times n_2 \times 3$ altogether, i.e., N samples of m^I and grid locations. FNO produces $N \times n_1 \times n_2 \times d_U$ outputs, corresponding to N number of d_U -valued functions u^I at all grid locations. Noting this, the optimization problem to train parameters θ reads as follows:

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^p \Theta} \frac{1}{N n_1 n_2} \sum_{l=1}^N \sum_{j=1}^{n_1} \sum_{k=1}^{n_2} \left\| u^I(x_{jk}^1, x_{jk}^2) - F_{NOp}(m^I)(x_{jk}^1, x_{jk}^2) \right\|^2. \quad (55)$$

Listing 9. Implementation of an FNO layer based on the Operator-Learning repository [8].

```
import torch
import torch.nn as nn
import torch.nn.functional as nnF

class FNO2DLayer(nn.Module):
    def __init__(self, in_channels, out_channels, modes1, modes2, apply_act, act):
        super(FNO2DLayer, self).__init__()
        ...
        # parameters in nonlocal transformation
        self.scale = (1 / (in_channels * out_channels))
        self.weights1 = nn.Parameter(self.scale * torch.rand(in_channels, \
                                                             out_channels, self.modes1, self.modes2, dtype = torch.cfloat))
        self.weights2 = nn.Parameter(self.scale * torch.rand(in_channels, \
                                                             out_channels, self.modes1, self.modes2, dtype = torch.cfloat))

        # parameters in linear transformation
        self.w = nn.Conv2d(self.out_channels, self.out_channels, 1)

    # complex multiplication
    def compl_mul2d(self, a, b):
        # (batch, in_channel, x,y ), (in_channel, out_channel, x,y)
        # -> (batch, out_channel, x,y)
        op = torch.einsum("bixy,ioxy->boxy", a, b)
        return op

    def fourier_transform(self, x):
        batchsize = x.shape[0]

        # compute Fourier coefficients
        x_ft = torch.fft.rfft2(x)

        # multiply retained Fourier modes
        out_ft = torch.zeros(batchsize, self.out_channels, x.size(-2), \
                             x.size(-1)//2 + 1, device=x.device, dtype=torch.cfloat)
        out_ft[:, :, :self.modes1, :self.modes2] = \
            self.compl_mul2d(x_ft[:, :, :self.modes1, :self.modes2], self.weights1)

        out_ft[:, :, -self.modes1:, :self.modes2] = \
            self.compl_mul2d(x_ft[:, :, -self.modes1:, :self.modes2], self.weights2)

        # return to physical space
        x = torch.fft.irfft2(out_ft, s=(x.size(-2), x.size(-1)))
        return x

    def linear_transform(self, x):
        return self.w(x)
```

Listing 9. Cont.

```

def forward(self, x):
    x = self.fourier_transform(x) + self.linear_transform(x)
    if self.apply_act:
        return self.act(x)
    else:
        return x

```

Listing 10. Core implementation of the FNO architecture.

```

...
from torch_fno2dlayer import FNO2DLayer

class FNO2D(nn.Module):
    def __init__(self, num_layers, width, fourier_modes1, fourier_modes2, \
        num_Y_components, save_file=None):
        super(FNO2D, self).__init__()
        ...
        # create hidden layers (FNO layers)
        self.fno_layers = nn.ModuleList()
        for _ in range(num_layers):
            self.fno_layers.append(FNO2DLayer(self.width, self.width, \
                self.fourier_modes1, self.fourier_modes2))

        # no activation in the last hidden layer
        self.fno_layers[-1].apply_act = False

        # define input-to-hidden projector: input has 3 components: m(x,y), x, y
        self.input_projector = nn.Linear(3, self.width)

        # define hidden-to-output projector
        self.output_projector = nn.Linear(self.width, self.num_Y_components)
        ...
    ...
    def forward(self, X):
        x = self.convert_np_to_tensor(X)

        # input-to-hidden projector
        x = self.input_projector(x)

        # rearrange x so that it has the shape (batch, width, x, y)
        x = x.permute(0, 3, 1, 2)

        # pass through hidden layers
        for i in range(self.num_layers):
            x = self.fno_layers[i](x)

        # rearrange x so that it has the shape (batch, x, y, width)
        x = x.permute(0, 2, 3, 1)

        # hidden-to-output projector
        x = self.output_projector(x)

        return x

    def train(self, train_data, test_data, batch_size, epochs, lr, ...):
        # similar to the train() of PCANet in Listing 8 and DeepONet in Listing 7

    def predict(self, X):
        # same as the predict() of PCANet in Listing 8

```


4.3.2. Architecture and Preliminary Results

In the implementation, three FNO layers are considered with hidden-layer output dimension $d_h = 22$. Other relevant parameters are listed in Table 2. Figures 5, A1 and A2 display FNO predictions for test input samples for the three model problems, together with the corresponding finite element solutions. FNO training and testing for the three problems are implemented in the following notebooks (repository [9], tag `survey26_v2`): FNO-Poisson, FNO-Linear Elasticity, and FNO-Hyperelasticity.

5. Neural Operators for Bayesian Inference

Bayesian inference is a powerful framework for solving inverse problems in which the goal is to infer unknown parameters or fields from observed data. In PDE-constrained inverse problems, posterior sampling procedures such as Markov chain Monte Carlo (MCMC) require repeated evaluations of the forward model, and the computational cost is often dominated by the associated likelihood evaluations. Neural operators can therefore serve as efficient surrogate forward models, accelerating these evaluations while maintaining acceptable approximation accuracy. The goal of this section is to demonstrate the integration of neural operator surrogates into the Bayesian inference framework and to evaluate the resulting inference performance.

We first formulate the Bayesian inverse problem in an abstract infinite-dimensional setting, following [11,13,18]. We then present the inference results for the Poisson problem in the main text, while the corresponding results for the linear elasticity and hyperelasticity models are provided in Appendix B. The neural operator architectures and training procedures are described in the previous section, and the trained models are used here as surrogate forward models within the Bayesian inference problems.

5.1. Abstract Bayesian Inference Problem in Infinite Dimensions

Consider a parameter field $m \in M$ to be inferred from observational data $g \in \mathbb{R}^{d_g}$ and the corresponding solution of the PDE, $u = F(m) \in U$. Let $\bar{\mathbf{B}} : U \rightarrow \mathbb{R}^{d_g}$ denote the state-to-observable map such that $\bar{\mathbf{B}}(u)$ gives the predicted observations corresponding to the state u . The Bayesian inverse problem is formulated through the relation

$$g = \bar{\mathbf{B}}(u) + \eta = \bar{\mathbf{B}}(F(m)) + \eta, \quad (56)$$

where $\eta \sim N(0, \Gamma_g)$ represents observational and modeling noise, and $\Gamma_g \in \mathbb{R}^{d_g \times d_g}$ is the covariance matrix. For simplicity, the covariance matrix is taken to be $\Gamma_g = \sigma_g^2 \mathbf{I}$, where σ_g^2 is the noise variance and $\mathbf{I}$ is the identity matrix.

In many applications, the admissible parameter space M_{ad} is a subset of M incorporating physical constraints. For example, diffusivity and Young's modulus must remain strictly positive. A transformation $m = m(w)$ with $w \in W$ is used to enforce the positivity constraint:

$$m(x) = \alpha_m \exp(w(x)) + \beta_m, \quad x \in D_W, \quad (57)$$

where α_m and β_m are constants. The inference problem is thus posed in terms of $w \in W$, where W is a Hilbert space.

Define the parameter-to-observable map $\mathbf{B} : W \rightarrow \mathbb{R}^{d_g}$ by

$$\mathbf{B}(w) = \bar{\mathbf{B}}(F(m(w))). \quad (58)$$

Using $\mathbf{B}$, the goal is to infer the probability distribution of w from

$$g = \mathbf{B}(w) + \eta. \quad (59)$$

Together with the noise model, the above equation defines the conditional probability distribution of the data $\mathbf{g}$ given w . Since the noise is assumed to be Gaussian, $\eta \sim N(0, \Gamma_g)$, the likelihood, i.e., the distribution of $\mathbf{g}$ conditioned on w , is also Gaussian: $\mathbf{g} \mid w \sim N(\mathbf{B}(w), \Gamma_g)$. Explicitly,

$$\pi_{\text{like}}(\mathbf{g} \mid w) = \frac{1}{\sqrt{\det(\Gamma_g)(2\pi)^{\frac{d_g}{2}}}} \exp\left[-\frac{1}{2}(\mathbf{g} - \mathbf{B}(w))^T \Gamma_g^{-1}(\mathbf{g} - \mathbf{B}(w))\right]. \quad (60)$$

It is useful to define the likelihood potential function

$$\Phi(w) := -\log \pi_{\text{like}}(\mathbf{g} \mid w), \quad (61)$$

where $\mathbf{g}$ is fixed, so that Φ is a function of w .

To solve (59), that is, to identify the distribution of w conditioned on the data $\mathbf{g}$ and the noise model, one must specify a prior measure μ^o on W that encodes prior knowledge. For example, this prior knowledge may include assumptions on the mean and covariance. Next, suppose the prior is Gaussian, $\mu^o = N(0, C)$, where the mean is zero and C is the covariance operator. With this Gaussian prior on w , the transformation (57) induces a log-normal prior on the parameter field $m \in M_{ad}$. The choice of C , α_m , and β_m reflects prior knowledge of m .

Bayes' rule relates the prior measure μ^o , the likelihood π_{like} , and the posterior measure μ^g , which is the distribution of w conditioned on the data $\mathbf{g}$, as follows:

$$\frac{d\mu^g}{d\mu^o}(w) = \frac{1}{Z} \pi_{\text{like}}(\mathbf{g} \mid w) = \frac{1}{Z} \exp(-\Phi(w)), \quad (62)$$

where Z is the normalizing constant given by

$$Z = \int_W \exp(-\Phi(w)) d\mu^o(w). \quad (63)$$

Bayes' rule (62) indicates how the posterior and prior measures are related. Given (62), it is straightforward to see that

$$\begin{aligned} \mathbb{E}_{\mu^g}[f] &= \int_W f(w) d\mu^g(w) \\ &= \int_W f(w) \frac{\exp(-\Phi(w))}{Z} d\mu^o(w) \\ &= \mathbb{E}_{\mu^o}\left[f(\cdot) \frac{\exp(-\Phi(\cdot))}{Z}\right]. \end{aligned} \quad (64)$$

The above identity shows that posterior expectations can be expressed as prior expectations weighted by the likelihood factor. In practice, direct evaluation of such expectations is generally not feasible in high or infinite dimensions, which motivates the use of sampling methods such as Markov chain Monte Carlo (MCMC). This is discussed next.

Markov Chain Monte Carlo (MCMC) Method to Sample from the Posterior Measure

To sample from the posterior measure μ^g (i.e., to generate samples $w \in W$ distributed according to μ^g), several algorithms are available (see [19] (Section 4) and [11]). This work uses the preconditioned Crank–Nicolson (pCN) method due to its simplicity. The pCN algorithm, based on [19], is as follows:

Preconditioned Crank–Nicolson (pCN) MCMC algorithm

1. Compute an initial sample $w^{(0)} \sim \mu^0 = N(0, C)$.
2. For $k = 0, 1, \dots, k_{mcmc} - 1$:
 - (a) Propose $v^{(k)} = w^{(k)} \sqrt{1 - \beta_{pCN}^2} + \beta_{pCN} \xi$, where $\xi \sim \mu^0$.
 - (b) Compute $a = \Phi(w^{(k)}) - \Phi(v^{(k)})$, and draw $b \sim \text{Uniform}[0, 1]$.
 - (c) If $\exp(a) > b$ (equivalently $a > \log(b)$), accept $v^{(k)}$ and set $w^{(k+1)} = v^{(k)}$. Otherwise, set $w^{(k+1)} = w^{(k)}$.
3. Discard the initial k_{burn} samples as burn-in, and use $\{w^{(k)}\}_{k=k_{burn}+1}^{k_{mcmc}}$ as samples from the posterior. The posterior mean is approximated by

$$\bar{w} = \frac{1}{k_{mcmc} - k_{burn}} \sum_{k=k_{burn}+1}^{k_{mcmc}} w^{(k)}. \quad (65)$$

In the above algorithm, β_{pCN} is the proposal hyperparameter. To verify that the acceptance rule behaves as intended, consider two cases. First, if the proposed sample $v^{(k)}$ has higher cost than the current sample $w^{(k)}$ (i.e., $\Phi(v^{(k)}) > \Phi(w^{(k)})$), then $a < 0$ and $\exp(a) < 1$, so the proposed sample may be accepted or rejected depending on the draw b . Second, if the proposed sample has lower cost than the current sample, then $a > 0$ and $\exp(a) > 1$, so the proposed sample is accepted regardless of b .

Listing 11 shows the MCMC implementation based on the pCN method. Functions `proposal` and `sample` contain the core pCN implementation. The listing also shows how surrogate models are used to compute the forward solution in the `solveFwd` function.

Listing 11. Core implementation of the pCN-based MCMC sampler.

```
...
import numpy as np
from scipy.interpolate import griddata
...
from state import State
from tracer import Tracer
...

class MCMC:
    def __init__(self, model, prior, data, sigma_noise, pcn_beta = 0.2, \
                 surrogate_to_use = None, surrogate_models = None, seed = 0):
        # model class that provides solveFwd()
        self.model = model
        ...
        # prior class that provides () and logPrior()
        self.prior = prior

        # data (dict) that provides x_obs, u_obs, m_true, u_true, etc.
        self.data = data

        # noise (std-dev) in the observations
        self.sigma_noise = sigma_noise

        # preconditioned Crank–Nicolson parameter
        self.pcn_beta = pcn_beta
        ...
        # current and proposed input parameter and state variables
        self.current = State(self.m_dim, self.u_dim, self.u_obs_dim)
        self.proposed = State(self.m_dim, self.u_dim, self.u_obs_dim)
```

Listing 11. Cont.

```

        self.init_sample = State(self.m_dim, self.u_dim, self.u_obs_dim)
        ...

        # surrogate models
        self.surrogate_to_use = surrogate_to_use
        self.surrogate_models = surrogate_models

        # tracer
        self.tracer = Tracer(self)
        self.log_file = None

    def solveFwd(self, current):
        if self.surrogate_to_use is not None:
            current.u = self.surrogate_models[self.surrogate_to_use].solveFwd(
                current.m)
        else:
            current.u = self.model.solveFwd(u = current.u, m = current.m,
                transform_m = True)

        return current.u

    def state_to_obs(self, u):
        # interpolate PDE solution on observation grid
        if self.u_comps == 1:
            return griddata(self.u_nodes, u, self.x_obs, method='linear')

        num_nodes = self.u_nodes.shape[0]
        u_xy = u.reshape(num_nodes, self.u_comps)
        obs_components = [griddata(self.u_nodes, u_xy[:, i], self.x_obs, \
            method='linear') for i in range(self.u_comps)]

        return np.column_stack(obs_components).reshape(-1)

    def logLikelihood(self, current):
        current.u = self.solveFwd(current)
        current.u_obs = self.state_to_obs(current.u)
        current.err_obs = current.u_obs - self.u_obs
        current.cost = 0.5 * np.linalg.norm(current.err_obs)**2 / self.sigma_noise
        **2
        current.log_likelihood = -current.cost

        return current.log_likelihood

    def logPosterior(self, current):
        current.log_prior = self.prior.logPrior(current.m)
        current.log_likelihood = self.logLikelihood(current)
        current.log_posterior = current.log_prior + current.log_likelihood
        return current.log_posterior

    def proposal(self, current, proposed):
        # preconditioned Crank-Nicolson
        proposed.m, proposed.log_prior = self.prior(proposed.m)
        return self.pcn_beta * proposed.m \
            + np.sqrt(1 - self.pcn_beta**2) * current.m

    def sample(self, current):
        # compute the proposed state
        self.proposed.m = self.proposal(current, self.proposed)
        self.proposed.log_posterior = self.logPosterior(self.proposed)

```

Listing 11. Cont.

```

# accept or reject based on the likelihood ratio
alpha = current.cost - self.proposed.cost
if alpha > np.log(np.random.uniform()):
    current.set(self.proposed)
    return 1

return 0

def run(self, init_m, n_samples, n_burnin, pcn_beta, sigma_noise, ...):
    ...
    # run the MCMC
    init_done = False
    for i in range(n_samples + n_burnin):
        # sample
        accept = self.sample(self.current)

        # postprocess/print
        self.process_and_print(i)

        if i < n_burnin:
            continue
        ...
        self.save(i, self.current, accept)

    # save the final state
    self.tracer.append(i, self.current, accept, force_save=True)

    # print message
    ...

```

5.2. Inference of Diffusivity in the Poisson Problem

This section considers Bayesian inference of the diffusivity field m in the Poisson problem using neural operators as surrogate forward models. For a given m , let $u = F(m)$ denote the finite element solution of the Poisson problem (20). The neural operator approximation is denoted by $F_{NOp}(m)$, where $NOp \in \{\text{DeepONet}, \text{PCANet}, \text{FNO}\}$. The inference problem is posed in terms of $w \in W$, with $m = \alpha_m \exp(w) + \beta_m$ used to enforce positivity of the diffusivity. The prior measure and synthetic data generation are described first, followed by the inference results.

5.2.1. Setup of the Forward Problem, Prior Measure, and Synthetic Data

The forward problem, prior measure, covariance operator, and transformation parameters are the same as those described in Section 3.1. Thus, $w \sim \mu_{\mathcal{Z}} = N(0, C)$ and m is obtained from w through (57).

Synthetic data is generated by selecting a fixed field w_{true} ; see w_{true} in the top panel of Figure 7. The corresponding true diffusivity field is obtained as $m_{true} = \alpha_m \exp(w_{true}) + \beta_m$, and the true solution is computed as $u_{true} = F(m_{true})$. The observational data $g_{true} \in \mathbb{R}^{d_g}$, with $d_g = 16^2$, is obtained by interpolating u_{true} on a 16×16 grid over $D_U = (0, 1)^2$. The top panel of Figure 7 also displays m_{true} , u_{true} , and g_{true} . The implementation can be found in the notebook `Generate_GroundTruth.ipynb` in the repository [9] (tag `survey26_v2`).

5.2.2. Inference Results

The MCMC parameters are $k_{mcmc} = 10,500$, $k_{burn} = 500$, $\beta_{PCN} = 0.2$, and $\sigma_g = 1.329$, where the covariance matrix in the noise model is $\Gamma_g = \sigma_g^2 I$. Here, σ_g is chosen as 5% of the mean of the observational data, i.e.,

$$\sigma_g = 0.05 \times \frac{1}{d_g} \sum_{i=1}^{d_g} g_{true_i}. \quad (66)$$

The same MCMC simulation is performed using four different forward models. In the first case, the state field u is computed using the finite element approximation of the Poisson problem, referred to here as the *true* forward model. In the remaining three cases, DeepONet, PCANet, and FNO are used as surrogate forward models. The trained neural operators from Section 4 are used, and their prediction accuracy for random samples from the prior is shown in Figure 5. The notebook `BayesianInversion.ipynb` (repository [9], tag `survey26_v2`) sets up the problem, loads the trained neural operators, and runs the MCMC simulations.

The acceptance rate and cost histories for the four MCMC simulations are shown in Figure 8. The inference results obtained using the *true* forward model, including the posterior mean and a representative posterior sample, are shown in Figure 7. The corresponding results obtained using DeepONet, PCANet, and FNO surrogates are shown in Figure 9.

Compared with the *true* forward model, all three neural operator surrogates produce posterior means of w with a slightly higher error. However, the error in the posterior mean of the transformed diffusivity field m remains approximately one percent. This indicates that neural operators provide effective surrogate forward models for the current Bayesian inference problem. This performance is aided by the fact that the ground-truth parameter field is close to the prior distribution used for training and inference. More substantial deviations from the training distribution would likely lead to larger surrogate-induced inference errors.

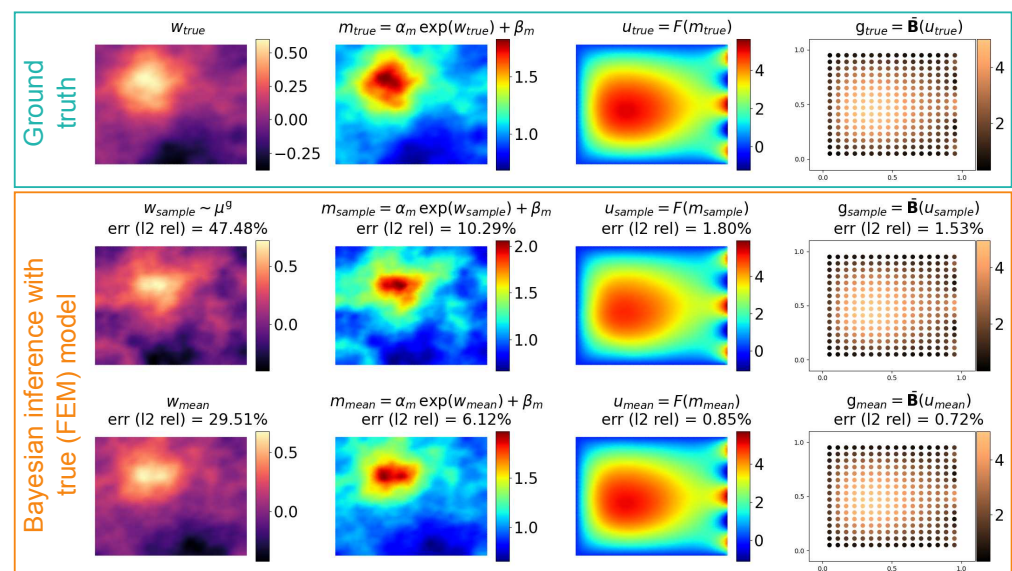


Figure 7. Bayesian inference of diffusivity in the Poisson problem using the *true* forward model (numerical approximation of the PDE). The top panel displays the synthetic field w_{true} , the corresponding diffusivity $m_{true} = \alpha_m \exp(w_{true}) + \beta_m$, the PDE solution $u_{true} = F(m_{true})$, and the observations $g_{true} = \mathbf{B}(u_{true})$. The observations are obtained by interpolating u_{true} on a 16×16 grid over $D_U = (0, 1)^2$, so that $g_{true} \in \mathbb{R}^{d_g}$ with $d_g = 16^2$. The lower panel displays a posterior sample w_{sample} drawn from the posterior measure μ^g (one realization from the MCMC chain) and the posterior mean w_{mean} defined in (65). The remaining columns display the corresponding diffusivity field m , the forward solution $u = F(m)$ used in the MCMC simulation, and the predicted observations $g = \mathbf{B}(u)$ associated with the corresponding w .

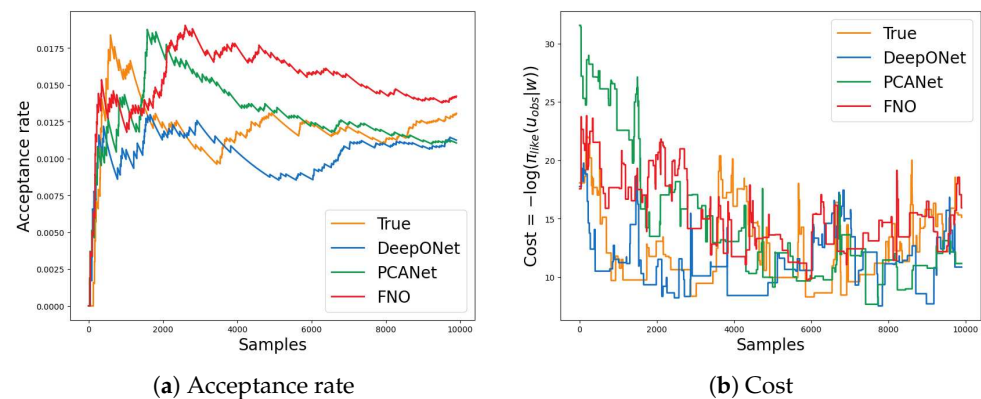


Figure 8. Acceptance rate and cost during MCMC simulation for the Poisson problem.

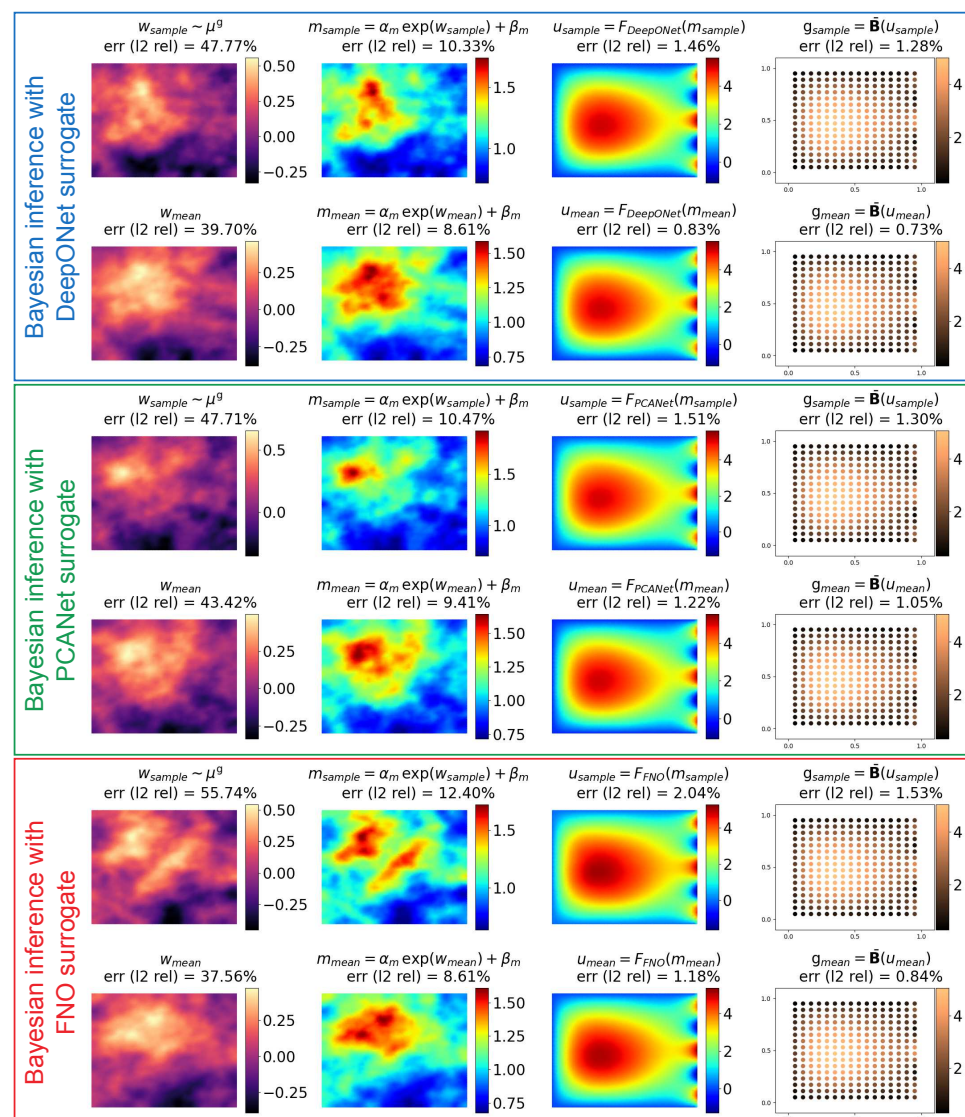


Figure 9. Comparison of Bayesian inference results for the Poisson problem using DeepONet, PCANet, and FNO surrogates. Each panel displays a posterior sample w_{sample} drawn from the posterior measure μ^g and the posterior mean w_{mean} . The corresponding diffusivity field m , forward solution u , and predicted observations $g = \tilde{\mathbf{B}}(u)$ are also shown, where F_{NOp} , $\text{NOp} \in \{\text{DeepONet}, \text{PCANet}, \text{FNO}\}$, is the neural operator approximation of the forward operator used in the MCMC simulation and $\tilde{\mathbf{B}}$ is the state-to-observable map. These results should be compared with the inference results obtained using the true forward model in Figure 7.

6. Neural Operator Performance Comparison

In this section, the performance of the trained neural operators is compared in terms of predictive accuracy for in-distribution and out-of-distribution input samples, as well as inference accuracy for Bayesian inverse problems. The architectures for the three neural operators are listed in Table 2; they are selected to have comparable numbers of trainable parameters, as indicated by p_{Θ} in Table 3. The Bayesian inference problems discussed in the previous section and the in- and out-of-distribution prediction tests considered here all use the same trained neural operators described in Section 4.

6.1. In-Distribution Performance

In this case, test samples are drawn from the same prior distribution as the training data for the neural operators in Section 4. Recall that, for all three problems, a log-normal prior is used to generate the training data. Specifically, a sample $w \sim N(0, C)$ is first drawn and then transformed to m through $m = \alpha_m \exp(w) + \beta_m$. The parameters used to generate the training data are listed below for reference:

Prior parameters for training data

Common parameters: $a_c = 0.005$, $b_c = 1$, $c_c = 0.2$,

Poisson problem: $\alpha_m = 1$, $\beta_m = 0$,

Linear and hyperelasticity problems: $\alpha_m = 100$, $\beta_m = 1000$.

The rows corresponding to `In-distribution error` in Table 3 are computed by taking the mean relative ℓ^2 -error over $N_{test} = 800$ test samples. For the Poisson problem, all three neural operators achieve mean errors below 1.5%, with FNO achieving the smallest error. For the two elasticity problems, DeepONet and FNO yield errors close to 0.03%, while PCANet yields errors that are approximately one order of magnitude smaller.

6.2. Out-of-Distribution (OOD) Performance

For OOD tests, two priors are considered, where the second prior is more distant from the training distribution than the first. In both cases, the covariance operator parameters are kept the same as those used to generate the training data, while the transformation parameters are changed. The transformation parameters for the two OOD priors are listed below.

Case 1 OOD prior parameters

Poisson problem: $\alpha_m = 1$, $\beta_m = 1$,

Linear and hyperelasticity problems: $\alpha_m = 100$, $\beta_m = 1500$.

Case 2 OOD prior parameters

Poisson problem: $\alpha_m = 2$, $\beta_m = 1$,

Linear and hyperelasticity problems: $\alpha_m = 1000$, $\beta_m = 1000$.

For both OOD cases, 100 samples are drawn, and the mean relative percentage ℓ^2 errors are reported in Table 3. The rows corresponding to `Out-of-distribution error (Case 1)` show that the errors increase substantially relative to the in-distribution case. For the Poisson problem, the errors range from about 13.5% to 34.1%, while for the elasticity problems, the errors are generally between about 13% and 25%. In Case 2, the errors

increase further, as expected, since this prior is farther from the training distribution. In this case, all three neural operators produce errors above 100% for the Poisson problem, above 65% for linear elasticity, and above 50% for hyperelasticity.

In Figure 10, representative OOD input samples and the corresponding forward and surrogate solutions for the Poisson problem are shown. The OOD samples correspond to Case 2. Similar plots for the elasticity problems under Case 2 OOD sampling are provided in Figures A3 and A4 of Appendix A. These figures show that the surrogate solutions deviate significantly from the true finite element solutions, which is consistent with the large errors reported in Table 3.

Table 3. Summary of neural operator performance for in-distribution input samples, out-of-distribution input samples, and Bayesian inference. The row below True lists the Bayesian inference errors obtained using the *true* forward model and provides benchmark values for assessing the neural operator surrogate results in the following rows.

Metric	Poisson	Linear Elasticity	Hyperelasticity
True			
Bayesian inference error	29.5%	21.1%	19.7%
DeepONet			
Trainable Params p_{Θ}	188,040	194,540	194,540
In-distribution error	1.319%	0.034%	0.032%
Out-of-distribution error (Case 1)	31.814%	18.064%	14.007%
Out-of-distribution error (Case 2)	186.6%	66.778%	52.654%
Bayesian inference error	39.7%	21.6%	21.9%
PCANet			
Trainable Params p_{Θ}	180,700	180,700	180,700
In-distribution error	0.884%	0.008%	0.007%
Out-of-distribution error (Case 1)	34.111%	17.551%	14.118%
Out-of-distribution error (Case 2)	202.861%	65.929%	52.973%
Bayesian inference error	43.4%	23.8%	21.5%
FNO			
Trainable Params p_{Θ}	187,485	187,508	187,508
In-distribution error	0.575%	0.028%	0.027%
Out-of-distribution error (Case 1)	13.512%	24.9%	13.238%
Out-of-distribution error (Case 2)	136.091%	96.265%	69.212%
Bayesian inference error	37.6%	22.2%	21.4%

6.3. Bayesian Inference Error

The Bayesian inference problems and results have been described in the previous section. Here, the main inference errors are included alongside the prediction errors to provide an overall performance comparison. Suppose w_{true} is the true discretized parameter field and w_{post} is the posterior mean obtained from MCMC sampling using either a neural

operator surrogate or the *true* finite element model. The relative ℓ^2 percentage error is defined as

$$\text{Inference error} = \frac{\|w_{\text{post}} - w_{\text{true}}\|}{\|w_{\text{true}}\|} \times 100\%.$$

The rows corresponding to Bayesian inference error in Table 3 show the inference error for the *true* model and the three neural operator surrogates. The *true* forward model gives the smallest inference error, as expected. The three surrogates give inference errors reasonably close to the errors obtained using the *true* model, although PCANet performs relatively poorly for the Poisson problem. One reason the surrogates perform reasonably well is that the ground-truth parameter field w_{true} is close to the training distribution and the prior used in Bayesian inference is the same as that used to generate the training data. If a significantly different w_{true} or a different prior were considered, the inference error from the surrogates would likely increase relative to that from the *true* model.

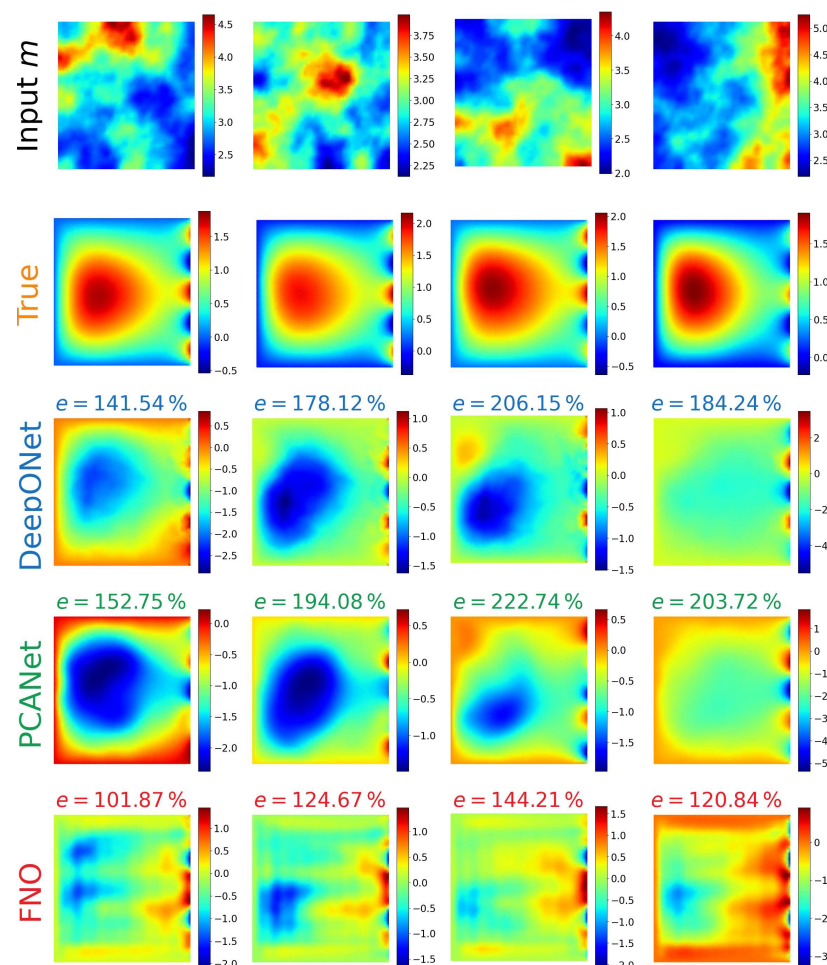


Figure 10. Representative out-of-distribution input samples and corresponding forward and surrogate solutions for the Poisson problem. Here, e denotes the relative percentage ℓ^2 error between the true finite element solution and the surrogate solution. The OOD samples correspond to Case 2 in Section 6.2.

7. Perspectives and Conclusions

While the focus of this article has been on the foundational aspects of operator learning and their practical implementation, the subsections below briefly review recent developments in neural operators, with particular emphasis on the growing family of architectures, emerging foundation models for PDEs, and the critical issue of controlling prediction accuracy.

7.1. The Growing Field of Neural Operators

The field of neural operators continues to evolve through a range of frameworks that adapt the basic operator learning idea to different discretizations, physical constraints, geometries, and application settings. One direction is to incorporate additional physical or differential information during training. Physics-Informed Neural Operators (PINOs) include the residual of the governing PDE in the loss function, in addition to losses associated with model outputs and data mismatch [1,20,21]. Derivative-Informed Neural Operators (DINOs) [22] incorporate derivative information into the training process to improve learning efficiency and accuracy, particularly for problems where gradients play a dominant role.

Another line of work concerns the representation of the nonlocal operator and the underlying domain. Graph Neural Operators (GNOs) [6] are based on graph discretizations of the spatial domain and are therefore well-suited for problems defined on irregular geometries. Their structure is similar to that of the FNO, with a different form of the nonlocal kernel operator. The Wavelet Neural Operator (WNO) [23] follows a related idea, in which the nonlocal operation is defined using wavelet transforms. The Laplace Neural Operator [24] extends operator learning to transient problems and non-periodic domains. Other works use attention-based or transformer-type parameterizations of the operator map, including GNOT [25], the operator transformer of [26], and factorized transformer models such as FactFormer [27]. These approaches broaden the class of admissible input representations beyond the regular-grid setting for which classical FNO is most natural.

Multi-resolution and multi-scale architectures form another important direction. U-NO [28] introduces a U-shaped encoder–decoder structure for deeper operator networks, while Factorized FNO [29] aims to improve parameter efficiency and scalability for high-resolution PDEs. Multiwavelet constructions [30] provide a principled way to separate coarse and fine scales and have been effective for multi-scale and coupled systems.

Geometry and domain complexity have also motivated several extensions. Local neural operators [31] and diffeomorphism neural operators [32] aim to preserve discretization flexibility while improving transfer across domains and boundary configurations. In [33], a deep learning-driven domain decomposition (DLD³) method is proposed, where an FNO is trained to learn the solution map on smaller subdomains of a larger problem, thereby allowing the framework to be applied to different geometries and boundary conditions. Recently, the Kolmogorov–Arnold Neural Network (KAN) has motivated alternative parameterizations of nonlinear maps [34].

There has also been considerable effort to extend neural operator frameworks to Bayesian settings, where uncertainty is modeled through probabilistic formulations of the learned operators; see [35–38]. These approaches aim to provide probabilistic predictions that reflect uncertainty in both the data and model parameters, alongside point estimates.

Digital twins are another application area where neural operators may be useful, especially when repeated state predictions are needed under varying model or control parameters. Digital twin frameworks are expanding due to advances in computational capacity, multi-physics modeling, sensing, and control technologies; see [39,40]. Neural operators can serve as fast surrogate models for predicting system states within such frameworks [41].

Neural operators have also demonstrated potential in Bayesian inverse problems, optimization, and control. In Bayesian inverse problems, they can accelerate posterior sampling by replacing expensive PDE solves with surrogate evaluations [42–44]. In optimization and control problems, the primary computational bottleneck is often the repeated evaluation of the forward solution for varying parameter fields [45]. A major challenge in these settings is the lack of reliable mechanisms for predicting surrogate accuracy. Training data is typically

generated from a prior distribution, whereas optimization and control procedures may require evaluations far outside this regime. Ref. [10] demonstrates that neural operators trained on a prior distribution can perform poorly when used within optimization loops. This issue is discussed in more detail in the next subsection.

7.2. Controlling Neural Operator Prediction Accuracy

Neural operators can provide rapid surrogate evaluations, but their practical use depends on whether their prediction error can be estimated, controlled, or corrected in regimes relevant to the downstream task. The ability to predict and control accuracy is a central issue in uncertainty quantification, Bayesian inference, optimization, and control problems, where small forward-model errors may propagate through repeated evaluations. Several key works in this direction are reviewed below.

The multi-level neural network framework [46] iteratively refines solutions by training successive networks to minimize residual errors from previous levels. This hierarchical approach progressively reduces approximation error. By capturing high-frequency components in subsequent networks, this framework addresses limitations of standard neural network approximations, which can have low sensitivity to higher-frequency modes. The multi-stage neural network framework [47] addresses convergence plateauing in deep learning by dividing training into sequential stages, with each stage fitting the residual from the previous one. This approach progressively refines the approximation. The methods in [46,47] are reported to achieve errors approaching machine precision in controlled settings.

The Galerkin neural network framework [48] integrates the classical Galerkin method with neural networks, using the neurons in a neural network layer as basis functions to approximate variational equations. It adaptively adds new neurons to a single-layer neural network, with each neuron corresponding to a basis function. By adding more neurons, the dimension of the approximation increases in a way that produces less error than lower-dimensional subspaces.

The residual-based error correction method [10,42], illustrated in Figure 11, builds on the idea of using lower-fidelity solutions to estimate modeling errors [49]. It treats the neural operator prediction as an initial guess and solves a variational problem to correct the residual error. This approach improves accuracy in applications like Bayesian inverse problems and topology optimization, where small errors can propagate and amplify. The trade-off in the residual-based error correction method is the introduction of a linear variational problem on the modeling error field, which must be solved and added to the neural operator prediction to obtain the improved prediction. For challenging nonlinear problems, the approach leads to significant speedups. In Bayesian settings, it enhances posterior estimates without increasing computational cost [42]. For the example optimization problem of seeking a diffusivity field in the Poisson equation that minimizes the compliance, neural operators, when used as surrogates for the forward problem, produced minimizers with significant errors (about 80%). Here, the error is defined as the norm of the difference between the minimizer obtained using the true forward model and that obtained using the surrogate. Neural operator surrogates with the residual-based error correction produced minimizers with errors below 6%; see Figure 12, where the optimized diffusivities obtained using the *true* model and the surrogates are compared. A complementary direction is PhysicsCorrect [50], a training-free correction framework that stabilizes neural PDE simulations by enforcing physics-based consistency at inference time. It is conceptually aligned with residual-based correction strategies such as [10], but avoids retraining and introduces only modest additional overhead.

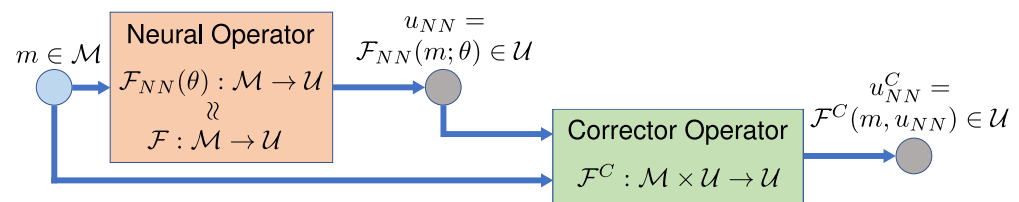


Figure 11. Residual-based error correction of neural operator predictions [10,42]. The corrector problem is a linear boundary value problem, and if the predictor u_{NN} is sufficiently close to the PDE solution u , the corrector gives quadratic error reduction [10].

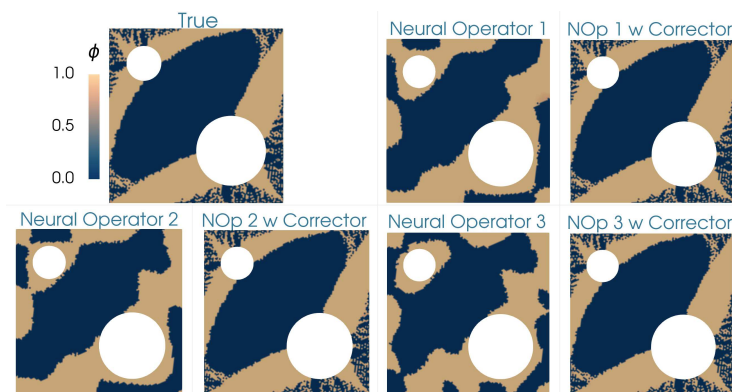


Figure 12. Optimized diffusivity in a Poisson equation obtained using the *true* model and neural operator surrogates with and without a corrector. Here, the forward problem is posed on a square domain with two circular voids. Zero Dirichlet boundary conditions are imposed on the internal void boundaries, and a constant flux is prescribed on the outer boundaries. The optimal diffusivity is obtained by minimizing the compliance, defined as the boundary integral of the product of flux and temperature over the flux boundary. For more details about the problem and results, see [10] (Section 4.3).

Recent work also makes it clear that controlling prediction accuracy is not only a matter of training larger models. *A posteriori* and residual-based indicators must be made reliable in norms that are meaningful for the PDE under consideration. In this direction, rigorous error analyses for DeepONet and FNO-type models [51,52] provide an important theoretical baseline, while newer work on neural functional *a posteriori* error estimates [53] aims to construct computable error majorants that can be used during or after training. These developments are significant because small empirical losses or averaged test errors do not, by themselves, provide reliable guarantees about the error of a specific prediction relevant to an optimization, inference, or control query.

Adaptive strategies are also playing an increasingly important role. The framework in [44] locally refines a pretrained surrogate in regions most relevant to posterior exploration. This is conceptually appealing because high accuracy is rarely required uniformly over the entire input space. Instead, one often seeks high fidelity only on a dynamically discovered subset of the parameter regime. Multi-fidelity and multi-level formulations provide another route to balancing cost and accuracy. Multi-fidelity deep neural operators [54], multi-fidelity DeepONets [55], and application-oriented multi-fidelity operator fusion strategies [56] exploit the fact that coarse simulations, reduced models, or monitoring data may be much cheaper to obtain than uniformly high-fidelity labels.

Out-of-distribution behavior remains especially challenging. PDEBench- and PDEArena-style evaluations have shown that generalization across coefficients, resolutions, time horizons, and physical regimes is uneven across architectures [57,58]. Theoretical work on out-of-distribution risk bounds [59], operator complexity [60,61], and data complexity [62] reinforces the same message from another direction: one should not expect uniformly reliable accuracy over broad classes of operators without either exploiting stronger problem

structure or incurring additional cost in data, model complexity, or online correction. Related results on DeepONet size lower bounds and neural scaling laws [63,64] further indicate that practical accuracy limits are tied not only to optimization but also to intrinsic approximation and sample-complexity barriers.

Therefore, the current picture is that improvements in accuracy are possible through residual correction, adaptive refinement, multi-level training, multi-fidelity learning, and stronger theory, but each mechanism introduces a cost–accuracy trade-off. Extra linear solves, extra training stages, local fine-tuning, larger pretrained backbones, and more structured loss functions can all improve fidelity, yet they reduce some of the computational speed advantage that originally motivated neural operators. For many downstream applications, this could still be a favorable trade-off, but it highlights why predictability of accuracy, rather than average-case speed alone, remains the central challenge.

The works reviewed above highlight several important limitations. First, methods that substantially improve the accuracy of neural operator predictions generally introduce additional computation, for example, by adding extra training stages, adaptive updates, or the solution of correction problems. Second, the relevant notion of accuracy is usually application-dependent; an error level acceptable for forward prediction may be unusable for optimization, posterior inference, or control. Third, distributional shifts remain unavoidable in many realistic workflows, and current error indicators are still far from being fully calibrated under such shifts. Addressing these issues will likely require tighter integration between operator learning, adaptive numerical analysis, and application-specific notions of goal-oriented error.

7.3. Foundation Models for PDEs

A prominent emerging trend in scientific machine learning is the construction of pretrained models that are not tied to a single PDE family, geometry, or discretization, but instead aim to learn transferable representations across broad classes of operator learning tasks. In the context of PDEs, this direction includes multi-physics pretraining [65], unified PDE solvers [66], Universal Physics Transformers [67], large-scale pretrained operator transformers such as DPOT [68], and PDEformer-style models that condition directly on a representation of the equation itself [69,70]. Related work such as OmniArch [71] further suggests that large-scale operator pretraining can improve sample efficiency, few-shot transfer, and robustness across related tasks. Compared to task-specific neural operators, these approaches seek more transferable latent representations of PDE dynamics, boundary interactions, and geometric variation.

At present, most PDE foundation models are predominantly data-driven. Their training typically relies on large collections of simulation trajectories drawn from multiple PDE families, resolutions, and parameter ranges [65,68,72]. In contrast, physics-informed variants often enter through residual regularization, alignment objectives, or fine-tuning rather than replacing large-scale simulation data altogether [71,73]. This distinction is important. Data-driven pretraining is currently the main mechanism for obtaining broad coverage across multiple equations and regimes, while physics-informed components are more often used to improve data efficiency, enforce consistency, or adapt a pretrained model to a smaller downstream dataset. In that sense, the present landscape is less about a single “foundation architecture” and more about a family of pretrained operator learning systems that differ in how they encode PDE structure, geometry, and physical constraints.

The potential applications of such models are substantial. For rapid surrogate modeling across parametric families, a pretrained operator can amortize the cost of repeated PDE solves over many-query settings. This is directly relevant to inverse problems, design optimization, control, uncertainty propagation, and real-time digital twins, where

the cost bottleneck is rarely a single forward solve but rather a large sequence of related solves [39,40,45]. A sufficiently transferable pretrained operator could also enable cross-domain transfer, for example, by moving from one geometry family to another, from one boundary-condition regime to another, or from one PDE class to a related coupled system with comparatively little task-specific retraining [66,67]. Recent scaling efforts such as AB-UPT [74] and emerging multi-physics pretraining frameworks [75] indicate that this direction is especially attractive for industrial simulation settings where geometry diversity and repeated evaluations dominate runtime.

At the same time, foundation models for PDEs raise difficult technical questions. The first is the lack of reliable error bounds. Even when zero-shot or few-shot performance is strong on held-out benchmarks, there is rarely a computable guarantee on the error for a new geometry, a new boundary condition, or a new coefficient regime. The second challenge is generalization outside the training regime. Broader pretraining improves average transfer performance, but it does not remove the need to reason carefully about out-of-distribution inputs, unseen physical couplings, or extrapolative parameter settings [10,42]. The third challenge is scale: higher-dimensional PDEs, strongly coupled multi-physics systems, and industrial geometries still place severe demands on both model capacity and training data. Benchmark ecosystems such as PDEBench, PDEArena, and The Well [57,58,72] help standardize evaluation, but they also make clear that data generation remains expensive and potentially biased toward the numerical solvers, regimes, and discretizations used to create the datasets.

There are also challenges of interpretability and physics consistency. A pretrained model that performs well across many systems may still obscure which physical mechanisms it has learned. It may also be unclear which conservation laws are embedded in the architecture and when a prediction should trigger solver-based verification. For this reason, foundation models for PDEs are unlikely to replace classical numerical methods outright. A more realistic direction is tight coupling with numerical solvers through warm starts, residual correction, adaptive verification, and solver-in-the-loop refinement. In that workflow, pretrained operator models provide fast approximate maps over broad classes of problems, while numerical analysis provides the local certification and correction required for high-consequence decisions. From the perspective of scientific computing, this hybrid view appears more plausible than a purely black-box replacement of established PDE solvers.

7.4. Final Thoughts

Neural operators have established themselves as powerful surrogates for parametric PDEs, offering a different computational paradigm in which mappings between function spaces are learned once and evaluated rapidly. This shift has clear advantages in many-query settings such as uncertainty quantification, Bayesian inference, optimization, and control, where the dominant cost lies in repeated forward solves rather than a single high-fidelity simulation.

At the same time, the core limitation is now well-understood: accuracy is not inherently predictable in the same way as in classical numerical methods. Neural operators do not provide built-in guarantees tied to discretization, mesh refinement, or variational structure. This gap becomes critical in downstream tasks where small errors can lead to qualitatively incorrect decisions, particularly in optimization, inverse problems, and control.

Recent advances suggest that this limitation is not insurmountable, but it requires a shift in perspective. Accuracy must be actively controlled rather than passively expected. Residual-based correction, adaptive refinement, multi-level training, and multi-fidelity formulations all point toward a paradigm in which neural operators provide fast approxi-

mate predictions, while additional mechanisms ensure reliability in task-relevant regimes. The emerging theme is that neural operators are most effective when embedded within a broader computational workflow rather than used as standalone replacements for PDE solvers.

Looking forward, three directions appear particularly important. First, developing reliable, computable, and meaningful error indicators for specific applications remains a central challenge. Second, scalable strategies for handling distributional shifts, especially in high-dimensional, multi-physics settings, are essential for real-world deployment. Third, integrating pretrained operator models with physics-based correction and solver-in-the-loop verification offers a promising path to combining generality with reliability. These observations suggest that progress in neural operators will depend less on isolated architectural advances and more on principled integration with numerical analysis and application-driven error control.

In this sense, the future of neural operators is likely to involve data-driven operator learning augmented by physics-based correction, adaptive numerical analysis, problem-specific validation, and goal-oriented error certification. Advancing this integration will be key to realizing the full potential of neural operators in scientific computing.

Funding: This work was supported in part by the National Science Foundation under the Engineering Research Initiation (ERI) award NSF #2502279. The author also acknowledges support from the South Dakota Board of Regents Competitive Research Grant (SDBOR CRG) program.

Data Availability Statement: Restrictions apply to the availability of these data. Data for this article was generated from the repository [9] (tag survey26_v2) by the author using their laptop and workstation and is made available in [<https://www.dropbox.com/scl/fo/5dg02otewg7j0bt7rhkuf/AOfAAc2SaWOgO-Yg25lITxs?rlkey=t900gee8y8z327y5f8wu4yc9&st=t9c8qimk&dl=0>] (accessed on 1 July 2026)] with the permission of the author.

Acknowledgments: The author dedicates this work to the late J. Tinsley Oden, to whom he is deeply grateful for his mentorship and support during his four years as a postdoctoral fellow and research associate at the University of Texas at Austin.

Conflicts of Interest: The author declares no conflict of interest.

Appendix A. Neural Operator Predictions for Elasticity Problems

This appendix collects additional neural operator prediction results for the linear elasticity and hyperelasticity problems. The three neural operator architectures, DeepONet, PCANet, and FNO, are described in Section 4. Their implementation details are outlined in Sections 4.1.2, 4.2.2, and 4.3.2, respectively, and the neural network parameters are listed in Table 2.

Figures A1 and A2 compare the finite element solutions with predictions obtained using DeepONet, PCANet, and FNO for four samples of Young's modulus in the linear elasticity and hyperelasticity problems, respectively. The relative percentage ℓ^2 error is also shown in each figure. For the elasticity problems, DeepONet and FNO yield errors close to 0.04%, while PCANet achieves errors that are approximately one order of magnitude smaller. This high accuracy also enables these surrogates to perform well in Bayesian inference problems, as shown in Appendix B. However, the next subsection shows decreased predictive performance for OOD samples.

Neural Operator Predictions for Out-of-Distribution Samples

In Figures A3 and A4, representative out-of-distribution input samples and the corresponding forward and surrogate solutions for the linear elasticity and hyperelasticity problems are shown, respectively. The OOD samples correspond to Case 2 in Section 6.2. The figures show that the surrogate solutions deviate significantly from the true finite element solutions, which is consistent with the large errors reported in Table 3.

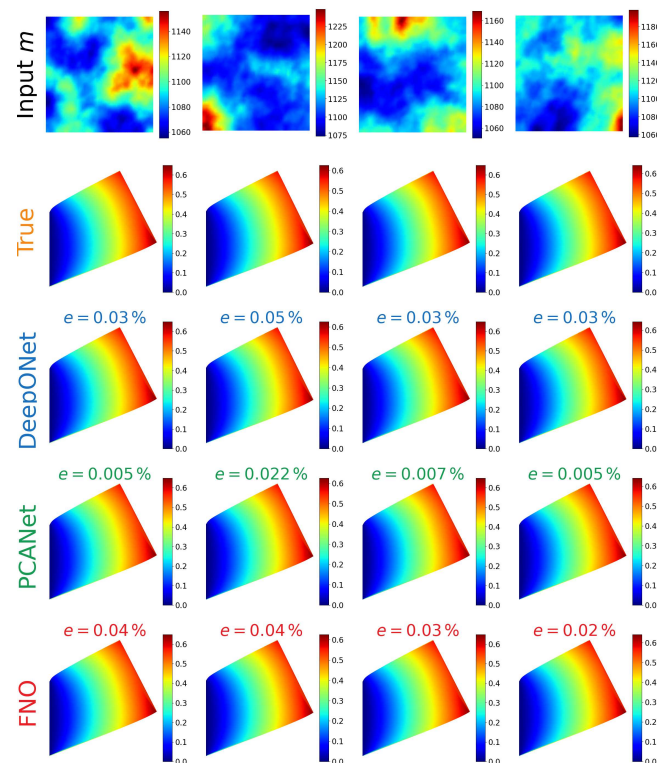


Figure A1. Comparison of DeepONet, PCANet, and FNO predictions with the finite element solution for four samples of the Young's modulus field in the linear elasticity problem. Here, e denotes the relative percentage ℓ^2 error between the true finite element solution and the surrogate solution. The displacement field u is visualized by plotting the deformed configuration of the domain with coordinates $z = x + u(x)$ for $x \in D_U$. The color map indicates the displacement magnitude $|u(x)|$.

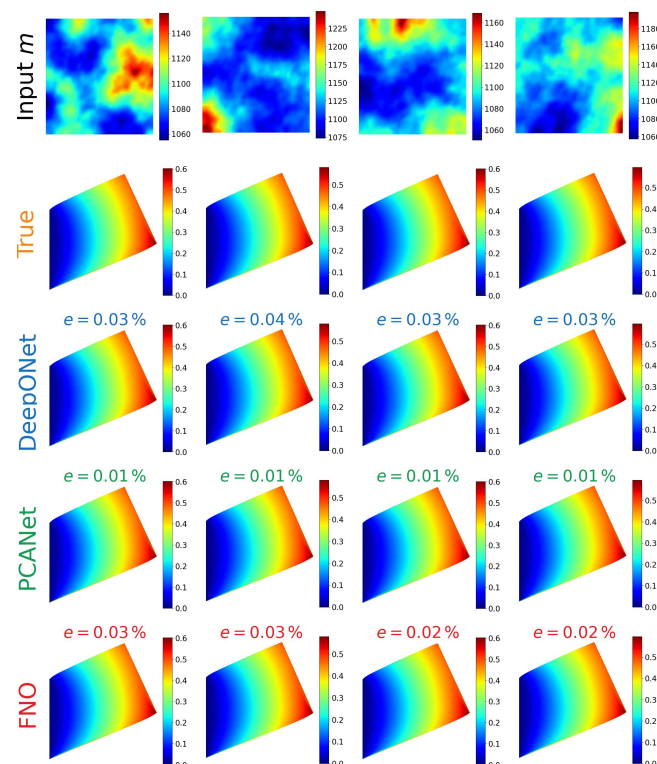


Figure A2. Comparison of DeepONet, PCANet, and FNO predictions with the finite element solution for four samples of the Young's modulus field in the hyperelasticity problem. The notation and visualization follow Figure A1.

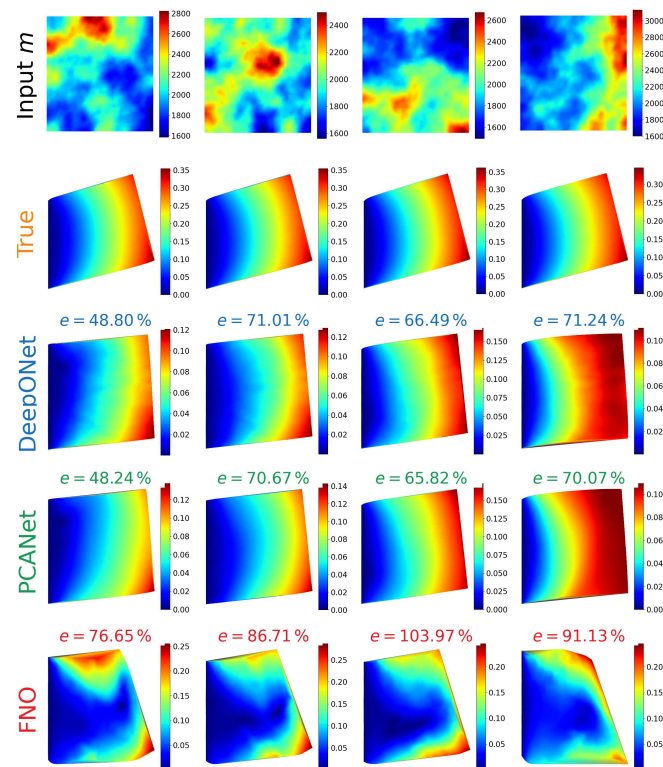


Figure A3. Representative out-of-distribution input samples and corresponding forward and surrogate solutions for the linear elasticity problem. The OOD samples correspond to Case 2 in Section 6.2. The notation and visualization follow Figure A1.

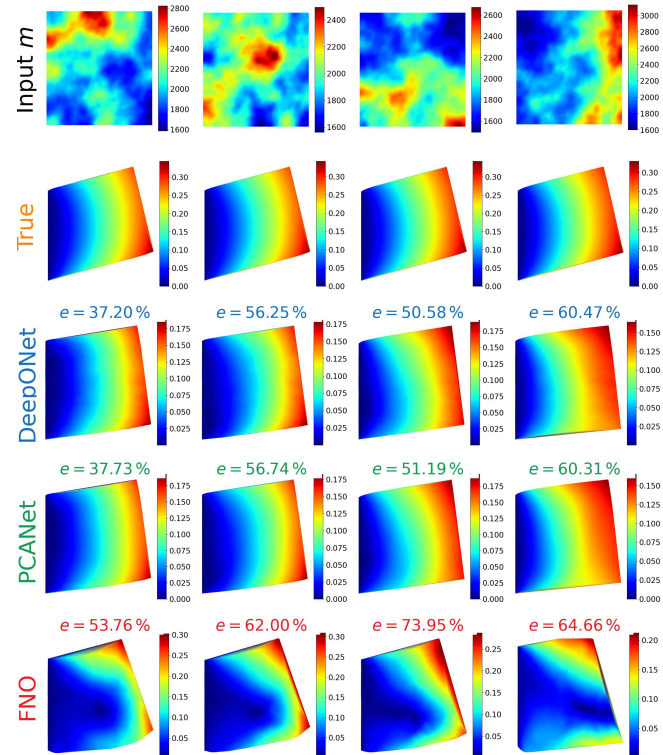


Figure A4. Representative out-of-distribution input samples and corresponding forward and surrogate solutions for the hyperelasticity problem. The OOD samples correspond to Case 2 in Section 6.2. The notation and visualization follow Figure A1.

Appendix B. Bayesian Inference of Young's Modulus in the Linear Elasticity and Hyperelasticity Problems

This section presents the use of neural operators as surrogates in the Bayesian inference of the Young's modulus field m in two elasticity problems. As in the Poisson case, positivity of Young's modulus is enforced by posing the inference problem in terms of the function $w \in W$, with $m = \alpha_m \exp(w) + \beta_m$.

Appendix B.1. Setup of the Forward Problem, Prior Measure, and Synthetic Data

The setup of the forward problem is the same as in Section 3.2 for linear elasticity and Section 3.3 for hyperelasticity. The Gaussian prior measure μ^0 on W is the same as $\mu_{\mathcal{Z}}$ used to generate the training data for the neural operators. The values of α_m and β_m , along with the covariance operator parameters, are given in Sections 3.2 and 3.3 for the two elasticity problems.

The data is generated synthetically following the same procedure as in the Poisson problem; see Section 5.2.1. In the elasticity problems, the data at each observation point consists of two displacement components, and therefore $g \in \mathbb{R}^{d_g}$ with $d_g = 2 \times 16^2$. The top panels of Figures A6 and A9 display the synthetic data for the linear elasticity and hyperelasticity problems, respectively. The implementations can be found in the notebooks

Appendix B.2. Inference Results for the Linear Elasticity Problem

The MCMC parameters are $k_{mcmc} = 10,500$, $k_{burn} = 500$, $\beta_{pCN} = 0.15$, and $\sigma_g = 0.0411$. Here, σ_g is chosen as 1% of the mean of g . The notebook `BayesianInversion.ipynb` in the repository [9] (tag `survey26_v2`) sets up the problem, loads the trained neural operators, and runs the MCMC simulations.

The acceptance rate and cost histories for the four MCMC simulations corresponding to the *true* and surrogate forward models are shown in Figure A5. The results obtained using the *true* forward model, including the posterior mean and a representative posterior sample, are shown in Figure A6. The results obtained using DeepONet, PCANet, and FNO surrogates are shown in Figure A7.

In this case, the posterior means obtained using the *true* forward model and the surrogate models are very similar. The error in the posterior mean of m for both the *true* model and the surrogates is less than one percent. Notably, the synthetic field w_{true} exhibits higher values along the diagonal, and the posterior mean successfully captures this variation.

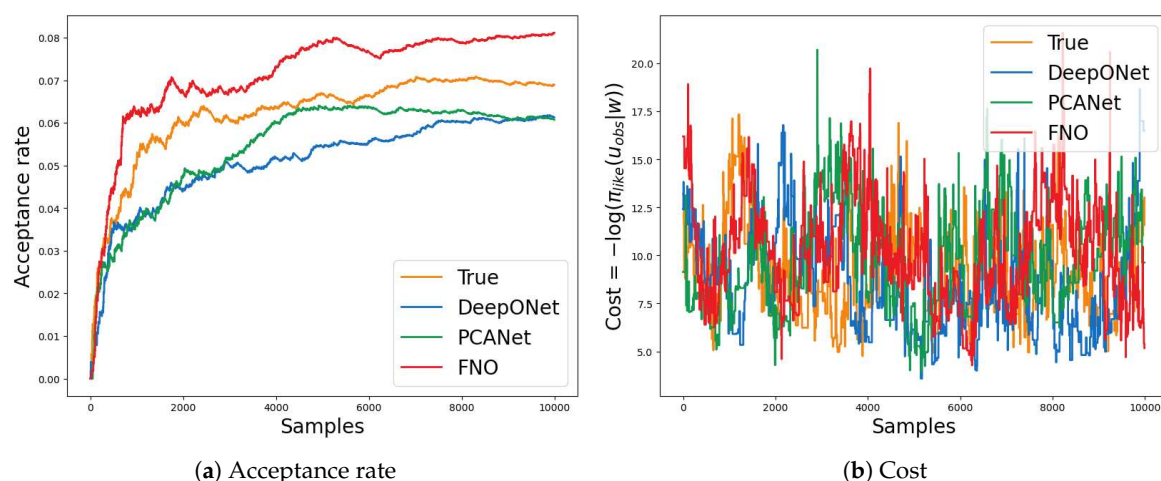


Figure A5. Acceptance rate and cost during MCMC simulation for the linear elasticity problem.

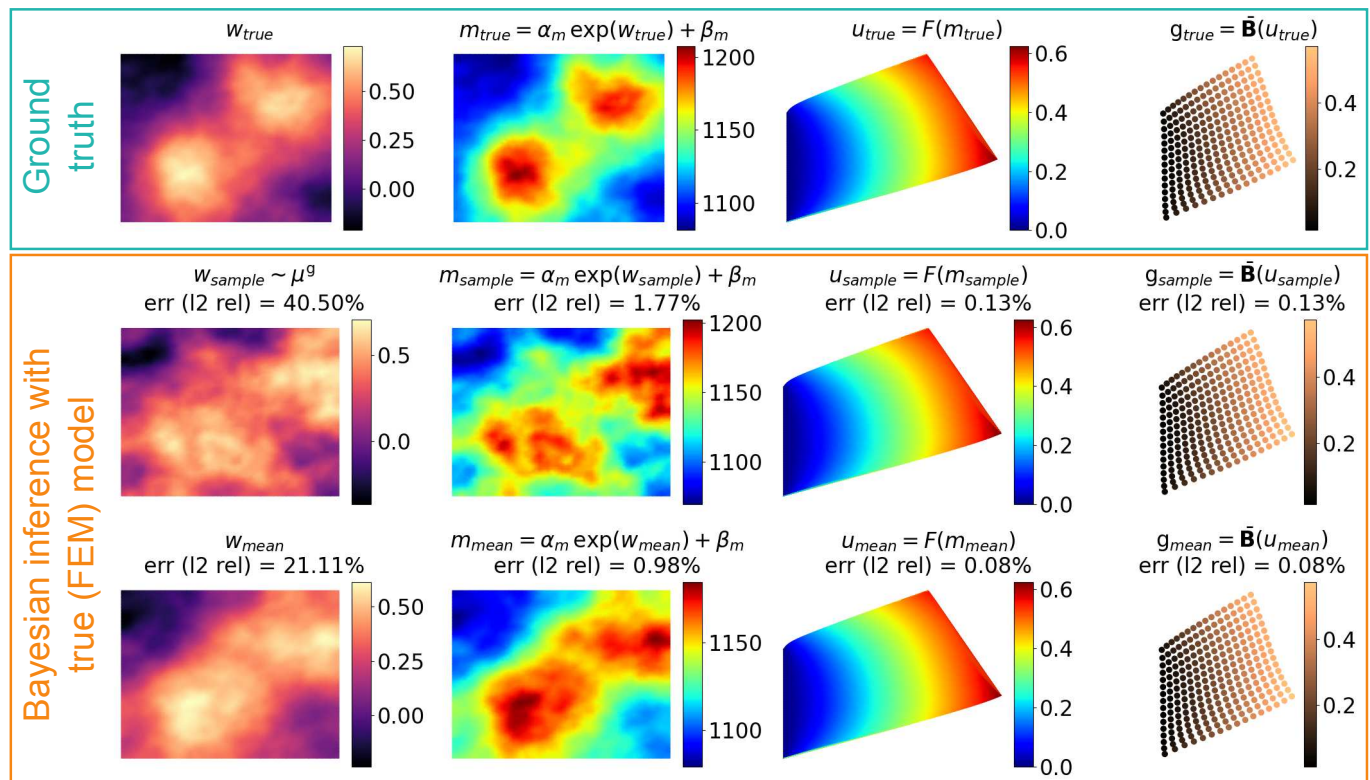


Figure A6. Bayesian inference of Young's modulus in the linear elasticity problem using the *true* forward model (numerical approximation of the PDE). The top panel displays the synthetic field w_{true} , the corresponding Young's modulus field $m_{true} = \alpha_m \exp(w_{true}) + \beta_m$, the displacement field $u_{true} = F(m_{true})$ (solution of the linear elasticity problem), and the observations $g_{true} = \bar{\mathbf{B}}(u_{true})$. The displacement field u_{true} is visualized by plotting the deformed configuration of the domain, with the color map indicating the magnitude $|u_{true}(x)|$. The observations g_{true} are obtained by interpolating u_{true} on a 16×16 grid over $D_U = (0, 1)^2$ and are visualized at the corresponding deformed observation locations $z_i = x_i + g_{true,i}$, with each point colored by the magnitude $|g_{true,i}|$ of the observed displacement vector. Thus, $g_{true} \in \mathbb{R}^{d_g}$ with $d_g = 2 \times 16^2$. The lower panel displays a posterior sample w_{sample} drawn from the posterior measure μ^g (one realization from the MCMC chain) and the posterior mean w_{mean} defined in (65). The remaining columns display the corresponding Young's modulus field m , the forward solution $u = F(m)$ used in the MCMC simulation, and the predicted observations $g = \bar{\mathbf{B}}(u)$ associated with the corresponding w .

Appendix B.3. Inference Results for the Hyperelasticity Problem

The MCMC parameters are the same as those used in the linear elasticity case. The acceptance rate and cost histories for the four MCMC simulations corresponding to the *true* and surrogate forward models are shown in Figure A8. The results obtained using the *true* forward model, including the posterior mean and a representative posterior sample, are shown in Figure A9. The results obtained using DeepONet, PCANet, and FNO surrogates are shown in Figure A10. The results are similar to the linear elasticity case, with the posterior means from the surrogate models closely matching the posterior mean obtained using the *true* forward model.

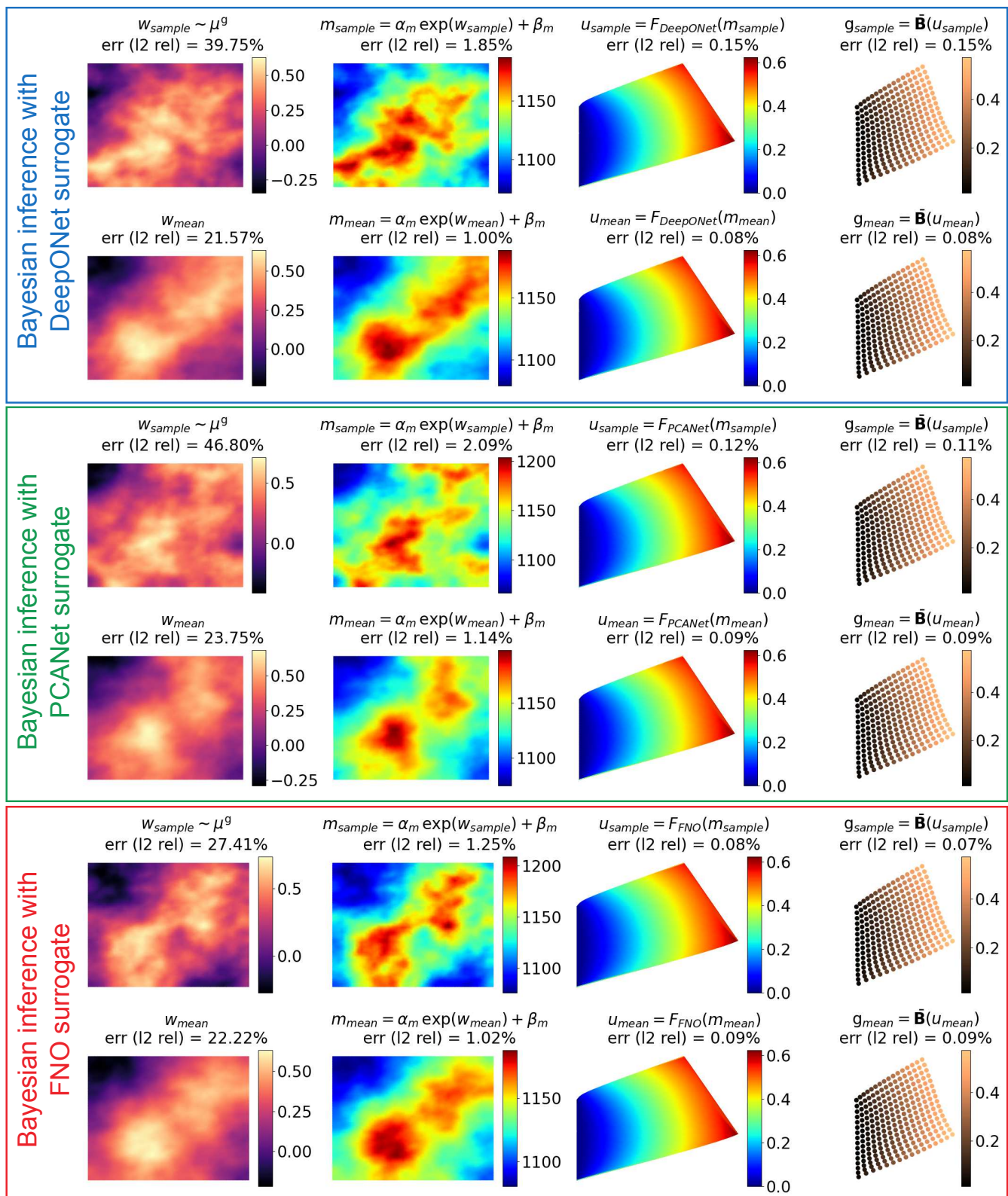


Figure A7. Comparison of Bayesian inference results for the linear elasticity problem using DeepONet, PCANet, and FNO surrogates. Each panel displays a posterior sample w_{sample} drawn from the posterior measure μ^g and the posterior mean w_{mean} . The corresponding Young's modulus field m , the forward solution $u = F(m)$, and predicted observations $g = \tilde{\mathbf{B}}(u)$ are also shown, where F_{NO} , $NO \in \{DeepONet, PCANet, FNO\}$ is the neural operator approximation of the forward operator used in the MCMC simulation and $\tilde{\mathbf{B}}$ is the state-to-observable map. These results should be compared with the inference results obtained using the *true* forward model in Figure A6. The visualizations of the displacement u and observable g are described in Figure A6.

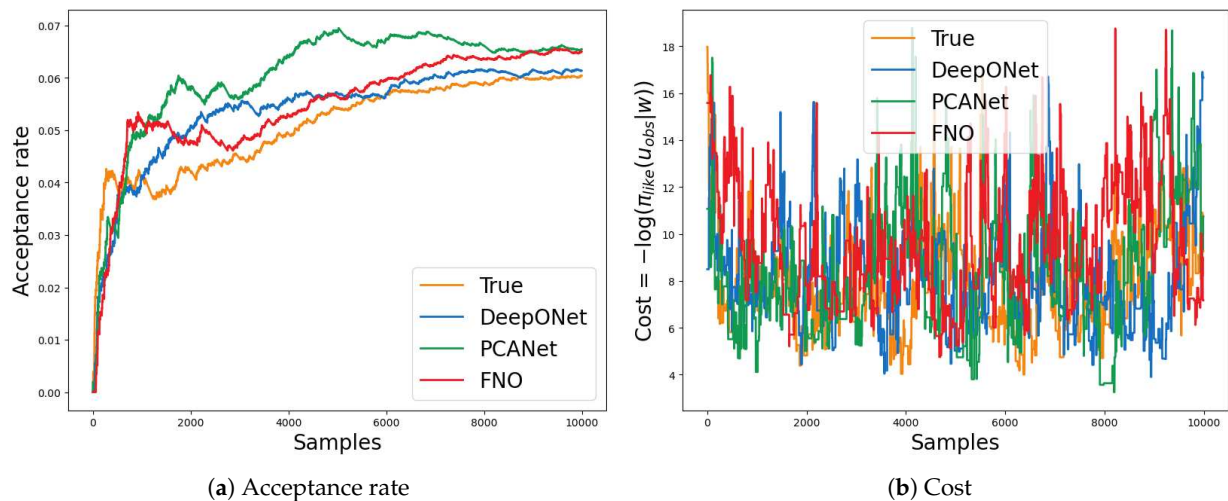


Figure A8. Acceptance rate and cost during MCMC simulation for the hyperelasticity problem.

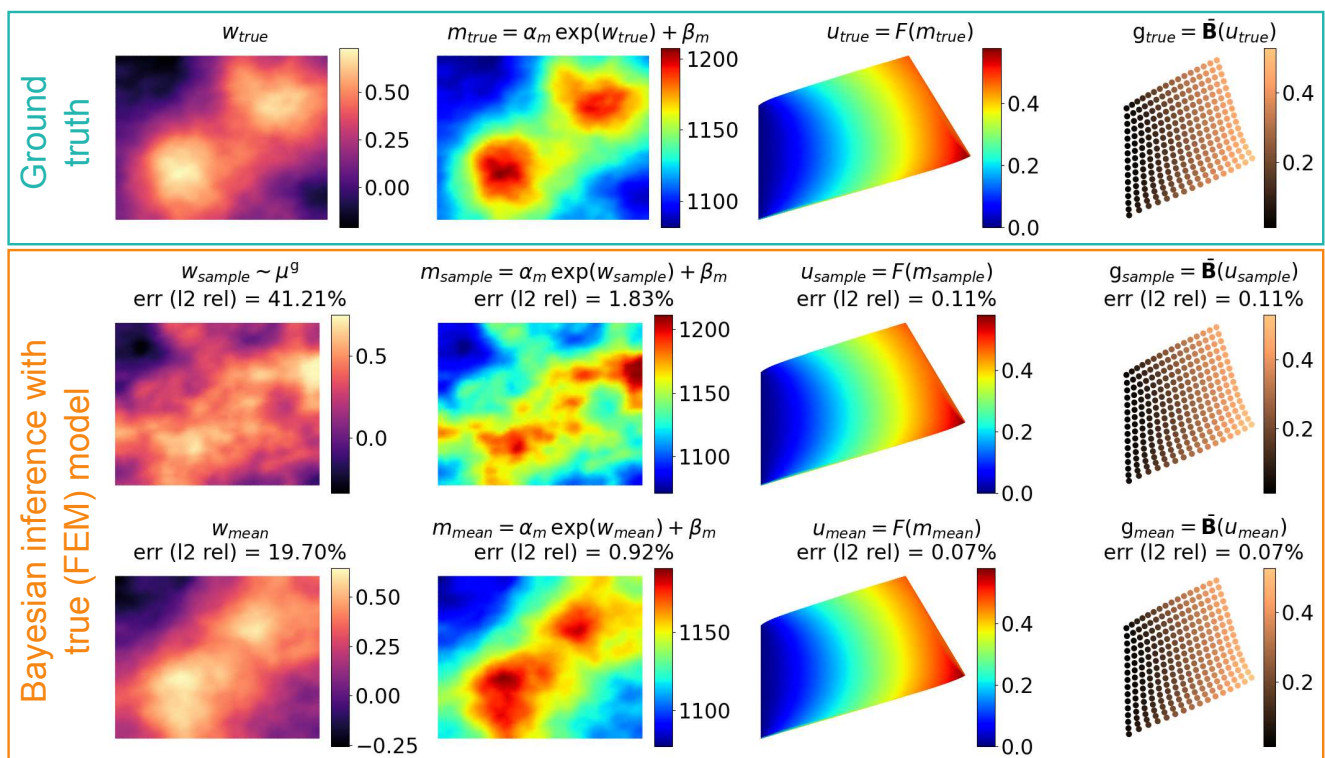


Figure A9. Bayesian inference of Young's modulus in the hyperelasticity problem using the *true* forward model (numerical approximation of the PDE). The top panel displays the synthetic field w_{true} , the corresponding Young's modulus field $m_{true} = \alpha_m \exp(w_{true}) + \beta_m$, the displacement field $u_{true} = F(m_{true})$ (solution of the hyperelasticity problem), and the observations $g_{true} = \tilde{\mathbf{B}}(u_{true})$. The observations are obtained by interpolating the displacement field u_{true} on a 16×16 grid over $D_U = (0, 1)^2$, so that $g_{true} \in \mathbb{R}^{d_g}$ with $d_g = 2 \times 16^2$. The lower panel displays a posterior sample w_{sample} drawn from the posterior measure μ^g (one realization from the MCMC chain) and the posterior mean w_{mean} defined in (65). The remaining columns display the corresponding Young's modulus field m , the forward solution $u = F(m)$ used in the MCMC simulation, and the predicted observations $g = \tilde{\mathbf{B}}(u)$ associated with the corresponding w . The visualizations of the displacement u and observable g are described in Figure A6.

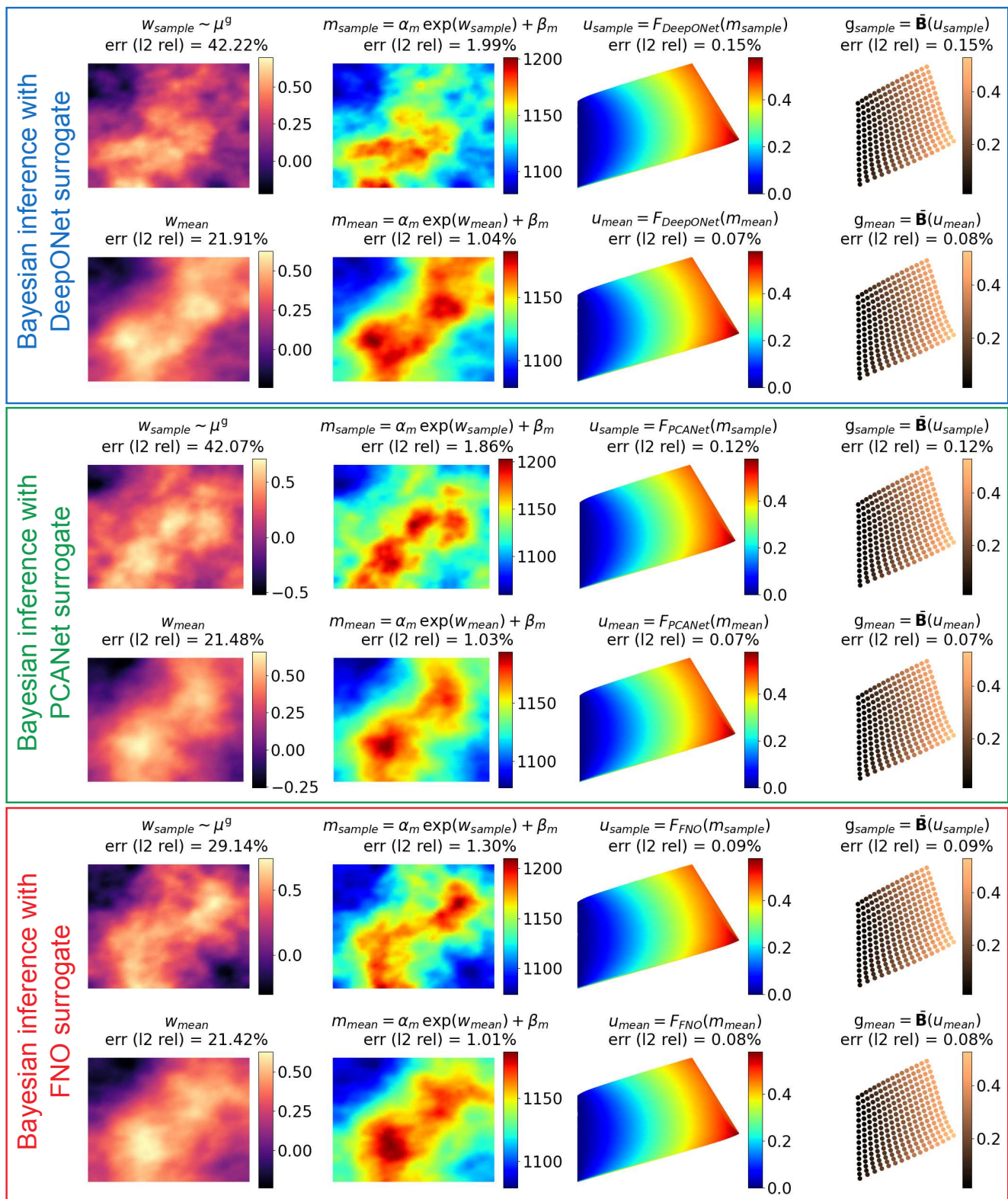


Figure A10. Comparison of Bayesian inference results for the hyperelasticity problem using DeepONet, PCANet, and FNO surrogates. Each panel displays a posterior sample w_{sample} drawn from the posterior measure μ^g and the posterior mean w_{mean} . The corresponding Young's modulus field m , the forward solution $u = F(m)$, and predicted observations $g = \tilde{\mathbf{B}}(u)$ are also shown, where F_{NOp} , $NOp \in \{DeepONet, PCANet, FNO\}$ is the neural operator approximation of the forward operator used in the MCMC simulation and $\tilde{\mathbf{B}}$ is the state-to-observable map. These results should be compared with the inference results obtained using the *true* forward model in Figure A9. The visualizations of the displacement u and observable g are described in Figure A6.

References

- Wang, S.; Wang, H.; Perdikaris, P. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Sci. Adv.* **2021**, *7*, eabi8605. [\[CrossRef\]](#) [\[PubMed\]](#)
- Lu, L.; Jin, P.; Pang, G.; Zhang, Z.; Karniadakis, G.E. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat. Mach. Intell.* **2021**, *3*, 218–229. [\[CrossRef\]](#)
- Bhattacharya, K.; Hosseini, B.; Kovachki, N.B.; Stuart, A.M. Model reduction and neural networks for parametric PDEs. *SMAI J. Comput. Math.* **2021**, *7*, 121–157. [\[CrossRef\]](#)
- Fresca, S.; Manzoni, A. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Comput. Methods Appl. Mech. Eng.* **2022**, *388*, 114181.
- Li, Z.; Kovachki, N.; Azizzadenesheli, K.; Liu, B.; Bhattacharya, K.; Stuart, A.; Anandkumar, A. Fourier neural operator for parametric partial differential equations. *arXiv* **2020**, arXiv:2010.08895.
- Kovachki, N.; Li, Z.; Liu, B.; Azizzadenesheli, K.; Bhattacharya, K.; Stuart, A.; Anandkumar, A. Neural operator: Learning maps between function spaces with applications to PDEs. *J. Mach. Learn. Res.* **2023**, *24*, 1–97.
- Azizzadenesheli, K.; Kovachki, N.; Li, Z.; Liu-Schiaffini, M.; Kossaifi, J.; Anandkumar, A. Neural operators for accelerating scientific simulations and design. *Nat. Rev. Phys.* **2024**, *6*, 320–328. [\[CrossRef\]](#)
- De Hoop, M.; Huang, D.Z.; Qian, E.; Stuart, A.M. The Cost-Accuracy Trade-Off in Operator Learning with Neural Networks. *arXiv* **2022**, arXiv:2203.13181.
- Jha, P.K. *CEADpx: Neural_Operators*, Version Survey 26_v2; Zenodo: Geneva, Switzerland, 2026. [\[CrossRef\]](#)
- Jha, P.K. Residual-based error corrector operator to enhance accuracy and reliability of neural operator surrogates of nonlinear variational boundary-value problems. *Comput. Methods Appl. Mech. Eng.* **2024**, *419*, 116595.
- Dashti, M.; Stuart, A.M., The Bayesian Approach to Inverse Problems. In *Handbook of Uncertainty Quantification*; Ghanem, R., Higdon, D., Owhadi, H., Eds.; Springer International Publishing: Cham, Switzerland, 2017; pp. 311–428. [\[CrossRef\]](#)
- Mandel, J. Introduction to infinite dimensional statistics and applications. *arXiv* **2023**, arXiv:2310.15818.
- Bui-Thanh, T.; Ghattas, O.; Martin, J.; Stadler, G. A computational framework for infinite-dimensional Bayesian inverse problems Part I: The linearized case, with application to global seismic inversion. *SIAM J. Sci. Comput.* **2013**, *35*, A2494–A2523. [\[CrossRef\]](#)
- Baratta, I.A.; Dean, J.P.; Dokken, J.S.; Habera, M.; HALE, J.; Richardson, C.N.; Rognes, M.E.; Scroggs, M.W.; Sime, N.; Wells, G.N. DOLFINx: The next generation FEniCS problem solving environment. *Zenodo* **2023**. [\[CrossRef\]](#)
- Korobeynikov, S.N.; Larichkin, A.Y.; Neff, P. Two types of compressible isotropic neo-Hookean material models. *arXiv* **2025**, arXiv:2506.22244.
- Jog, C.S. *Continuum Mechanics*; Cambridge University Press: Cambridge, UK, 2015; Volume 1.
- Lu, L.; Jin, P.; Karniadakis, G.E. DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. *arXiv* **2019**, arXiv:1910.03193.
- Stuart, A.M. Inverse problems: A Bayesian perspective. *Acta Numer.* **2010**, *19*, 451–559. [\[CrossRef\]](#)
- Cotter, S.L.; Roberts, G.O.; Stuart, A.M.; White, D. MCMC Methods for Functions: Modifying Old Algorithms to Make Them Faster. *Stat. Sci.* **2013**, *28*, 424–446. [\[CrossRef\]](#)
- Li, Z.; Zheng, H.; Kovachki, N.; Jin, D.; Chen, H.; Liu, B.; Azizzadenesheli, K.; Anandkumar, A. Physics-informed neural operator for learning partial differential equations. *ACM/JMS J. Data Sci.* **2024**, *1*, 1–27. [\[CrossRef\]](#)
- Goswami, S.; Bora, A.; Yu, Y.; Karniadakis, G.E. Physics-informed deep neural operator networks. In *Machine Learning in Modeling and Simulation: Methods and Applications*; Springer: Cham, Switzerland, 2023; pp. 219–254.
- O’Leary-Roseberry, T.; Chen, P.; Villa, U.; Ghattas, O. Derivative-informed neural operator: An efficient framework for high-dimensional parametric derivative learning. *J. Comput. Phys.* **2024**, *496*, 112555.
- Tripura, T.; Chakraborty, S. Wavelet neural operator: A neural operator for parametric partial differential equations. *arXiv* **2022**, arXiv:2205.02191.
- Cao, Q.; Goswami, S.; Karniadakis, G.E. Laplace neural operator for solving differential equations. *Nat. Mach. Intell.* **2024**, *6*, 631–640. [\[CrossRef\]](#)
- Hao, Z.; Wang, Z.; Su, H.; Ying, C.; Dong, Y.; Liu, S.; Cheng, Z.; Song, J.; Zhu, J. GNOT: A general neural operator transformer for operator learning. In *Proceedings of the International Conference on Machine Learning*; PMLR: Cambridge, MA, USA, 2023; pp. 12556–12569.
- Li, Z.; Meidani, K.; Farimani, A.B. Transformer for partial differential equations’ operator learning. *arXiv* **2022**, arXiv:2205.13671.
- Li, Z.; Shu, D.; Barati Farimani, A. Scalable transformer for pde surrogate modeling. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 28010–28039. [\[CrossRef\]](#)
- Rahman, M.A.; Ross, Z.E.; Azizzadenesheli, K. U-no: U-shaped neural operators. *arXiv* **2022**, arXiv:2204.11127.
- Tran, A.; Mathews, A.; Xie, L.; Ong, C.S. Factorized fourier neural operators. *arXiv* **2021**, arXiv:2111.13802.
- Gupta, G.; Xiao, X.; Bogdan, P. Multiwavelet-based operator learning for differential equations. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 24048–24062.

31. Li, H.; Ye, X.; Jiang, P.; Qin, G.; Wang, T. Local neural operator for solving transient partial differential equations on varied domains. *Comput. Methods Appl. Mech. Eng.* **2024**, *427*, 117062. [[CrossRef](#)]
32. Zhao, Z.; Liu, C.; Li, Y.; Chen, Z.; Liu, X. Diffeomorphism neural operator for various domains and parameters of partial differential equations. *Commun. Phys.* **2025**, *8*, 15. [[CrossRef](#)]
33. Vemparala, B.; Yang, M.; Soghrati, S. Deep learning-driven domain decomposition (DLD3): A generalizable AI-driven framework for structural analysis. *Comput. Methods Appl. Mech. Eng.* **2024**, *432*, 117446.
34. Liu, Z.; Wang, Y.; Vaidya, S.; Ruehle, F.; Halverson, J.; Soljačić, M.; Hou, T.Y.; Tegmark, M. KAN: Kolmogorov-arnold networks. *arXiv* **2024**, arXiv:2404.19756.
35. Psaros, A.F.; Meng, X.; Zou, Z.; Guo, L.; Karniadakis, G.E. Uncertainty quantification in scientific machine learning: Methods, metrics, and comparisons. *J. Comput. Phys.* **2023**, *477*, 111902. [[CrossRef](#)]
36. Jospin, L.V.; Laga, H.; Boussaid, F.; Buntine, W.; Bennamoun, M. Hands-on Bayesian neural networks—A tutorial for deep learning users. *IEEE Comput. Intell. Mag.* **2022**, *17*, 29–48.
37. Garg, S.; Chakraborty, S. Variational Bayes Deep Operator Network: A data-driven Bayesian solver for parametric differential equations. *arXiv* **2022**, arXiv:2206.05655.
38. Singh, P.K.; Farrell-Maupin, K.A.; Faghihi, D. A framework for strategic discovery of credible neural network surrogate models under uncertainty. *Comput. Methods Appl. Mech. Eng.* **2024**, *427*, 117061. [[CrossRef](#)]
39. Juarez, M.G.; Botti, V.J.; Giret, A.S. Digital twins: Review and challenges. *J. Comput. Inf. Sci. Eng.* **2021**, *21*, 030802. [[CrossRef](#)]
40. Wagg, D.; Worden, K.; Barthorpe, R.; Gardner, P. Digital twins: State-of-the-art and future directions for modeling and simulation in engineering dynamics applications. *ASCE-ASME J. Risk Uncertain. Eng. Syst. Part B Mech. Eng.* **2020**, *6*, 030901.
41. Kobayashi, K.; Alam, S.B. Deep neural operator-driven real-time inference to enable digital twin solutions for nuclear energy systems. *Sci. Rep.* **2024**, *14*, 2101. [[PubMed](#)]
42. Cao, L.; O’Leary-Roseberry, T.; Jha, P.K.; Oden, J.T.; Ghattas, O. Residual-based error correction for neural operator accelerated infinite-dimensional Bayesian inverse problems. *J. Comput. Phys.* **2023**, *486*, 112104.
43. Cao, L.; O’Leary-Roseberry, T.; Ghattas, O. Derivative-informed neural operator acceleration of geometric MCMC for infinite-dimensional Bayesian inverse problems. *arXiv* **2024**, arXiv:2403.08220.
44. Gao, Z.; Yan, L.; Zhou, T. Adaptive operator learning for infinite-dimensional Bayesian inverse problems. *SIAM/ASA J. Uncertain. Quantif.* **2024**, *12*, 1389–1423. [[CrossRef](#)]
45. Shukla, K.; Oommen, V.; Peyvan, A.; Penwarden, M.; Plewacki, N.; Bravo, L.; Ghoshal, A.; Kirby, R.M.; Karniadakis, G.E. Deep neural operators as accurate surrogates for shape optimization. *Eng. Appl. Artif. Intell.* **2024**, *129*, 107615.
46. Aldirany, Z.; Cottureau, R.; Laforest, M.; Prudhomme, S. Multi-level neural networks for accurate solutions of boundary-value problems. *Comput. Methods Appl. Mech. Eng.* **2024**, *419*, 116666.
47. Wang, Y.; Lai, C.Y. Multi-stage neural networks: Function approximator of machine precision. *J. Comput. Phys.* **2024**, *504*, 112865. [[CrossRef](#)]
48. Ainsworth, M.; Dong, J. Galerkin neural networks: A framework for approximating variational equations with error control. *SIAM J. Sci. Comput.* **2021**, *43*, A2474–A2501. [[CrossRef](#)]
49. Jha, P.K.; Oden, J.T. Goal-oriented a-posteriori estimation of model error as an aid to parameter estimation. *J. Comput. Phys.* **2022**, *470*, 111575. [[CrossRef](#)]
50. Huang, X.; Perdikaris, P. PhysicsCorrect: A Training-Free Approach for Stable Neural PDE Simulations. *Proc. AAAI Conf. Artif. Intell.* **2026**, *40*, 22057–22065. [[CrossRef](#)]
51. Lanthaler, S.; Mishra, S.; Karniadakis, G.E. Error estimates for DeepONets: A deep learning framework in infinite dimensions. *Trans. Math. Its Appl.* **2022**, *6*, tnac001. [[CrossRef](#)]
52. Kovachki, N.; Lanthaler, S.; Mishra, S. On universal approximation and error bounds for Fourier neural operators. *J. Mach. Learn. Res.* **2021**, *22*, 1–76.
53. Fanaskov, V.; Rudikov, A.; Oseledets, I. Neural functional a posteriori error estimates. *arXiv* **2024**, arXiv:2402.05585.
54. Lu, L.; Pestourie, R.; Johnson, S.G.; Romano, G. Multifidelity deep neural operators for efficient learning of partial differential equations with application to fast inverse design of nanoscale heat transport. *Phys. Rev. Res.* **2022**, *4*, 023210. [[CrossRef](#)]
55. Howard, A.A.; Perego, M.; Karniadakis, G.E.; Stinis, P. Multifidelity deep operator networks for data-driven and physics-informed problems. *J. Comput. Phys.* **2023**, *493*, 112462.
56. Xu, C.; Cao, B.T.; Yuan, Y.; Meschke, G. A multi-fidelity deep operator network (DeepONet) for fusing simulation and monitoring data: Application to real-time settlement prediction during tunnel construction. *Eng. Appl. Artif. Intell.* **2024**, *133*, 108156.
57. Takamoto, M.; Praditia, T.; Leiteritz, R.; MacKinlay, D.; Alesiani, F.; Pflüger, D.; Niepert, M. PDEBench: An extensive benchmark for scientific machine learning. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 1596–1611. [[CrossRef](#)]
58. Gupta, J.K.; Brandstetter, J. Towards multi-spatiotemporal-scale generalized pde modeling. *arXiv* **2022**, arXiv:2209.15616.
59. Benitez, J.A.L.; Furuya, T.; Faucher, F.; Kratsios, A.; Tricoche, X.; de Hoop, M.V. Out-of-distributional risk bounds for neural operators with applications to the Helmholtz equation. *J. Comput. Phys.* **2024**, *513*, 113168. [[CrossRef](#)]

60. Lanthaler, S. Operator learning with PCA-Net: Upper and lower complexity bounds. *J. Mach. Learn. Res.* **2023**, *24*, 1–67.
61. Lanthaler, S.; Stuart, A.M. The parametric complexity of operator learning. *IMA J. Numer. Anal.* **2026**, *46*, 647–712.
62. Kovachki, N.B.; Lanthaler, S.; Mhaskar, H. Data complexity estimates for operator learning. *arXiv* **2024**, arXiv:2405.15992.
63. Mukherjee, A.; Roy, A. Size lowerbounds for deep operator networks. *arXiv* **2023**, arXiv:2308.06338.
64. Liu, H.; Zhang, Z.; Liao, W.; Schaeffer, H. Neural scaling laws of deep ReLU and deep operator network: A theoretical study. *arXiv* **2024**, arXiv:2410.00357.
65. McCabe, M.; Blancard, B.R.-S.; Parker, L.H.; Ohana, R.; Cranmer, M.; Bietti, A.; Eickenberg, M.; Golkar, S.; Krawezik, G.; Lanusse, F.; et al. Multiple Physics Pretraining for Physical Surrogate Models. *arXiv* **2023**, arXiv:2310.02994.
66. Shen, J.; Marwah, T.; Talwalkar, A. UPS: Efficiently building foundation models for pde solving via cross-modal adaptation. *arXiv* **2024**, arXiv:2403.07187.
67. Alkin, B.; Fürst, A.; Schmid, S.; Gruber, L.; Holzleitner, M.; Brandstetter, J. Universal physics transformers: A framework for efficiently scaling neural operators. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 25152–25194. [[CrossRef](#)]
68. Hao, Z.; Su, C.; Liu, S.; Berner, J.; Ying, C.; Su, H.; Anandkumar, A.; Song, J.; Zhu, J. DPOT: Auto-regressive denoising operator transformer for large-scale pde pre-training. *arXiv* **2024**, arXiv:2403.03542.
69. Ye, Z.; Huang, X.; Chen, L.; Liu, H.; Wang, Z.; Dong, B. PDEformer: Towards a foundation model for one-dimensional partial differential equations. *arXiv* **2024**, arXiv:2402.12652.
70. Ye, Z.; Liu, Z.; Wu, B.; Jiang, H.; Chen, L.; Zhang, M.; Huang, X.; Zou, Q.M.; Liu, H.; Dong, B.; et al. PDEformer-2: A versatile foundation model for two-dimensional partial differential equations. *arXiv* **2025**, arXiv:2507.15409.
71. Chen, T.; Zhou, H.; Li, Y.; Wang, H.; Gao, C.; Shi, R.; Zhang, S.; Li, J. OmniArch: Building foundation model for scientific computing. *arXiv* **2024**, arXiv:2402.16014.
72. Ohana, R.; McCabe, M.; Meyer, L.; Morel, R.; Agocs, F.J.; Beneitez, M.; Berger, M.; Burkhart, B.; Dalziel, S.B.; Fielding, D.B.; et al. The Well: A large-scale collection of diverse physics simulations for machine learning. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 44989–45037. [[CrossRef](#)]
73. Zhong, W.; Meidani, H. Physics-informed geometry-aware neural operator. *Comput. Methods Appl. Mech. Eng.* **2025**, *434*, 117540.
74. Alkin, B.; Bleeker, M.; Kurler, R.; Kronlachner, T.; Sonleitner, R.; Dorfer, M.; Brandstetter, J. AB-UPT: Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers. *arXiv* **2025**, arXiv:2502.09692.
75. Masliaev, M.; Gusarov, D.; Markov, I.; Hvatov, A. Towards Universal Neural Operators through Multiphysics Pretraining. *arXiv* **2025**, arXiv:2511.10829.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.